

七、 实验七 多周期 CPU 实现

1 实验目的

1. 在单周期 CPU 实验完成的提前下，理解多周期的概念。
2. 熟悉并掌握多周期 CPU 的原理和设计。
3. 进一步提升运用 verilog 语言进行电路设计的能力。
4. 为后续实现流水线 cpu 的课程设计打下基础。

2 实验设备

1. 装有 Xilinx Vivado 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 实验任务

1. 本次实验是对单周期 CPU 实验的拔高，也是为流水线 CPU 打下基础。前期的实验准备同单周期 CPU 的实验，在单周期 CPU 中只要求实现了十几条指令，但此处要求扩展到 30 多条指令。

多周期 CPU 是指，一条指令需要花费多个周期才能完成所有操作，在每个周期内只做一部分操作，比如：取指、译码、执行、访存、写回，此时，一条指令执行完，共需 5 个周期，每个周期只做一部分操作。

将 CPU 划分为多周期的优势在于，每个时钟周期内 CPU 需要做的工作就变少，因此频率可以更高，且每个部件做的事情单一了，比如取指部件只负责从指令存储器中取出指令，因此 CPU 可以进行流水工作，也相当于一个时钟周期完成一条指令。频率更高，依然相当于是一个周期完成一条指令，因此 CPU 可以运行的更快。

本次实验就是将实验六所实现的单周期 CPU 划分为多周期的，并扩展指令到 30 多条。

2. 依据单周期实验的设计框图，将其划分为多个功能块，每个功能块占用一个周期完成，即每个功能块从上一个功能块获取信息，作相关动作，完成后将结果锁存到寄存器中，作为下一个功能块的输入。建议划分为教课书上的 5 个功能块：取指、译码、执行、访存、写回，将理论与实践相结合。

3. 画出划分后的多周期 CPU 的框图，大致框图如图 8.1。从图中可以看出指令每个周期走完一个功能块，进入下一个功能块。标注的 clk 箭头是去往相邻模块的中间锁存器，是因为每个模块的输出需要锁存到寄存器中，下一个模块会从该寄存器中读出数据作为自己的输入，寄存器的锁存是需要时钟控制的。值得注意的是，写回模块所做的就是从访存模块获取要写入寄存器堆的数据和目的寄存器，送往寄存器堆，其所需的 clk 信号是最终发生在寄存器堆的写操作上的。自己设计的框图中要求力求精细，可参考教科书上的 5 级流水的框图。

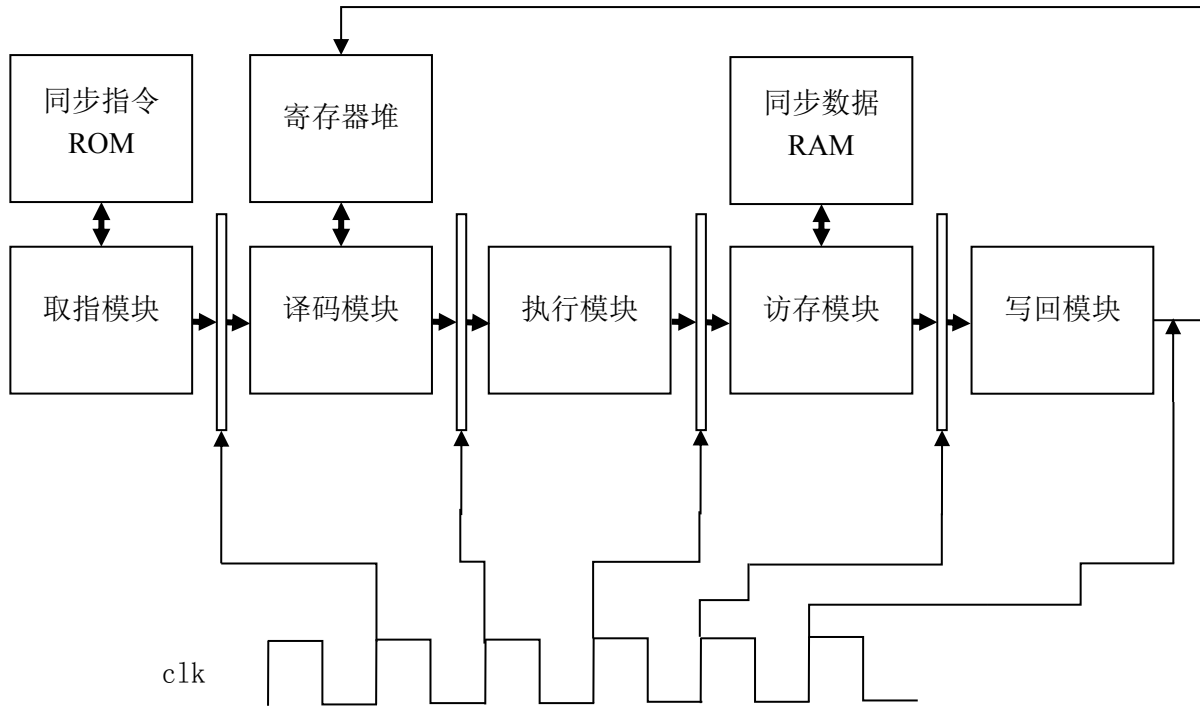


图 8.1 多周期 CPU 的大致框图

4. 本次实验是需要用到之前实验的结果的，比如 ALU 模块、寄存器堆模块、指令 ROM 模块和数据 RAM 模块，其中 ROM 和 RAM 建议使用调用库 IP 实例化的同步存储器，因为存储器在实际应用中基本都是同步读写的，为了更贴近真实情况，此处建议使用同步 RAM 和 ROM。

5. 在实验五中生成的同步 RAM 和 ROM，都是在发送地址后的下一拍才能获得对应数据的。故在在使用同步存储器时，从指令和数据存储器中读取数据就需要等待一拍时钟了，即取指令需要两拍时间，load 操作也需要两拍时间。在真实的处理器系统中，取指令和访存其实都是需要多拍时钟的。

6. 本次实验，同样需要完成表 8.1 和表 8.2 的填写。

表 8.1 mips 基础指令特性归纳表

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	目的寄存器	功能描述
R 型指令	addu rd, rs, rt	000000 rs rt rd 00000 100001	[rs]	[rt]	rd	GPR[rd]=GPR[rs]+GPR[rt]
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)	rt	GPR[rt]=GPR[rs]+sign_ext(imm)
J 型指令	j target	000010 target	PC	target		跳转, PC={PC[31:28],target,2'b00}

表 8.2 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1, \$0, #1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001

7. 根据设计的实验方案，使用 verilog 编写相应代码。
8. 对编写的代码进行仿真，得到正确的波形图。
9. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 8.2。外围模块中需调用封装好的 LCD 触摸屏模块，观察多周期 CPU 的内部状态，比如 32 个寄存器的值，各模块 PC 的值等。并且需要利用触摸功能输入特定数据 RAM 地址，从该 RAM 的调试端口读出数据显示在屏上，以达到实时观察数据存储器内部数据变化的效果。通过这些手段，可以在板上充分验证 CPU 的正确性。

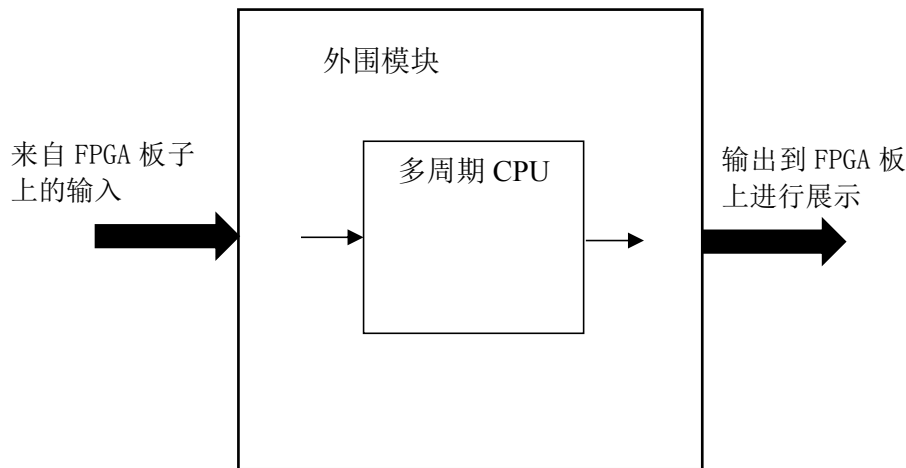


图 8.2 多周期 CPU 设计实验的顶层模块大致框图

10. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

注意：

①：MIPS 架构中有延迟槽的设定，其本意是加快流水 CPU 的执行速度，故在多周期 CPU 中该设定无意义，反而带来了 CPU 实现上的麻烦，故建议在多周期 CPU 中不考虑延迟槽技术。

②：MIPS 架构中分支和跳转指令参与计算的 PC 值均为延迟槽指令对应的 PC (即分支跳转指令的 PC+4)，而由于多周期不考虑延迟槽，故在实验中分支跳转指令参与计算使用本指令的 PC 值，故跳转的偏移量设置需要注意下。比如一条指令

“beq, r0, r0, #2” 在不考虑延迟槽的多周期 CPU 中，其跳转的目标地址为 beq 指令后

面的第 2 条。而在考虑延迟槽的流水 CPU 中，其跳转的目标地址为 beq 指令后面的第 3 条（即延迟槽指令后面的第 2 条）。这里需要理解清楚。

③：一般而言控制 CPU 运转的时钟是由 FPGA 板上的时钟输出提供的，但为了方便演示，我们需要在每一个时钟里查看一条指令的运算结果，故演示时理想的时钟是手动输入的，可以使用 FPGA 板上的脉冲开关代替时钟。

4 实验要求

1. 做好预习：

- 1) 复习单周期 CPU 的实验内容，归纳常用的 MIPS 指令，确定自己准备实现的 MIPS 指令，对其进行分析，完成表 8.1 的填写；
- 2) 依据自己设计中实现的指令，编写一段不少于 40 行的汇编程序，要求包含所有实现的指令，完成表 8.2 的填写；
- 3) 认真学习多周期的概念，了解流水线的概念，明白划分为多周期的意义；
- 4) 认真学习 CPU 各模块的功能，确认模块的划分。设计本次实验的方案，画出实验方案的设计框图，即补充完善图 8.1；
- 5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 8.2。

2. 实验实施：

- 1) 确认多周期 CPU 的设计框图的正确性；
- 2) 编写 verilog 代码，将表 8.2 中自己编写的汇编程序翻译为二进制，以 coe 文件的方式初始化到指令 ROM 中；
- 3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料，在仿真时需要将生成指令 ROM 时产生的 .mif 文件拷贝到工程目录下，才能仿真成功；
- 4) 完成调用多周期 CPU 的外围模块的设计，并编写代码；
- 5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

- 1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先解读表 8.2 中自己编写的汇编程序，然后采用手动输入时钟，每个周期查看 CPU 状态，按照检查人员的要求进行演示，检查指令运行结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

- 1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

5.1 实验原理

多周期 CPU 实现的指令集是在单周期设计的基础上增加 20 条算术逻辑运算指令、数据传送指令和控制指令，共实现了 36 条指令。表 8.3 为多周期 CPU 新增的指令。表 8.4 为多周期 CPU 所实现的指令的解析。

表 8.3 多周期 CPU 新增的指令

指令名称	汇编指令	功能描述
无符号小于置位	sltu rd,rs,rt	$GPR[rd] = (zero(GPR[rs]) < zero(GPR[rt]))$
跳转和链接寄存器	jalr rs	$GPR[31] = PC, PC = GPR[rs]$
跳转寄存器	jr rs	$PC = GPR[rs]$
向量逻辑左移字	sllv rd,rt,rs	$GPR[rd] = zero(GPR[rt]) << (GPR[rs] \% 32)$
算术右移字	sra rd,rt,shf	$GPR[rd] = sign(GPR[rt]) >> shf$
向量算术右移字	srav rd,rt,rs	$GPR[rd] = sign(GPR[rt]) >> (GPR[rs] \% 32)$
向量逻辑右移字	srlv rd,rt,rs	$GPR[rd] = zero(GPR[rt]) >> GPR[rs]$
立即数小于置位	slti rt,rs,imm	$GPR[rt] = (sign(GPR[rs]) < sign_ext(imm))$
立即数无符号小于置位	sltiu rt,rs,imm	$GPR[rt] = (zero(GPR[rs]) < sign_ext(imm))$
大于或等于零跳转	bgez rs,offset	if $GPR[rs] \geq 0$ then $PC = PC + sign_ext(offset) << 2$
大于零跳转	bgtz rs,offset	if $GPR[rs] > 0$ then $PC = PC + sign_ext(offset) << 2$
小于或等于零跳转	blez rs,offset	if $GPR[rs] \leq 0$ then $PC = PC + sign_ext(offset) << 2$
小于零跳转	bltz rs,offset	if $GPR[rs] < 0$ then $PC = PC + sign_ext(offset) << 2$
装载字节	lb rt,offset(base)	$GPR[rt] = sign(Mem[GPR[base] + sign_ext(offset)])$
无符号装载字节	lbu rt,offset(base)	$GPR[rt] = zero(Mem[GPR[base] + sign_ext(offset)])$
存储字节	sb rt,offset(base)	$Mem[GPR[base] + sign_ext(offset)] = GPR[rt]$
立即数按位与	andi rt,rs,imm	$GPR[rt] = GPR[rs] \& zero_ext(imm)$
立即数按位或	ori rt,rs,imm	$GPR[rt] = GPR[rs] zero_ext(imm)$
立即数按位异或	xori rt,rs,imm	$GPR[rt] = GPR[rs] \wedge zero_ext(imm)$
跳转和链接	jal target	$GPR[31] = PC, PC = \{PC[31:28], target << 2\}$

表 8.4 多周期 CPU 实现的 mips 指令特性归纳

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	addu rd,rs,rt	000000 rs rt rd 0000 0 100001	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] + GPR[rt]$
	subu rd,rs,rt	000000 rs rt rd 0000 0 100011	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] - GPR[rt]$
	slt rd,rs,rt	000000 rs rt rd 0000 0 101010	[rs]	[rt]		rd	$GPR[rd] = (sign(GPR[rs]) < sign(GPR[rt]))$
	sltu rd,rs,rt	000000 rs rt rd 0000 0 101011	[rs]	[rt]		rd	$GPR[rd] = (zero(GPR[rs]) < zero(GPR[rt]))$
	jalr rs	000000 rs 00000 111 11 00000 001001	[rs]			31	$GPR[31] = PC, PC = GPR[rs]$
	jr rs	000000 rs 00000000 00 00000 001000	[rs]				$PC = GPR[rs]$

表 8.4 多周期 CPU 实现的 mips 指令特性归纳 (续)

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	and rd,rs,rt	000000 rs rt rd 00000 100100	[rs]	[rt]		rd	GPR[rd]=GPR[rs]&GPR[rt]
	nor rd,rs,rt	000000 rs rt rd 00000 100111	[rs]	[rt]		rd	GPR[rd]=~(GPR[rs] GPR[rt])
	or rd,rs,rt	000000 rs rt rd 00000 100101	[rs]	[rt]		rd	GPR[rd]=GPR[rs] GPR[rt]
	xor rd,rs,rt	000000 rs rt rd 00000 100110	[rs]	[rt]		rd	GPR[rd]=GPR[rs]^GPR[rt]
	sll rd,rt,shf	000000 00000 rt rd shf 000000		[rt]		rd	GPR[rd]=zero(GPR[rt])<<shf
	sllv rd,rt,rs	000000 rs rt rd 00000 000100	[rs]	[rt]		rd	GPR[rd]=zero(GPR[rt])<<(GPR[rs]%32)
	sra rd,rt,shf	000000 00000 rt rd shf 000011		[rt]		rd	GPR[rd]=sign(GPR[rt])>>shf
	srav rd,rt,rs	000000 rs rt rd 00000 000111	[rs]	[rt]		rd	GPR[rd]=sign(GPR[rt])>>(GPR[rs]%32)
	srl rd,rt,shf	000000 00000 rt rd shf 000010		[rt]		rd	GPR[rd]=zero(GPR[rt])>>shf
	srlv rd,rt,rs	000000 rs rt rd 00000 000110	[rs]	[rt]		rd	GPR[rd]=zero(GPR[rt])>>GPR[rs]
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)		rt	GPR[rt]=GPR[rs]+ sign_ext(imm)
	slti rt,rs,imm	001010 rs rt imm	[rs]	sign_ext(imm)		rt	GPR[rt]=(sign(GPR[rs])< sign_ext(imm))
	sltiu rt,rs,imm	001011 rs rt imm	[rs]	sign_ext(imm)		rt	GPR[rt]=(zero(GPR[rs])< sign_ext(imm))
	beq rs,rt,offset	000100 rs rt offset	[rs]	[rt]			if GPR[rs]=GPR[rt] then PC=PC+sign_ext(offset)<<2
	bgez rs,offset	000001 rs 00001 offset	[rs]				if GPR[rs]≥0 then PC=PC+sign_ext(offset)<<2
	bgtz rs,offset	000111 rs 00000 offset	[rs]				if GPR[rs]>0 then PC=PC+sign_ext(offset)<<2
	blez rs,offset	000110 rs 00000 offset	[rs]				if GPR[rs]≤0 then PC=PC+sign_ext(offset)<<2
	bltz rs,offset	000001 rs 00000 offset	[rs]				if GPR[rs]<0 then PC=PC+sign_ext(offset)<<2
	bne rs,rt,offset	000101 rs rt offset	[rs]	[rt]			if GPR[rs]≠GPR[rt] then PC=PC+sign_ext(offset)<<2
	lw rt,offset(b)	100011 b rt offset	[b]	sign_ext(offset)		rt	GPR[rt]=Mem[GPR[b]+sign_ext(offset)]
	sw rt,offset(b)	101011 b rt offset	[b]	sign_ext(offset)	[rt]		Mem[GPR[b]+sign_ext(offset)]=GPR[rt]
	lb rt,offset(b)	100000 b rt offset	[b]	sign_ext(offset)		rt	GPR[rt]=sign(Mem[GPR[b]+sign_ext(offset)])
	lbu rt,offset(b)	100100 b rt offset	[b]	sign_ext(offset)		rt	GPR[rt]=zero(Mem[GPR[b]+sign_ext(offset)])
	sb rt,offset(b)	101000 b rt offset	[b]	sign_ext(offset)	[rt]		Mem[GPR[b]+sign_ext(offset)]=GPR[rt]

andi rt,rs,imm	001100 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]&zero_ext (imm)
lui rt,imm	001111 00000 rt imm		{imm, 16'd0}		rt	GPR[rt]= {imm, 16'd0}
ori rt,rs,imm	001101 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs] zero_ext (imm)

表 8.4 多周期 CPU 实现的 mips 指令特性归纳 (续)

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
I 型指令	xori rt,rs,imm	001110 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]^zero_ext (imm)
J 型指令	j target	000010 target					PC={PC[31:28],target<<2}
	jal target	000011 target					GPR[31]=PC,PC={PC[31:28],target<<2}

多周期 CPU 设计在单周期 CPU 基础上，主要做两部分改进。第一部分是控制单元，增加控制电路使每一个时钟只有一个阶段的电路产生的结果有效，并锁存上一阶段的结果用于后续阶段的运行；第二部分是数据通路，增加实现新增指令的电路。

第一部分的改进主要是增加状态机控制及增加各阶段之间的用于锁存的寄存器。由于有 5 个阶段，状态机共有 6 个状态：空闲 (IDLE)、取指 (FETCH)、译码 (DECODE)、执行 (EXE)、访存 (MEM)、写回 (WB)，如下图：

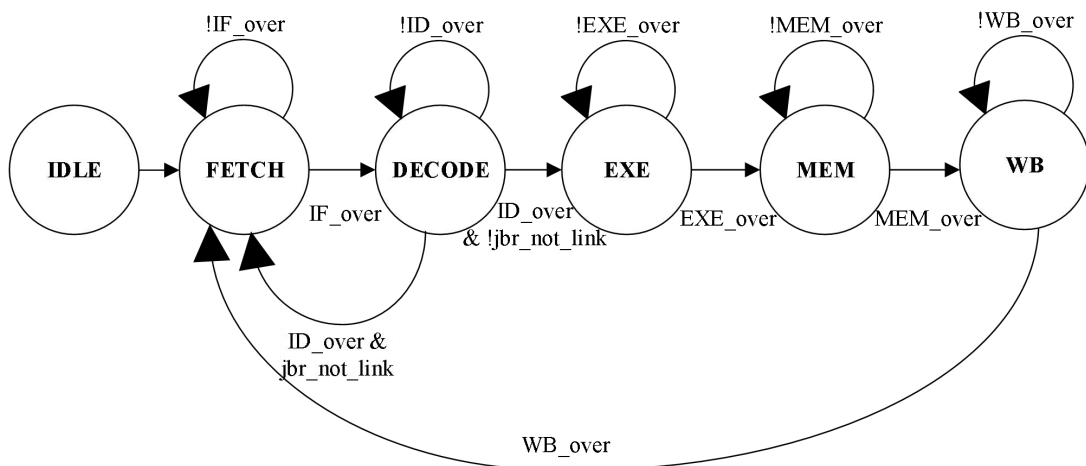


图 8.3 多周期 CPU 的状态机

空闲 (IDLE) 状态：CPU 在复位时，所有阶段电路都无效，CPU 等待复位结束开始下一状态——取指。

取指 (FETCH) 状态：在状态机进入取指状态的同时，PC 更新为下一 PC 值。故取指状态下，将 PC 值作为指令存储器的地址去取指令。由于同步指令存储器在下一时钟周期返回指令，因此取指需要两个时钟周期的时间。当取指结束后锁存取指阶段产生的结果——当前 PC 值和指令。

译码 (DECODE) 状态：类似于单周期 CPU 的译码阶段，主要完成指令译码、读寄存器、判断跳转等。控制单元区分各条指令并产生用于译码、执行、访存、写回的控制信号。当译码结束后锁存译码阶段产生的结果用于下一状态执行：分别用于执行、

访存、写回的控制信号、用于执行阶段的两个源操作数、用于访存阶段的写入内存数据、用于写回阶段的写寄存器地址。

执行（EXE）状态：ALU 模块完成操作。当执行结束后锁存执行阶段产生的结果及前级传递的结果：用于访存、写回的控制信号、ALU 结果、内存写入数据、寄存器写地址。

访存（MEM）状态：完成对数据存储器的读或写，并选择出将要写回寄存器的值。当访存结束后锁存访存阶段产生的结果及前级传递的结果：用于写回的控制信号、写回数据、写回地址。

写回（WB）状态：完成寄存器写入。

CPU 复位结束，状态机由 IDLE 进入取指状态，其后在每次上一级结束信号有效的时进入下一状态，写回级结束后返回取指级。当然也有例外，当指令是跳转而非链接跳转指令时，在译码状态后直接返回取下一指令不需要经过执行等后续阶段。

多周期 CPU 的实现框图如下：

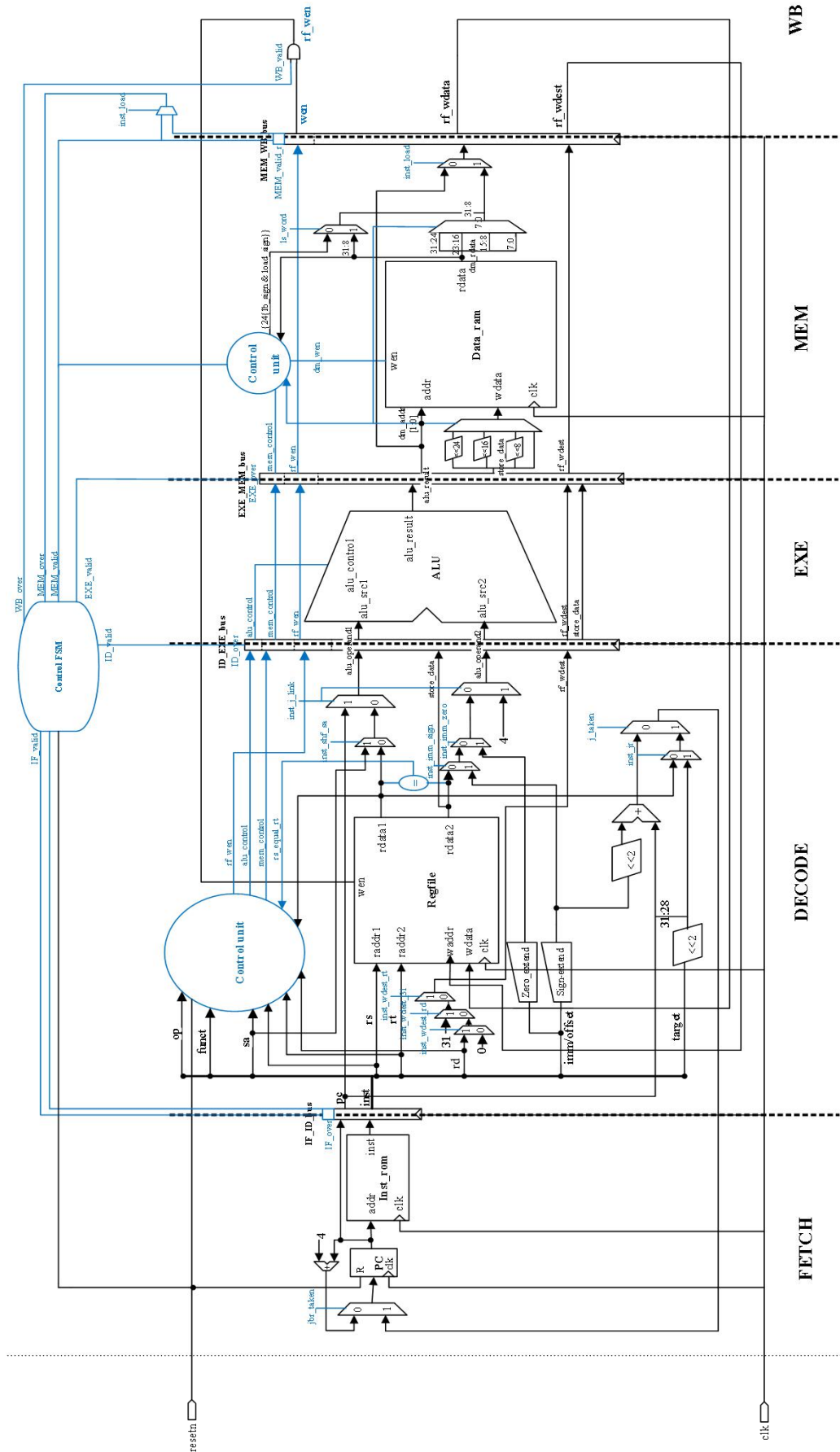


图 8.4 多周期 CPU 的实现框图

利用多周期 CPU 实现的 36 条指令，可以编写一段汇编程序，用于测试实现的多周期 CPU 的正确性。表 8.5 给出了一个测试程序。

表 8.5 多周期 CPU 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1, \$0,#1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001
04H	sll \$2, \$1,#4	[\$2] = 0000_0010H	00011100	0000_0000_0000_0001_0001_0001_0000_0000
08H	addu \$3, \$2,\$1	[\$3] = 0000_0011H	00411821	0000_0000_0100_0001_0001_1000_0010_0001
0CH	srl \$4, \$2,#2	[\$4] = 0000_0004H	00022082	0000_0000_0000_0010_0010_0000_1000_0010
10H	slti \$25,\$4,#5	[\$25] = 0000_0001H	28990005	0010_1000_1001_1001_0000_0000_0000_0101
14H	bgez \$25,#16	跳转到 54H	07210010	0000_0111_0010_0001_0000_0000_0001_0000
18H	subu \$5, \$3,\$4	[\$5] = 0000_000DH	00642823	0000_0000_0110_0100_0010_1000_0010_0011
1CH	sw \$5, #20(\$0)	Mem[0000_0014H] = 0000_000DH	AC050014	1010_1100_0000_0101_0000_0000_0001_0100
20H	nor \$6, \$5,\$2	[\$6] = FFFF_FFE2H	00A23027	0000_0000_1010_0010_0011_0000_0010_0111
24H	or \$7, \$6,\$3	[\$7] = FFFF_FFF3H	00C33825	0000_0000_1100_0011_0011_1000_0010_0101
28H	xor \$8, \$7,\$6	[\$8] = 0000_0011H	00E64026	0000_0000_1110_0110_0100_0000_0010_0110
2CH	sw \$8, #28(\$0)	Mem[0000_001CH] = 0000_0011H	AC08001C	1010_1100_0000_1000_0000_0000_0001_1100
30H	beq \$8, \$3,#2	跳转到 38H	11030002	0001_0001_0000_0011_0000_0000_0000_0010
34H	slt \$9, \$6,\$7	不执行	00C7482A	0000_0000_1100_0111_0100_1000_0010_1010
38H	addiu \$1, \$0,#8	[\$1] = 0000_0008H	24010008	0010_0100_0000_0001_0000_0000_0000_1000
3CH	lw \$10,#20(\$1)	[\$10] = 0000_0011H	8C2A0014	1000_1100_0010_1010_0000_0000_0001_0100
40H	bne \$10,\$5,#4	跳转到 50H	15450004	0001_0101_0100_0101_0000_0000_0000_0100
44H	and \$11,\$2,\$1	不执行	00415824	0000_0000_0100_0001_0101_1000_0010_0100
48H	sw \$11,#28(\$1)	不执行	AC2B001C	1010_1100_0010_1011_0000_0000_0001_1100
4CH	sw \$4, #16(\$1)	不执行	AC240010	1010_1100_0010_0100_0000_0000_0001_0000
50H	jal #25	跳转到 64H, [\$31] = 0000_0054H	0C000019	0000_1100_0000_0000_0000_0000_0001_1001
54H	lui \$12,#12	[\$12] = 000C_0000H	3C0C000C	0011_1100_0000_1100_0000_0000_0000_1100
58H	sra \$26,\$12,\$2	[\$26] = 0000_000CH	004CD007	0000_0000_0100_1100_1101_0000_0000_0111
5CH	sllv \$27,\$26,\$1	[\$27] = 0000_0018H	003AD804	0000_0000_0011_1010_1101_1000_0000_0100
60H	jalr \$27	跳转到 18H, [\$31] = 0000_0064H	0360F809	0000_0011_0110_0000_1111_1000_0000_1001
64H	sb \$26,#5(\$3)	MEM[0000_0016H] = 000C_000DH	A07A0005	1010_0000_0111_1010_0000_0000_0000_0101
68H	sltu \$13,\$3,\$3	[\$13] = 0000_0000H	0063682B	0000_0000_0110_0011_0110_1000_0010_1011
6CH	bgtz \$13,#3	不跳转	1DA00003	0001_1101_1010_0000_0000_0000_0000_0011
70H	sllv \$14,\$6,\$4	[\$14] = FFFF_FE20H	00867004	0000_0000_1000_0110_0111_0000_0000_0100
74H	sra \$15,\$14,#2	[\$15] = FFFF_FF88H	000E7883	0000_0000_0000_1110_0111_1000_1000_0011
78H	srlv \$16,\$15,\$1	[\$16] = 00FF_FFFFH	002F8006	0000_0000_0010_1111_1000_0000_0000_0110
7CH	blez \$16,#8	不跳转	1A000008	0001_1010_0000_0000_0000_0000_0000_1000
80H	sra \$16,\$15,\$1	[\$16] = FFFF_FFFFH	002F8007	0000_0000_0010_1111_1000_0000_0000_0111

表 8.5 多周期 CPU 测试所用汇编程序详述 (续)

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
84H	addiu \$11,\$0,#140	[\$11] = 0000_008CH	240B008C	0010_0100_0000_1011_0000_0000_1000_1100
88H	bltz \$16, #6	跳转到 A0H	06000006	0000_0110_0000_0000_0000_0000_0000_0110
8CH	lw \$28,#3(\$10)	[\$28] = 000C_000DH /000C 880DH	8D5C0003	1000_1101_0101_1100_0000_0000_0000_0011
90H	bne \$28,\$29,#7	不跳转/跳转 ACH	179D0007	0001_0111_1001_1101_0000_0000_0000_0111
94H	sb \$15,#8(\$5)	Mem[0000_0015H] = 0000_0088H	A0AF0008	1010_0000_1010_1111_0000_0000_0000_1000
98H	lb \$18,#8(\$5)	[\$18] =FFFF_FF88H	80B20008	1000_0000_1011_0010_0000_0000_0000_1000
9CH	lbu \$19,#8(\$5)	[\$19] = 0000_0088H	90B30008	1001_0000_1011_0011_0000_0000_0000_1000
A0H	sltiu \$24,\$15,#0xFFFF	[\$24] = 0000_0001H	2DF8FFFF	0010_1101_1111_1000_1111_1111_1111_1111
A4H	or \$29,\$12,\$5	[\$29] = 000C000DH	0185E825	0000_0001_1000_0101_1110_1000_0010_0101
A8H	jr \$11	跳转指令 8CH	01600008	0000_0001_0110_0000_0000_0000_0000_1000
ACH	andi \$20,\$15,#0xFFFF	[\$20] = 0000_FF88H	31F4FFFF	0011_0001_1111_0100_1111_1111_1111_1111
B0H	ori \$21,\$15,#0xFFFF	[\$21] =FFFF_FFFFH	35F5FFFF	0011_0101_1111_0101_1111_1111_1111_1111
B4H	xori \$22,\$15,#0xFFFF	[\$22] = FFFF_0077H	39F6FFFF	0011_1001_1111_0110_1111_1111_1111_1111
B8H	j #00H	跳转指令 00H	08000000	0000_1000_0000_0000_0000_0000_0000_0000

5.2 实验顶层模块框图

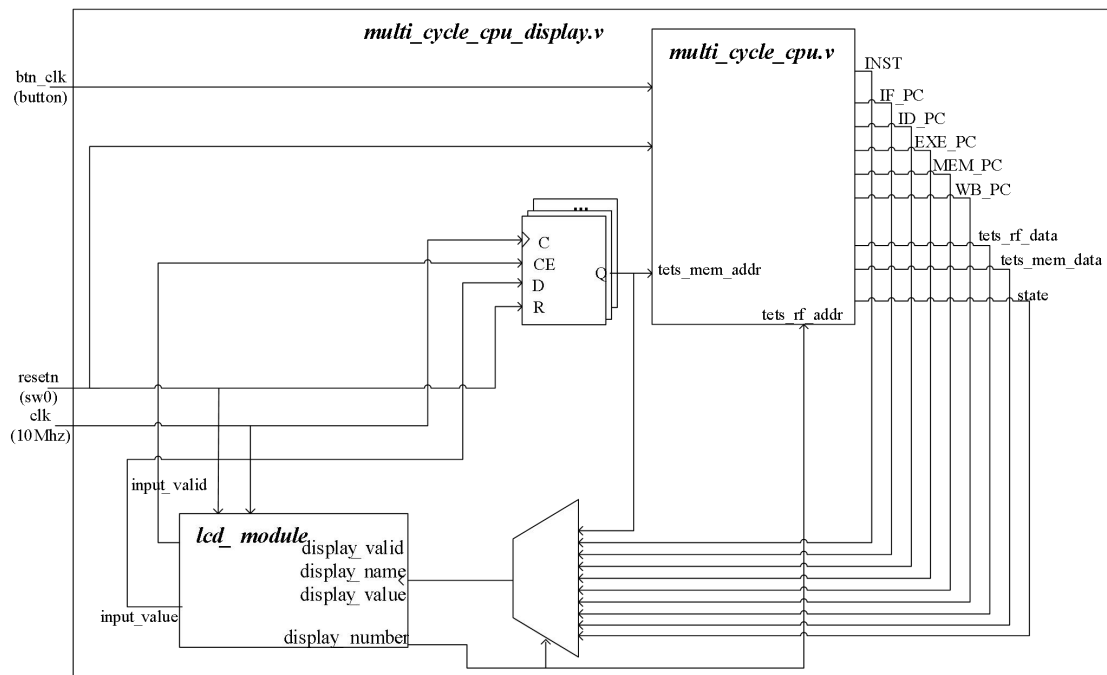


图 8.5 多周期 CPU 参考设计的顶层模块框图

5.3 源代码

multi_cycle_cpu_display.v 源代码:

```
1 `timescale 1ns / 1ps
//*****
2 *
```

```
3 // > 文件名: multi_cycle_cpu_display.v
4 // > 描述 :多周期 CPU 显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5 // > 作者 : LOONGSON
6 // > 日期 : 2016-04-14
7 //*****
8 module multi_cycle_cpu_display( // 多周期 cpu
9     //时钟与复位信号
10     input clk,
11     input resetn, //后缀"n"代表低电平有效
12
13     //脉冲开关, 用于产生脉冲 clk, 实现单步执行
14     input btn_clk,
15
16     //触摸屏相关接口, 不需要更改
17     output lcd_rst,
18     output lcd_cs,
19     output lcd_rs,
20     output lcd_wr,
21     output lcd_rd,
22     inout[15:0] lcd_data_io,
23     output lcd_bl_ctr,
24     inout ct_int,
25     inout ct_sda,
26     output ct_scl,
27     output ct_rstn
28 );
29 //-----{时钟和复位信号}begin
30 //不需要更改, 用于单步调试
31     wire cpu_clk; //单周期 CPU 里使用脉冲开关作为时钟, 以实现单步执行
32     reg btn_clk_r1;
33     reg btn_clk_r2;
34     always @(posedge clk)
35     begin
36         if (!resetn)
37         begin
38             btn_clk_r1<= 1'b0;
39         end
40         else
41         begin
42             btn_clk_r1 <= ~btn_clk;
43         end
44
45         btn_clk_r2 <= btn_clk_r1;
46     end
47
48     wire clk_en;
49     assign clk_en = !resetn || (!btn_clk_r1 && btn_clk_r2);
50     BUFGCE cpu_clk_cg(.I(clk), .CE(clk_en), .O(cpu_clk));
51 //-----{时钟和复位信号}end
52 //-----{调用多周期 CPU 模块}begin
```

```
53 //用于在 FPGA 板上显示结果
54 wire [ 4:0] rf_addr; //扫描寄存器堆的地址
55 wire [31:0] rf_data; //寄存器堆从调试端口读出的数据
56 reg [31:0] mem_addr; //要观察的内存地址
57 wire [31:0] mem_data; //内存地址对应的数据
58 wire [31:0] IF_pc; //IF 模块的 PC
59 wire [31:0] IF_inst; //IF 模块取出的指令
60 wire [31:0] ID_pc; //ID 模块的 PC
61 wire [31:0] EXE_pc; //EXE 模块的 PC
62 wire [31:0] MEM_pc; //MEM 模块的 PC
63 wire [31:0] WB_pc; //WB 模块的 PC
64 wire [31:0] display_state; //展示 CPU 当前状态
65 multi_cycle_cpu cpu(
66     .clk (cpu_clk ),
67     .resetn (resetn ),
68
69     .rf_addr (rf_addr ),
70     .mem_addr(mem_addr),
71     .rf_data (rf_data ),
72     .mem_data(mem_data),
73     .IF_pc (IF_pc ),
74     .IF_inst (IF_inst ),
75     .ID_pc (ID_pc ),
76     .EXE_pc (EXE_pc ),
77     .MEM_pc (MEM_pc ),
78     .WB_pc (WB_pc ),
79     .display_state (display_state)
80 );
81 //-----{调用单周期 CPU 模块}end
82
83 //-----{调用触摸屏模块}begin-----//
84 //-----{实例化触摸屏}begin
85 //此小节不需要更改
86 reg display_valid;
87 reg [39:0] display_name;
88 reg [31:0] display_value;
89 wire [5 :0] display_number;
90 wire input_valid;
91 wire [31:0] input_value;
92
93 lcd_module lcd_module(
94     .clk (clk ), //10Mhz
95     .resetn (resetn ),
96
97     //调用触摸屏的接口
98     .display_valid (display_valid ),
99     .display_name (display_name ),
100     .display_value (display_value ),
101     .display_number (display_number),
102     .input_valid (input_valid ),
103     .input_value (input_value ),
```

```
104
105      //lcd 触摸屏相关接口，不需要更改
106      .lcd_rst      (lcd_rst      ),
107      .lcd_cs       (lcd_cs       ),
108      .lcd_rs        (lcd_rs        ),
109      .lcd_wr        (lcd_wr        ),
110      .lcd_rd        (lcd_rd        ),
111      .lcd_data_io   (lcd_data_io   ),
112      .lcd_bl_ctr    (lcd_bl_ctr    ),
113      .ct_int        (ct_int        ),
114      .ct_sda        (ct_sda        ),
115      .ct_scl        (ct_scl        ),
116      .ct_rstn       (ct_rstn       )
117  );
118  //-----{实例化触摸屏}end
119
120  //-----{从触摸屏获取输入}begin
121  //根据实际需要输入的数修改此小节，
122  //建议对每一个数的输入，编写单独一个 always 块
123      always @(posedge clk)
124      begin
125          if (!resetn)
126          begin
127              mem_addr <= 32'd0;
128          end
129          else if (input_valid)
130          begin
131              mem_addr <= input_value;
132          end
133      end
134      assign rf_addr = display_number-6'd11;
135  //-----{从触摸屏获取输入}end
136
137  //-----{输出到触摸屏显示}begin
138  //根据需要显示的数修改此小节，
139  //触摸屏上共有 44 块显示区域，可显示 44 组 32 位数据
140  //44 块显示区域从 1 开始编号，编号为 1~44，
141      always @(posedge clk)
142      begin
143          if (display_number >6'd10 && display_number <6'd43 )
144          begin //块号 5~36 显示 32 个通用寄存器的值
145              display_valid <= 1'b1;
146              display_name[39:16] <= "REG";
147              display_name[15: 8] <= {4'b0011,3'b000,rf_addr[4]};
148              display_name[7 : 0] <= {4'b0011,rf_addr[3:0]};
149              display_value      <= rf_data;
150          end
151          else
152          begin
153              case(display_number)
154                  6'd1 : //显示 IF 模块的 PC
```

```
155         begin
156             display_valid <= 1'b1;
157             display_name  <= "IF_PC";
158             display_value <= IF_pc;
159         end
160         6'd2 : //显示 IF 模块的指令
161         begin
162             display_valid <= 1'b1;
163             display_name  <= "IF_IN";
164             display_value <= IF_inst;
165         end
166         6'd3 : //显示 ID 模块的 PC
167         begin
168             display_valid <= 1'b1;
169             display_name  <= "ID_PC";
170             display_value <= ID_pc;
171         end
172         6'd4 : //显示 EXE 模块的 PC
173         begin
174             display_valid <= 1'b1;
175             display_name  <= "EXEPC";
176             display_value <= EXE_pc;
177         end
178         6'd5 : //显示 MEM 模块的 PC
179         begin
180             display_valid <= 1'b1;
181             display_name  <= "MEMPC";
182             display_value <= MEM_pc;
183         end
184         6'd6 : //显示 WB 模块的 PC
185         begin
186             display_valid <= 1'b1;
187             display_name  <= "WB_PC";
188             display_value <= WB_pc;
189         end
190         6'd7 : //显示要观察的内存地址
191         begin
192             display_valid <= 1'b1;
193             display_name  <= "MADDR";
194             display_value <= mem_addr;
195         end
196         6'd8 : //显示该内存地址对应的数据
197         begin
198             display_valid <= 1'b1;
199             display_name  <= "MDATA";
200             display_value <= mem_data;
201         end
202         6'd9 : //显示 CPU 当前状态
203         begin
204             display_valid <= 1'b1;
205             display_name  <= "STATE";
```

```
206             display_value <= display_state;
207         end
208         default :
209             begin
210                 display_valid <= 1'b0;
211                 display_name  <= 40'd0;
212                 display_value <= 32'd0;
213             end
214         endcase
215     end
216 end
217 //-----{输出到触摸屏显示}end
218 //-----{调用触摸屏模块}end-----//
219 endmodule
```

multi_cycle_cpu.v 源代码:

```
1  `timescale 1ns / 1ps
2  /**
3   * > 文件名: multi_cycle_cpu.v
4   * > 描述   : 多周期 CPU 模块, 共实现 36 条指令
5   * >         指令 rom 和数据 ram 均实例化 xilinx IP 得到, 为同步读写
6   * > 作者    : LOONGSON
7   * > 日期     : 2016-04-14
8   */
9  module multi_cycle_cpu( // 多周期 cpu
10      input clk,          // 时钟
11      input resetn,       // 复位信号, 低电平有效
12
13      //display data
14      input  [4:0] rf_addr,
15      input  [31:0] mem_addr,
16      output [31:0] rf_data,
17      output [31:0] mem_data,
18      output [31:0] IF_pc,
19      output [31:0] IF_inst,
20      output [31:0] ID_pc,
21      output [31:0] EXE_pc,
22      output [31:0] MEM_pc,
23      output [31:0] WB_pc,
24      output [31:0] display_state
25  );
26  //-----{控制多周期的状态机}begin-----
27  //
28      reg [2:0] state;          // 当前状态
29      reg [2:0] next_state;    // 下一状态
30
31      //展示当前处理器正在执行哪个模块
32      assign display_state = {29'd0, state};
33      // 状态机状态
34      parameter IDLE = 3'd0;    // 开始
```

```
34     parameter FETCH  = 3'd1; // 取指
35     parameter DECODE = 3'd2; // 译码
36     parameter EXE    = 3'd3; // 执行
37     parameter MEM     = 3'd4; // 访存
38     parameter WB      = 3'd5; // 写回
39
40     always @ (posedge clk)          // 当前状态
41     begin
42         if (!resetn) begin          // 如果复位信号有效
43             state <= IDLE;          // 当前状态为 开始
44         end
45         else begin                  // 否则
46             state <= next_state;    // 为下一状态
47         end
48     end
49
50     wire IF_over;                  // IF 模块已执行完
51     wire ID_over;                  // ID 模块已执行完
52     wire EXE_over;                 // EXE 模块已执行完
53     wire MEM_over;                 // MEM 模块已执行完
54     wire WB_over;                  // WB 模块已执行完
55     wire jbr_not_link; //分支指令(非 link 类), 只走 IF 和 ID 级
56     always @ (*)                  // 下一状态
57     begin
58         case (state)
59             IDLE :
60             begin
61                 next_state = FETCH; // 开始->取指
62             end
63             FETCH:
64             begin
65                 if (IF_over)
66                 begin
67                     next_state = DECODE; // 取指->译码
68                 end
69                 else
70                 begin
71                     next_state = FETCH; // 取指->译码
72                 end
73             end
74             DECODE:
75             begin
76                 if (ID_over)
77                 begin // 译码->执行或写回
78                     next_state = jbr_not_link ? FETCH : EXE;
79                 end
80                 else
81                 begin
82                     next_state = DECODE; // 取指->译码
83                 end
84             end
85         end
86     end
```

```

85         EXE:
86         begin
87             if (EXE_over)
88                 begin
89                     next_state = MEM;          // 执行->访存
90                 end
91             else
92                 begin
93                     next_state = EXE;          // 取指->译码
94                 end
95             end
96         MEM:
97         begin
98             if (MEM_over)
99                 begin
100                     next_state = WB;          // 访存->写回
101                 end
102             else
103                 begin
104                     next_state = MEM;          // 取指->译码
105                 end
106             end
107         WB:
108         begin
109             if (WB_over)
110                 begin
111                     next_state = FETCH;        // 写回->取指
112                 end
113             else
114                 begin
115                     next_state = WB;          // 取指->译码
116                 end
117             end
118         default : next_state = IDLE;
119     endcase
120 end
121 //5 模块的 valid 信号
122 wire IF_valid;
123 wire ID_valid;
124 wire EXE_valid;
125 wire MEM_valid;
126 wire WB_valid;
127 assign IF_valid = (state == FETCH ); //当前状态为取指时, IF 级有效
128 assign ID_valid = (state == DECODE); //当前状态为译码时, ID 级有效
129 assign EXE_valid = (state == EXE ); //当前状态为执行时, EXE 级有效
130 assign MEM_valid = (state == MEM ); //当前状态为访存时, MEM 级有效
131 assign WB_valid = (state == WB ); //当前状态为写回时, WB 级有效
132 //-----{控制多周期的状态机}end-----

```

```

133 //
134 //-----{5 级间的总线}begin-----
135 //
136 wire [ 63:0] IF_ID_bus;    // IF->ID 级总线
137 wire [149:0] ID_EXE_bus;  // ID->EXE 级总线
138 wire [105:0] EXE_MEM_bus; // EXE->MEM 级总线
139 wire [ 69:0] MEM_WB_bus;  // MEM->WB 级总线
140
141 //锁存以上总线信号
142 reg [ 63:0] IF_ID_bus_r;
143 reg [149:0] ID_EXE_bus_r;
144 reg [105:0] EXE_MEM_bus_r;
145 reg [ 69:0] MEM_WB_bus_r;
146
147 //IF 到 ID 的锁存信号
148 always @(posedge clk)
149 begin
150     if(IF_over)
151     begin
152         IF_ID_bus_r <= IF_ID_bus;
153     end
154 end
155 //ID 到 EXE 的锁存信号
156 always @(posedge clk)
157 begin
158     if(ID_over)
159     begin
160         ID_EXE_bus_r <= ID_EXE_bus;
161     end
162 end
163 //EXE 到 MEM 的锁存信号
164 always @(posedge clk)
165 begin
166     if(EXE_over)
167     begin
168         EXE_MEM_bus_r <= EXE_MEM_bus;
169     end
170 end
171 //MEM 到 WB 的锁存信号
172 always @(posedge clk)
173 begin
174     if(MEM_over)
175     begin
176         MEM_WB_bus_r <= MEM_WB_bus;
177     end
178 end
179 //-----{5 级间的总线}end-----
180 //
181 //-----{其他交互信号}begin-----

```

```

//
181 //跳转总线
182 wire [ 32:0] jbr_bus;
183
184 //IF 与 inst_rom 交互
185 wire [31:0] inst_addr;
186 wire [31:0] inst;
187
188 //MEM 与 data_ram 交互
189 wire [ 3:0] dm_wen;
190 wire [31:0] dm_addr;
191 wire [31:0] dm_wdata;
192 wire [31:0] dm_rdata;
193
194 //ID 与 regfile 交互
195 wire [ 4:0] rs;
196 wire [ 4:0] rt;
197 wire [31:0] rs_value;
198 wire [31:0] rt_value;
199
200 //WB 与 regfile 交互
201 wire rf_wen;
202 wire [ 4:0] rf_wdest;
203 wire [31:0] rf_wdata;
204 //-----{其他交互信号}end-----
205 //
206 //-----{各模块实例化}begin-----
207 //
208 wire next_fetch; //即将运行取指模块, 需要先锁存 PC 值
209 //当前状态为 decode, 且指令为跳转分支指令(非 link 类), 且 decode 执行完
210 成
211 //或者, 当前状态为 wb, 且 wb 执行完成, 则即将进入 fetch 状态
212 assign next_fetch = (state==DECODE & ID_over &
213 jbr_not_link)
214 | (state==WB & WB_over);
215
216 fetch IF_module( // 取指级
217 .clk (clk ), // I, 1
218 .resetn (resetn ), // I, 1
219 .IF_valid (IF_valid ), // I, 1
220 .next_fetch(next_fetch), // I, 1
221 .inst (inst ), // I, 32
222 .jbr_bus (jbr_bus ), // I, 33
223 .inst_addr (inst_addr ), // O, 32
224 .IF_over (IF_over ), // O, 1
225 .IF_ID_bus (IF_ID_bus ), // O, 64
226
227 //展示 PC 和取出的指令
228 .IF_pc (IF_pc ),
229 .IF_inst (IF_inst )
230 );

```

```

227
228     decode ID_module(                                // 译码级
229         .ID_valid      (ID_valid      ), // I, 1
230         .IF_ID_bus_r   (IF_ID_bus_r   ), // I, 64
231         .rs_value      (rs_value      ), // I, 32
232         .rt_value      (rt_value      ), // I, 32
233         .rs             (rs             ), // O, 5
234         .rt             (rt             ), // O, 5
235         .jbr_bus       (jbr_bus       ), // O, 33
236         .jbr_not_link  (jbr_not_link), // O, 1
237         .ID_over       (ID_over       ), // O, 1
238         .ID_EXE_bus    (ID_EXE_bus    ), // O, 150
239
240         //展示 PC
241         .ID_pc         (ID_pc         )
242     );
243
244     exe EXE_module(                                    // 执行级
245         .EXE_valid      (EXE_valid      ), // I, 1
246         .ID_EXE_bus_r   (ID_EXE_bus_r   ), // I, 150
247         .EXE_over       (EXE_over       ), // O, 1
248         .EXE_MEM_bus    (EXE_MEM_bus    ), // O, 106
249
250         //展示 PC
251         .EXE_pc         (EXE_pc         )
252     );
253
254     mem MEM_module(                                    // 访存级
255         .clk            (clk            ), // I, 1
256         .MEM_valid      (MEM_valid      ), // I, 1
257         .EXE_MEM_bus_r  (EXE_MEM_bus_r  ), // I, 106
258         .dm_rdata       (dm_rdata       ), // I, 32
259         .dm_addr        (dm_addr        ), // O, 32
260         .dm_wen         (dm_wen         ), // O, 4
261         .dm_wdata       (dm_wdata       ), // O, 32
262         .MEM_over       (MEM_over       ), // O, 1
263         .MEM_WB_bus     (MEM_WB_bus     ), // O, 70
264
265         //展示 PC
266         .MEM_pc         (MEM_pc         )
267     );
268
269     wb WB_module(                                       // 写回级
270         .WB_valid      (WB_valid      ), // I, 1
271         .MEM_WB_bus_r  (MEM_WB_bus_r  ), // I, 70
272         .rf_wen        (rf_wen        ), // O, 1
273         .rf_wdest       (rf_wdest       ), // O, 5
274         .rf_wdata       (rf_wdata       ), // O, 32
275         .WB_over       (WB_over       ), // O, 1
276
277         //展示 PC

```

```

278         .WB_pc      (WB_pc      )
279     );
280
281     inst_rom inst_rom_module(          // 指令存储器
282         .clk         (clk         ), // I, 1 ,时钟
283         .addra        (inst_addr[9:2]), // I, 8 ,指令地址
284         .douta        (inst        )  // O, 32,指令
285     );
286
287     regfile rf_module(          // 寄存器堆模块
288         .clk          (clk          ), // I, 1
289         .wen           (rf_wen       ), // I, 1
290         .raddr1        (rs            ), // I, 5
291         .raddr2        (rt            ), // I, 5
292         .waddr         (rf_wdest     ), // I, 5
293         .wdata         (rf_wdata     ), // I, 32
294         .rdata1        (rs_value     ), // O, 32
295         .rdata2        (rt_value     ), // O, 32
296
297         //display rf
298         .test_addr(rf_addr),
299         .test_data(rf_data)
300     );
301
302     data_ram data_ram_module( // 数据存储模块
303         .clk           (clk           ), // I, 1, 时钟
304         .wea           (dm_wen        ), // I, 1, 写使能
305         .addra         (dm_addr[9:2]), // I, 8, 读地址
306         .dina          (dm_wdata      ), // I, 32, 写数据
307         .douta         (dm_rdata      ), // O, 32, 读数据
308
309         //display mem
310         .clkb          (clk           ),
311         .web           (4'd0          ),
312         .addrb         (mem_addr[9:2]),
313         .doutb         (mem_data      ),
314         .dinb          (32'd0         )
315     );
316     //-----{各模块实例化}end-----
317     endmodule

```

fetch.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3  //  > 文件名: fetch.v
4  //  > 描述   :多周期 CPU 的取指模块
5  //  > 作者   : LOONGSON
6  //  > 日期   : 2016-04-14
7  /**

```

```

8  `define STARTADDR 32'd0    // 程序起始地址为 0
9  module fetch(                // 取指级
10     input                    clk,        // 时钟
11     input                    resetn,      // 复位信号，低电平有效
12     input                    IF_valid,    // 取指级有效信号
13     input                    next_fetch,  // 取下一条指令，用来锁存 PC 值
14     input [31:0] inst,          // inst_rom 取出的指令
15     input [32:0] jbr_bus,       // 跳转总线
16     output [31:0] inst_addr,  // 发往 inst_rom 的取指地址
17     output reg               IF_over,    // IF 模块执行完成
18     output [63:0] IF_ID_bus,  // IF->ID 总线
19
20     //展示 PC 和取出的指令
21     output [31:0] IF_pc,
22     output [31:0] IF_inst
23 );
24
25 //-----{程序计数器 PC}begin
26     wire [31:0] next_pc;
27     wire [31:0] seq_pc;
28     reg [31:0] pc;
29     //跳转 pc
30     wire jbr_taken;
31     wire [31:0] jbr_target;
32     assign {jbr_taken, jbr_target} = jbr_bus; //跳转总线
33
34     assign seq_pc[31:2] = pc[31:2] + 1'b1;    //下一指令地址：
PC=PC+4
35     assign seq_pc[1 :0] = pc[1:0];
36
37     // 新指令：若指令跳转，为跳转地址；否则为下一指令
38     assign next_pc = jbr_taken ? jbr_target : seq_pc;
39
40     always @(posedge clk)    // PC 程序计数器
41     begin
42         if (!resetn)
43         begin
44             pc <= `STARTADDR; // 复位，取程序起始地址
45         end
46         else if (next_fetch)
47         begin
48             pc <= next_pc;    // 不复位，取新指令
49         end
50     end
51 //-----{程序计数器 PC}end
52
53 //-----{发往 inst_rom 的取指地址}begin
54     assign inst_addr = pc;
55 //-----{发往 inst_rom 的取指地址}end
56
57 //-----{IF 执行完成}begin

```

```

58      //由于指令 rom 为同步读写的,
59      //取数据时, 有一拍延时
60      //即发地址的下一拍时钟才能得到对应的指令
61      //故取指模块需要两拍时间
62      //将 IF_valid 锁存一拍即是 IF_over 信号
63      always @(posedge clk)
64      begin
65          IF_over <= IF_valid;
66      end
67      //如果指令 rom 为异步读的, 则 IF_valid 即是 IF_over 信号,
68      //即取指一拍完成
69      //-----{IF 执行完成}end
70
71      //-----{IF->ID 总线}begin
72      assign IF_ID_bus = {pc, inst};
73      //-----{IF->ID 总线}end
74
75      //-----{展示 IF 模块的 PC 值和指令}begin
76      assign IF_pc      = pc;
77      assign IF_inst = inst;
78      //-----{展示 IF 模块的 PC 值和指令}end
79      endmodule

```

decode.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3   * > 文件名: decode.v
4   * > 描述   : 多周期 CPU 的译码模块
5   * > 作者   : LOONGSON
6   * > 日期   : 2016-04-14
7   */
8  module decode(                                // 译码级
9      input          ID_valid,                  // 译码级有效信号
10     input          [ 63:0] IF_ID_bus_r,       // IF->ID 总线
11     input          [ 31:0] rs_value,          // 第一源操作数值
12     input          [ 31:0] rt_value,          // 第二源操作数值
13     output         [  4:0] rs,                 // 第一源操作数地址
14     output         [  4:0] rt,                 // 第二源操作数地址
15     output         [ 32:0] jbr_bus,           // 跳转总线
16     output          jbr_not_link,             // 指令为跳转分支指令, 且非 link 类指令
17     output          ID_over,                  // ID 模块执行完成
18     output         [149:0] ID_EXE_bus,       // ID->EXE 总线
19
20     //展示 PC
21     output         [ 31:0] ID_pc
22 );
23 //-----{IF->ID 总线}begin
24     wire [31:0] pc;
25     wire [31:0] inst;
26     assign {pc, inst} = IF_ID_bus_r; // IF->ID 总线传 PC 和指令

```

```
27 //-----{IF->ID 总线}end
28
29 //-----{指令译码}begin
30     wire [5:0] op;
31     wire [4:0] rd;
32     wire [4:0] sa;
33     wire [5:0] funct;
34     wire [15:0] imm;
35     wire [15:0] offset;
36     wire [25:0] target;
37
38     assign op      = inst[31:26]; // 操作码
39     assign rs      = inst[25:21]; // 源操作数 1
40     assign rt      = inst[20:16]; // 源操作数 2
41     assign rd      = inst[15:11]; // 目标操作数
42     assign sa      = inst[10:6];  // 特殊域, 可能存放偏移量
43     assign funct   = inst[5:0];   // 功能码
44     assign imm     = inst[15:0];  // 立即数
45     assign offset  = inst[15:0];  // 地址偏移量
46     assign target  = inst[25:0];  // 目标地址
47
48     // 实现指令列表
49     wire inst_ADDU, inst_SUBU, inst_SLT, inst_AND;
50     wire inst_NOR, inst_OR, inst_XOR, inst_SLL;
51     wire inst_SRL, inst_ADDIU, inst_BEQ, inst_BNE;
52     wire inst_LW, inst_SW, inst_LUI, inst_J;
53     wire inst_SLTU, inst_JALR, inst_JR, inst_SLLV;
54     wire inst_SRA, inst_SRAV, inst_SRLV, inst_SLTIU;
55     wire inst_SLTI, inst_BGEZ, inst_BGTZ, inst_BLEZ;
56     wire inst_BLTZ, inst_LB, inst_LBU, inst_SB;
57     wire inst_ANDI, inst_ORI, inst_XORI, inst_JAL;
58     wire op_zero; // 操作码全 0
59     wire sa_zero; // sa 域全 0
60     assign op_zero = ~(|op);
61     assign sa_zero = ~(|sa);
62     assign inst_ADDU = op_zero & sa_zero & (funct == 6'b100001); // 无符号加法
63     assign inst_SUBU = op_zero & sa_zero & (funct == 6'b100011); // 无符号减法
64     assign inst_SLT = op_zero & sa_zero & (funct == 6'b101010); // 小于则置位
65     assign inst_SLTU = op_zero & sa_zero & (funct == 6'b101011); // 无符号小则置
66     assign inst_JALR = op_zero & (rt==5'd0) & (rd==5'd31)
67         & sa_zero & (funct == 6'b001001); // 跳转寄存器并链接
68     assign inst_JR = op_zero & (rt==5'd0) & (rd==5'd0)
69         & sa_zero & (funct == 6'b001000); // 跳转寄存器
70     assign inst_AND = op_zero & sa_zero & (funct == 6'b100100); // 与运算
71     assign inst_NOR = op_zero & sa_zero & (funct == 6'b100111); // 或非运算
72     assign inst_OR = op_zero & sa_zero & (funct == 6'b100101); // 或运算
73     assign inst_XOR = op_zero & sa_zero & (funct == 6'b100110); // 异或运算
74     assign inst_SLL = op_zero & (rs==5'd0) & (funct == 6'b000000); // 逻辑左移
75     assign inst_SLLV = op_zero & sa_zero & (funct == 6'b000100); // 变量逻辑左移
76     assign inst_SRA = op_zero & (rs==5'd0) & (funct == 6'b000011); // 算术右移
77     assign inst_SRAV = op_zero & sa_zero & (funct == 6'b000111); // 变量算术右移
```

```

78     assign inst_SRL    = op_zero & (rs==5'd0) & (funct == 6'b000010); //逻辑右移
79     assign inst_SRLV   = op_zero & sa_zero    & (funct == 6'b000110); //变量逻辑右移
80     assign inst_ADDIU  = (op == 6'b001001);           //立即数无符号加法
81     assign inst_SLTI   = (op == 6'b001010);           //小于立即数则置位
82     assign inst_SLTIU  = (op == 6'b001011);           //无符号小于立即数则置位
83     assign inst_BEQ    = (op == 6'b000100);           //判断相等跳转
84     assign inst_BGEZ   = (op == 6'b000001) & (rt==5'd1); //大于等于 0 跳转
85     assign inst_BGTZ   = (op == 6'b000111) & (rt==5'd0); //大于 0 跳转
86     assign inst_BLEZ   = (op == 6'b000110) & (rt==5'd0); //小于等于 0 跳转
87     assign inst_BLTZ   = (op == 6'b000001) & (rt==5'd0); //小于 0 跳转
88     assign inst_BNE    = (op == 6'b000101);           //判断不等跳转
89     assign inst_LW     = (op == 6'b100011);           //从内存装载字
90     assign inst_SW     = (op == 6'b101011);           //向内存存储字
91     assign inst_LB     = (op == 6'b100000);           //load 字节 (符号扩展)
92     assign inst_LBU    = (op == 6'b100100);           //load 字节 (无符号扩展)
93     assign inst_SB     = (op == 6'b101000);           //向内存存储字节
94     assign inst_ANDI   = (op == 6'b001100);           //立即数与
95     assign inst_LUI    = (op == 6'b001111) & (rs==5'd0); //立即数装载高半字节
96     assign inst_ORI    = (op == 6'b001101);           //立即数或
97     assign inst_XORI   = (op == 6'b001110);           //立即数异或
98     assign inst_J      = (op == 6'b000010);           //跳转
99     assign inst_JAL    = (op == 6'b000011);           //跳转和链接
100
101     //跳转分支指令
102     wire inst_jr;      //寄存器跳转指令
103     wire inst_j_link;  //链接跳转指令
104     assign inst_jr     = inst_JALR | inst_JR;
105     assign inst_j_link = inst_JAL  | inst_JALR;
106     assign jbr_not_link= inst_J    | inst_JR      //全部非 link 类跳转指令
107                     | inst_BEQ   | inst_BNE   | inst_BGEZ
108                     | inst_BGTZ  | inst_BLEZ  | inst_BLTZ;
109
110     //load store
111     wire inst_load;
112     wire inst_store;
113     assign inst_load  = inst_LW | inst_LB | inst_LBU; // load 指
114     assign inst_store = inst_SW | inst_SB;           // store 指
115
116     //alu 操作分类
117     wire inst_add, inst_sub, inst_slt, inst_sltu;
118     wire inst_and, inst_nor, inst_or, inst_xor;
119     wire inst_sll, inst_srl, inst_sra, inst_lui;
120     assign inst_add = inst_ADDU | inst_ADDIU | inst_load
121                 | inst_store | inst_j_link; // 做加法
122     assign inst_sub = inst_SUBU;           // 减法
123     assign inst_slt = inst_SLT | inst_SLTI; // 有符号小于置
124     assign inst_sltu= inst_SLTIU | inst_SLTU; // 无符号小于置

```

```

125     assign inst_and = inst_AND | inst_ANDI;           // 逻辑与
126     assign inst_nor = inst_NOR;                       // 逻辑或非
127     assign inst_or  = inst_OR  | inst_ORI;           // 逻辑或
128     assign inst_xor = inst_XOR | inst_XORI;          // 逻辑或非
129     assign inst_sll = inst_SLL | inst_SLLV;          // 逻辑左移
130     assign inst_srl = inst_SRL | inst_SRLV;          // 逻辑右移
131     assign inst_sra = inst_SRA | inst_SRAV;          // 算术右移
132     assign inst_lui = inst_LUI;                       // 立即数装载高
133 位
134     //使用 sa 域作为偏移量的移位指令
135     wire inst_shf_sa;
136     assign inst_shf_sa = inst_SLL | inst_SRL | inst_SRA;
137
138     //依据立即数扩展方式分类
139     wire inst_imm_zero; //立即数 0 扩展
140     wire inst_imm_sign; //立即数符号扩展
141     assign inst_imm_zero = inst_ANDI | inst_LUI | inst_ORI | inst_XORI;
142     assign inst_imm_sign = inst_ADDIU | inst_SLTI | inst_SLTIU
143                             | inst_load | inst_store;
144
145     //依据目的寄存器号分类
146     wire inst_wdest_rt; // 寄存器堆写入地址为 rt 的指令
147     wire inst_wdest_31; // 寄存器堆写入地址为 31 的指令
148     wire inst_wdest_rd; // 寄存器堆写入地址为 rd 的指令
149     assign inst_wdest_rt = inst_imm_zero | inst_ADDIU |
inst_SLTI
150                             | inst_SLTIU | inst_load;
151     assign inst_wdest_31 = inst_JAL;
152     assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT | inst_SLTU
153                             | inst_JALR | inst_AND | inst_NOR | inst_OR
154                             | inst_XOR | inst_SLL | inst_SLLV | inst_SRA
155                             | inst_SRAV | inst_SRL | inst_SRLV;
156 //-----{指令译码}end
157
158 //-----{分支指令执行}begin
159     //无条件跳转
160     wire j_taken;
161     wire [31:0] j_target;
162     assign j_taken = inst_J | inst_JAL | inst_jr;
163     //寄存器跳转地址为 rs_value, 其他跳转为{pc[31:28],target,2'b00}
164     assign j_target = inst_jr ? rs_value :
{pc[31:28],target,2'b00};
165
166     //branch 指令
167     wire rs_equql_rt;
168     wire rs_ez;
169     wire rs_ltz;
170     assign rs_equql_rt = (rs_value == rt_value); // GPR[rs]==GPR[rt]
171     assign rs_ez = ~(|rs_value); // rs 寄存器值为 0
172     assign rs_ltz = rs_value[31]; // rs 寄存器值小于 0

```

```

173     wire br_taken;
174     wire [31:0] br_target;
175     assign br_taken = inst_BEQ  & rs_equql_rt      // 相等跳转
176                      | inst_BNE  & ~rs_equql_rt    // 不等跳转
177                      | inst_BGEZ & ~rs_ltz         // 大于等于 0 跳
转
178                      | inst_BGTZ & ~rs_ltz & ~rs_ez // 大于 0 跳转
179                      | inst_BLEZ & (rs_ltz | rs_ez) // 小于等于 0 跳
转
180                      | inst_BLTZ & rs_ltz;         // 小于 0 跳转
181     // 分支跳转目标地址: PC=PC+offset<<2
182     assign br_target[31:2] = pc[31:2] + {{14{offset[15]}}, offset};
183     assign br_target[1:0] = pc[1:0];
184
185     //jump and branch 指令
186     wire jbr_taken;
187     wire [31:0] jbr_target;
188     assign jbr_taken = j_taken | br_taken;
189     assign jbr_target = j_target ? j_target : br_target;
190
191     //ID 到 IF 的跳转总线
192     assign jbr_bus = {jbr_taken, jbr_target};
193 //-----{分支指令执行}end
194
195 //-----{ID 执行完成}begin
196     //由于是多周期的, 不存在数据相关
197     //故 ID 模块一拍就能完成所有操作
198     //故 ID_valid 即是 ID_over 信号
199     assign ID_over = ID_valid;
200 //-----{ID 执行完成}end
201
202 //-----{ID->EXE 总线}begin
203     //EXE 需要用到的信息
204     //ALU 两个源操作数和控制信号
205     wire [11:0] alu_control;
206     wire [31:0] alu_operand1;
207     wire [31:0] alu_operand2;
208
209     //所谓链接跳转是将跳转返回的 PC 值存放到 31 号寄存器里
210     //在多周期 CPU 里, 不考虑延迟槽, 故链接跳转需要计算 PC+4, 存放到 31 号寄存器
里
211     assign alu_operand1 = inst_j_link ? pc :
212                          inst_shf_sa ? {27'd0,sa} : rs_value;
213     assign alu_operand2 = inst_j_link ? 32'd4 :
214                          inst_imm_zero ? {16'd0, imm} :
215                          inst_imm_sign ? {{16{imm[15]}}, imm} : rt_value;
216     assign alu_control = {inst_add,          // ALU 操作码, 独热编码
217                          inst_sub,
218                          inst_slt,
219                          inst_sltu,
220                          inst_and,

```

```

221             inst_nor,
222             inst_or,
223             inst_xor,
224             inst_sll,
225             inst_srl,
226             inst_sra,
227             inst_lui};
228     //访存需要用到的 load/store 信息
229     wire lb_sign; //load 一字节为有符号 load
230     wire ls_word; //load/store 为字节还是字,0:byte;1:word
231     wire [3:0] mem_control; //MEM 需要使用的控制信号
232     wire [31:0] store_data; //store 操作的存的数据
233     assign lb_sign = inst_LB;
234     assign ls_word = inst_LW | inst_SW;
235     assign mem_control = {inst_load,
236                          inst_store,
237                          ls_word,
238                          lb_sign };
239
240     //写回需要用到的信息
241     wire rf_wen; //写回的寄存器写使能
242     wire [4:0] rf_wdest; //写回的目的寄存器
243     assign rf_wen = inst_wdest_rt | inst_wdest_31 |
inst_wdest_rd;
244     assign rf_wdest = inst_wdest_rt ? rt ://在不写寄存器堆时, 设置为
0
245                      inst_wdest_31 ? 5'd31 :
246                      inst_wdest_rd ? rd : 5'd0;
247     assign store_data = rt_value;
248     assign ID_EXE_bus = {alu_control,alu_operand1,alu_operand2,//EXE 需要使用的信息
249                        mem_control,store_data, //MEM 需要使用的信号
250                        rf_wen, rf_wdest, //WB 需要使用的信号
251                        pc}; //PC 值
252 //-----{ID->EXE 总线}end
253
254 //-----{展示 ID 模块的 PC 值}begin
255     assign ID_pc = pc;
256 //-----{展示 ID 模块的 PC 值}end
257 endmodule

```

exe.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: exe.v
4  //    > 描述   :多周期 CPU 的执行模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  module exe( // 执行级
9      input   EXE_valid, // 执行级有效信号

```

```
10     input      [149:0] ID_EXE_bus_r, // ID->EXE 总线
11     output          EXE_over,      // EXE 模块执行完成
12     output      [105:0] EXE_MEM_bus, // EXE->MEM 总线
13
14     //展示 PC
15     output      [ 31:0] EXE_pc
16 );
17 //-----{ID->EXE 总线}begin
18     //EXE 需要用到的信息
19     //ALU 两个源操作数和控制信号
20     wire [11:0] alu_control;
21     wire [31:0] alu_operand1;
22     wire [31:0] alu_operand2;
23
24     //访存需要用到的 load/store 信息
25     wire [3:0] mem_control; //MEM 需要使用的控制信号
26     wire [31:0] store_data; //store 操作的存的数据
27
28     //写回需要用到的信息
29     wire      rf_wen;      //写回的寄存器写使能
30     wire [4:0] rf_wdest;   //写回的目的寄存器
31
32     //pc
33     wire [31:0] pc;
34     assign {alu_control,
35             alu_operand1,
36             alu_operand2,
37             mem_control,
38             store_data,
39             rf_wen,
40             rf_wdest,
41             pc          } = ID_EXE_bus_r;
42 //-----{ID->EXE 总线}end
43
44 //-----{ALU}begin
45     wire [31:0] alu_result;
46
47     alu alu_module(
48         .alu_control  (alu_control ), // I, 12, ALU 控制信号
49         .alu_src1     (alu_operand1), // I, 32, ALU 操作数 1
50         .alu_src2     (alu_operand2), // I, 32, ALU 操作数 2
51         .alu_result   (alu_result  ) // O, 32, ALU 结果
52     );
53 //-----{ALU}end
54
55 //-----{EXE 执行完成}begin
56     //由于是多周期的, 不存在数据相关
57     //且所有 ALU 运算都可在一拍内完成
58     //故 EXE 模块一拍就能完成所有操作
59     //故 EXE_valid 即是 EXE_over 信号
60     assign EXE_over = EXE_valid;
```

```

61 //-----{EXE 执行完成}end
62
63 //-----{EXE->MEM 总线}begin
64     assign EXE_MEM_bus = {mem_control,store_data,    //load/store 信息和 store 数据
65                           alu_result,                //alu 运算结果
66                           rf_wen,rf_wdest,           // WB 需要使用的信号
67                           pc};                      // PC
68 //-----{EXE->MEM 总线}end
69
70 //-----{展示 EXE 模块的 PC 值}begin
71     assign EXE_pc = pc;
72 //-----{展示 EXE 模块的 PC 值}end
73 endmodule

```

mem.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3   * > 文件名: mem.v
4   * > 描述 : 多周期 CPU 的访存模块
5   * > 作者 : LOONGSON
6   * > 日期 : 2016-04-14
7   */
8  module mem(                                // 访存级
9      input          clk,                    // 时钟
10     input          MEM_valid,              // 访存级有效信号
11     input [105:0] EXE_MEM_bus_r,          // EXE->MEM 总线
12     input [31:0] dm_rdata,                // 访存读数据
13     output [31:0] dm_addr,                // 访存读写地址
14     output reg [3:0] dm_wen,               // 访存写使能
15     output reg [31:0] dm_wdata,           // 访存写数据
16     output          MEM_over,              // MEM 模块执行完成
17     output [69:0] MEM_WB_bus,             // MEM->WB 总线
18
19     //展示 PC
20     output [31:0] MEM_pc
21 );
22 //-----{EXE->MEM 总线}begin
23     //访存需要用到的 load/store 信息
24     wire [3:0] mem_control; //MEM 需要使用的控制信号
25     wire [31:0] store_data; //store 操作的存的数据
26
27     //alu 运算结果
28     wire [31:0] alu_result;
29
30     //写回需要用到的信息
31     wire rf_wen; //写回的寄存器写使能
32     wire [4:0] rf_wdest; //写回的目的寄存器
33
34     //pc
35     wire [31:0] pc;

```

```
36     assign {mem_control,
37             store_data,
38             alu_result,
39             rf_wen,
40             rf_wdest,
41             pc          } = EXE_MEM_bus_r;
42 //-----{EXE->MEM 总线}end
43
44 //-----{load/store 访存}begin
45     wire inst_load; //load 操作
46     wire inst_store; //store 操作
47     wire ls_word;    //load/store 为字节还是字,0:byte;1:word
48     wire lb_sign;    //load 一字节为有符号 load
49     assign {inst_load,inst_store,ls_word,lb_sign} = mem_control;
50
51     //访存读写地址
52     assign dm_addr = alu_result;
53
54     //store 操作的写使能
55     always @ (*) // 内存写使能信号
56     begin
57         if (MEM_valid && inst_store) // 访存级有效时,且为 store 操作
58         begin
59             if (ls_word)
60             begin
61                 dm_wen <= 4'b1111; // 存储字指令,写使能全 1
62             end
63             else
64             begin // SB 指令,需要依据地址底两位,确定对应的写使能
65                 case (dm_addr[1:0])
66                     2'b00 : dm_wen <= 4'b0001;
67                     2'b01 : dm_wen <= 4'b0010;
68                     2'b10 : dm_wen <= 4'b0100;
69                     2'b11 : dm_wen <= 4'b1000;
70                     default : dm_wen <= 4'b0000;
71                 endcase
72             end
73         end
74         else
75         begin
76             dm_wen <= 4'b0000;
77         end
78     end
79
80     //store 操作的写数据
81     always @ (*) // 对于 SB 指令,需要依据地址低两位,移动 store 的字节至对应位置
82     begin
83         case (dm_addr[1:0])
84             2'b00 : dm_wdata <= store_data;
85             2'b01 : dm_wdata <= {16'd0, store_data[7:0],
86                                     8'd0};
```

```

86      2'b10    : dm_wdata <= {8'd0, store_data[7:0]},
16'd0};
87      2'b11    : dm_wdata <= {store_data[7:0], 24'd0};
88      default : dm_wdata <= store_data;
89      endcase
90  end
91
92      //load 读出的数据
93      wire      load_sign;
94      wire [31:0] load_result;
95      assign load_sign = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7] :
96                          (dm_addr[1:0]==2'd1) ? dm_rdata[15] :
97                          (dm_addr[1:0]==2'd2) ? dm_rdata[23] : dm_rdata[31] ;
98      assign load_result[7:0] = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7:0 ] :
99                          (dm_addr[1:0]==2'd1) ? dm_rdata[15:8 ] :
100                         (dm_addr[1:0]==2'd2) ? dm_rdata[23:16] :
101                         dm_rdata[31:24] ;
102      assign load_result[31:8]= ls_word ? dm_rdata[31:8] :
103                          {24{lb_sign & load_sign}};
104  //-----{load/store 访存}end
105
106  //-----{MEM 执行完成}begin
107      //由于数据 RAM 为同步读写的,
108      //故对 load 指令, 取数据时, 有一拍延时
109      //即发地址的下一拍时钟才能得到 load 的数据
110      //故 mem 在进行 load 操作时有需要两拍时间才能取到数据
111      //而对其他操作, 则只需要一拍时间
112      reg MEM_valid_r;
113      always @(posedge clk)
114      begin
115          MEM_valid_r <= MEM_valid;
116      end
117      assign MEM_over = inst_load ? MEM_valid_r :MEM_valid;
118      //如果数据 ram 为异步读的, 则 MEM_valid 即是 MEM_over 信号,
119      //即 load 一拍完成
120  //-----{MEM 执行完成}end
121
122  //-----{MEM->WB 总线}begin
123      wire [31:0] mem_result; //MEM 传到 WB 的 result 为 load 结果或 ALU 结
果
124      assign mem_result = inst_load ? load_result : alu_result;
125
126      assign MEM_WB_bus = {rf_wen,rf_wdest, // WB 需要使用的信号
127                          mem_result,      // 最终要写回寄存器的数据
128                          pc};             // PC 值
129  //-----{MEM->WB 总线}begin
130
131  //-----{展示 MEM 模块的 PC 值}begin
132      assign MEM_pc = pc;
133  //-----{展示 MEM 模块的 PC 值}end
134  endmodule

```

wb.v 源代码:

```
1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: wb.v
4  //    > 描述   :多周期 CPU 的写回模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  module wb(                                // 写回级
9      input      WB_valid,                  // 写回级有效
10     input  [69:0] MEM_WB_bus_r,           // MEM->WB 总线
11     output      rf_wen,                    // 寄存器写使能
12     output  [ 4:0] rf_wdest,               // 寄存器写地址
13     output  [31:0] rf_wdata,               // 寄存器写数据
14     output      WB_over,                   // WB 模块执行完成
15
16     //展示 PC
17     output  [ 31:0] WB_pc
18 );
19 //-----{MEM->WB 总线}begin
20     //寄存器堆写使能和写地址
21     wire wen;
22     wire [4:0] wdest;
23
24     //MEM 传来的 result
25     wire [31:0] mem_result;
26
27     //pc
28     wire [31:0] pc;
29     assign {wen,wdest,mem_result,pc} = MEM_WB_bus_r;
30 //-----{MEM->WB 总线}end
31
32 //-----{WB 执行完成}begin
33     //WB 模块只是传递寄存器堆的写使能/写地址和写数据
34     //可在一拍内完成
35     //故 WB_valid 即是 WB_over 信号
36     assign WB_over = WB_valid;
37 //-----{WB 执行完成}end
38
39 //-----{WB->regfile 信号}begin
40     assign rf_wen   = wen & WB_valid;
41     assign rf_wdest = wdest;
42     assign rf_wdata = mem_result;
43 //-----{WB->regfile 信号}end
44
45 //-----{展示 WB 模块的 PC 值}begin
46     assign WB_pc = pc;
47 //-----{展示 WB 模块的 PC 值}end
48 endmodule
```

多周期 CPU 参考设计使用的 inst_rom 和 data_ram 均为实例化的同步存储器 IP。
多周期 CPU 参考设计使用的 regfile.v 和 alu.v，依然是实验三和实验四中给出的代码，
本实验中直接通过“Add Copy of Source”添加到工程中即可，此处就不在附上源码了。
其中 alu.v 还调用了实验一中的 adder.v。

约束文件 multi_cycle_cpu.xdc 源代码：

```
1      #时钟信号连接
2      set_property PACKAGE_PIN AC19 [get_ports clk]
3
4      #脉冲开关，用于输入作为复位信号，低电平有效
5      set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7      #脉冲开关，用于输入作为单步执行的 clk
8      set_property PACKAGE_PIN Y5 [get_ports btn_clk]
9
10     set_property IOSTANDARD LVCMOS33 [get_ports clk]
11     set_property IOSTANDARD LVCMOS33 [get_ports resetn]
12     set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
13
14     #触摸屏引脚连接
15     set_property PACKAGE_PIN J25 [get_ports lcd_rst]
16     set_property PACKAGE_PIN H18 [get_ports lcd_cs]
17     set_property PACKAGE_PIN K16 [get_ports lcd_rs]
18     set_property PACKAGE_PIN L8 [get_ports lcd_wr]
19     set_property PACKAGE_PIN K8 [get_ports lcd_rd]
20     set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
21     set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
22     set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
23     set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
24     set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
25     set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
26     set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
27     set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
28     set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
29     set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
30     set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
31     set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
32     set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
33     set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
34     set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
35     set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
36     set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
37     set_property PACKAGE_PIN L19 [get_ports ct_int]
38     set_property PACKAGE_PIN J24 [get_ports ct_sda]
39     set_property PACKAGE_PIN H21 [get_ports ct_scl]
40     set_property PACKAGE_PIN G24 [get_ports ct_rstn]
41
```

```
42 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
43 set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
44 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
45 set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
46 set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
47 set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
48 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
67 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
68
69 set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
```

八、 课程设计 静态 5 级流水线 CPU 实现

1 课程设计目的

1. 在多周期 CPU 实验完成的提前下，深入理解 CPU 流水线的概念。
2. 熟悉并掌握流水线 CPU 的原理和设计。
3. 最终检验运用 verilog 语言进行电路设计的能力。
4. 通过亲自设计实现静态 5 级流水线 CPU，加深对计算机组成原理和体系结构理论知识的理解。
5. 培养对 CPU 设计的兴趣，加深对 CPU 现有架构的理解和深思。

2 实验设备

1. 装有 xilinx ISE 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 课程设计任务

1. 本课程设计是对之前课程实验的拔高。前期的课程设计准备同多周期 CPU 的实验，主体部分可以直接使用多周期 CPU 实验的设计方案，但在多周期 CPU 中只要求实现了 30 多条指令，此处要求扩展到 40 多条指令。

多周期 CPU 在单周期基础上提高了时钟频率，但并没有改善执行一条指令的时间，且存在资源闲置的问题，例如当指令在执行级有效时，译码级实际上在空转。若每一级都在执行有效的指令，将解决资源闲置的问题。最理想的情况是，当第一条指令从取指级转换到下一级译码时，第二条指令进入取指级，当第一条指令完成译码进入执行级时，第二条指令进入译码级，第三条指令进入取指级……静态 5 级流水 CPU 就是基于这样的设计思路。

在流水 CPU 中，当在一时钟周期内完成了某一条指令的全部执行时（写回级完成），则有望在下一时钟周期内完成下条指令的执行，因此依然相当于是一个周期完成一条指令，而时钟频率更高，因此 CPU 可以运行的更快。

本次课程就是将上次所实现的多周期 CPU 更改结构为静态 5 级流水线 CPU。

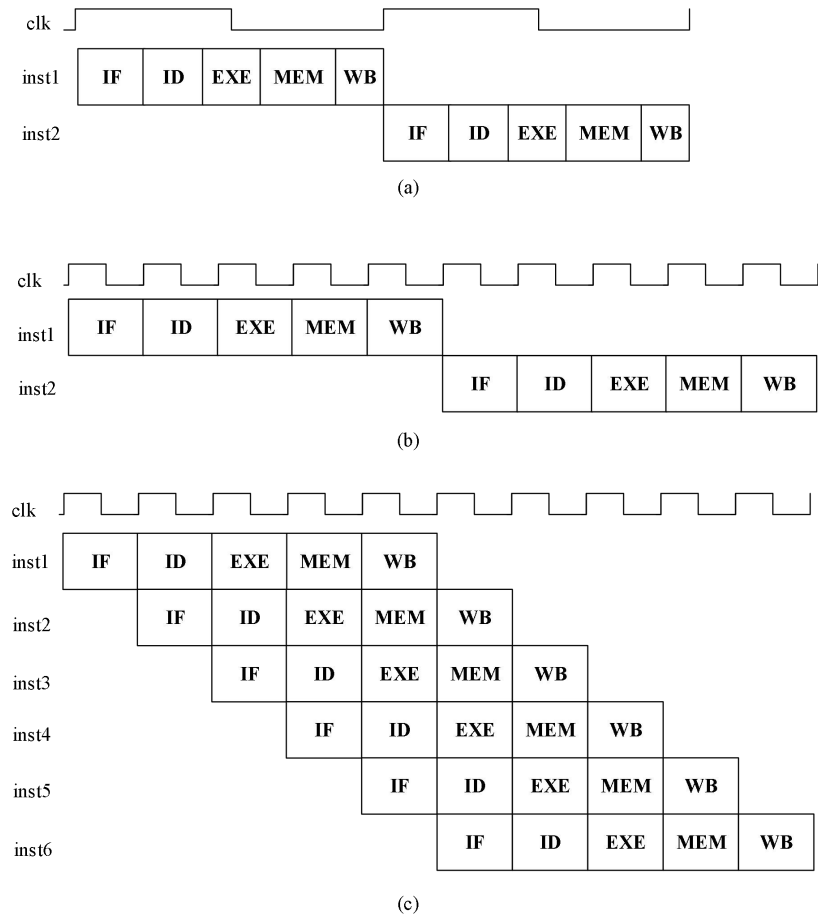


图 9.1 (a)单周期; (b)多周期; (c)5 级流水 CPU 时空图

2. 本次课程设计的设计框图基本同多周期 CPU 实验的设计框图。需要注意 5 个部件都是同时运转的，但对每条指令而言，依然是依次工作的，见图 9.2。

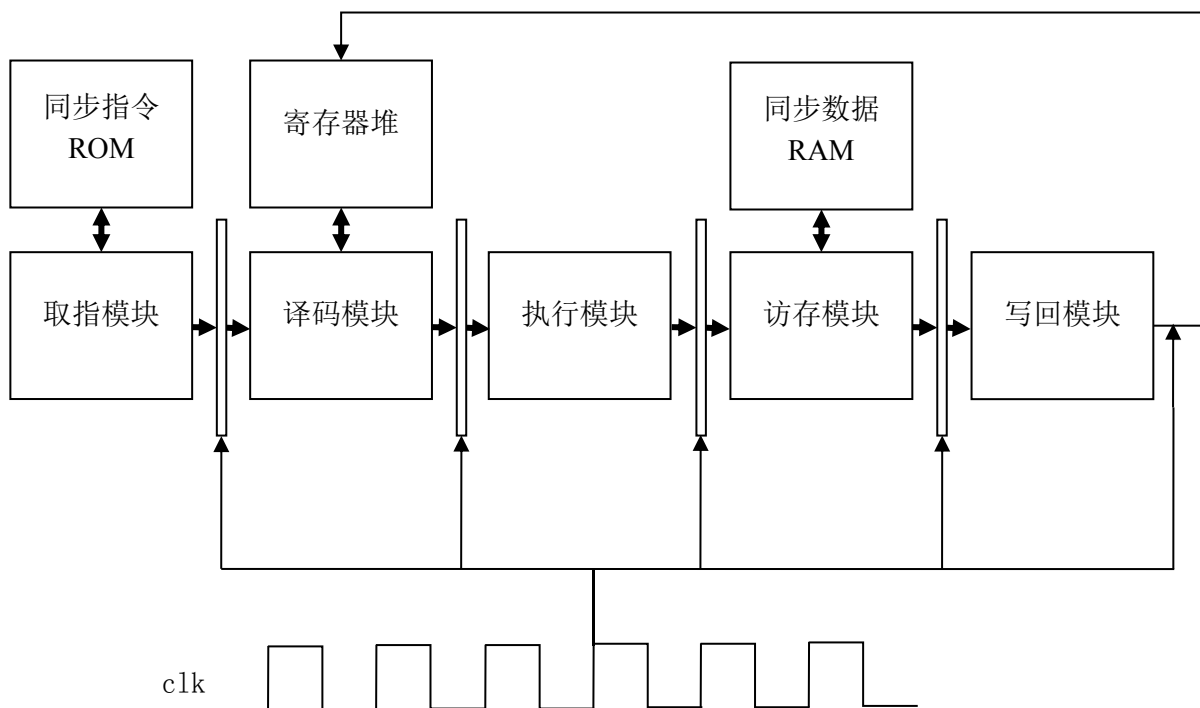


图 9.2 静态 5 级流水 CPU 的大致框图

3. 本次课程设计的关键和难点在于流水线的控制，比如一条指令何时可以从译码级进入执行级，一条指令何时需要堵在流水线中。本次课程设计暂不考虑前递技术，因此有数据相关时就需要堵在流水线中。

4. MIPS 架构中有延迟槽的设定，其本意是加快流水 CPU 的执行速度。在之前单周期和多周期 CPU 实验中未支持延迟槽，但在流水 CPU 中需要支持该设定，因为只有硬件支持了延迟槽技术，用通用编译器编译出来的 MIPS 二进制执行文件才能在自己设计的 CPU 中运行正确。

5. MIPS 架构中分支和跳转指令参与计算的 PC 值均为延迟槽指令对应的 PC (即分支跳转指令的 PC+4), 在本课程设计中尤其需要注意这一点。比如一条指令

“beq, r0, r0, #2” 在不考虑延迟槽的多周期 CPU 中，其跳转的目标地址为 beq 指令后面的第 2 条。而在考虑延迟槽的流水 CPU 中，其跳转的目标地址为 beq 指令后面的第 3 条（即延迟槽指令后面的第 2 条）。在编写测试程序时就需要注意分支跳转指令的偏移量。

6. 本次实验，同样需要完成表 9.1 和表 9.2 的填写。

表 9.1 mips 基础指令特性归纳表

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	目的寄存器	功能描述
R 型指令	addu rd, rs, rt	000000 _{rs} _{rt} _{rd} 000000 100001	[rs]	[rt]	rd	GPR[rd]=GPR[rs]+GPR[rt]
I 型指令	addiu rt,rs,imm	001001 _{rs} _{rt} _{imm}	[rs]	sign_ext(imm)	rt	GPR[rt]=GPR[rs]+sign_ext(imm)
J 型指令	j target	000010 target	next_pc	target		跳转, PC={next_pc[31:28],target,2'b00}

表 9.2 测试所用汇编程序描述表

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
00H	addiu \$1, \$0, #1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001

7. 根据设计的实验方案，使用 verilog 编写相应代码。

8. 对编写的代码进行仿真，得到正确的波形图。

9. 将以上设计作为一个单独的模块，设计一个外围模块去调用该模块，见图 9.3。外围模块中需调用封装好的 LCD 触摸屏模块，观察 CPU 的内部状态，比如 32 个寄存器的值，各级 PC 的值等。并且需要利用触摸功能输入特定数据 RAM 地址，从该 RAM 的调试端口读出数据显示在屏上，以达到实时观察数据存储器内部数据变化的效果。通过这些手段，可以在板上充分验证 CPU 的正确性。

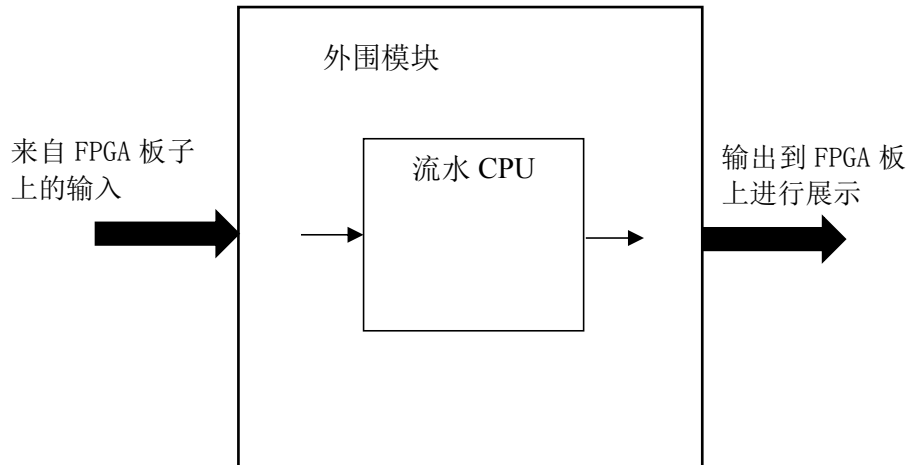


图 9.3 静态 5 级流水 CPU 设计实验的顶层模块大致框图

10. 将编写的代码进行综合布局布线，并下载到实验箱中的 FPGA 板子上进行演示。

注意：一般而言控制 CPU 运转的时钟是由 FPGA 板上的时钟输出提供的，但为了方便演示，我们需要在每一个时钟里查看一条指令的运算结果，故演示时理想的时钟是手动输入的，可以使用 FPGA 板上的脉冲开关代替时钟。

4 课程设计要求

1. 做好预习：

1) 复习好上一次多周期 CPU 的实验，归纳常用的 MIPS 指令，确定自己准备实现的 MIPS 指令，对其进行分析，完成表 9.1 的填写；

2) 认真学习流水线的概念，明白流水线的意义和架构，理解数据相关、控制相关和结构相关，尤其需要注意分支跳转指令及其延迟槽指令的处理；

3) 依据自己设计中实现的指令，编写一段不少于 50 行的汇编程序，要求包含所有实现的指令，完成表 9.2 的填写。要求标注出指令间存在的相关，指出 CPU 可能存在的阻塞；

4) 在多周期 CPU 实验的设计框图的基础上，完善课程设计的设计框图，即补充完善图 9.2；

5) 如果对 FPGA 板了解的话，可确定设计中与 FPGA 板上交互的接口，画出包含外围模块的整体设计框图，即补充完善图 9.3。

2. 实验实施：

1) 确认流水 CPU 的设计框图的正确性；

2) 编写 verilog 代码，将表 9.2 中自己编写的汇编程序翻译为二进制，以 coe 文件的方式初始化到指令 ROM 中；

3) 对该模块进行仿真，得出正确的波形，截图作为实验报告结果一项的材料，在仿真时需要将生成指令 ROM 时产生的.mif 文件拷贝到工程目录下，才能仿真成功；

4) 完成调用流水 CPU 的外围模块的设计，并编写代码；

5) 对代码进行综合布局布线下载到实验箱里 FPGA 板上，进行上板验证。

3. 实验检查：

1) 完成上板验证后，让指导老师或助教进行检查，进行现场演示。先解读表 9.2 中自己编写的汇编程序，然后采用手动输入时钟，每个周期查看 CPU 状态，按照检查人员的要求进行演示，检查指令运行结果的正确性，可对演示结果进行拍照作为实验报告结果一项的材料。

4. 实验报告的撰写：

1) 实验结束后，需按照规定的格式完成实验报告的撰写。

5 参考设计

5.1 实验原理

流水 CPU 实现的指令集是在多周期 CPU 的指令集上增加 9 条指令，主要是乘法指令，与 HI、LO、协处理器 0 (Coprocessor 0, cp0) 寄存器传送数据的指令以及特权指令，共实现了 45 条指令。表 9.3 为新增指令列表。表 9.4 为流水 CPU 所实现的指令的解析。

表 9.3 5 级流水线 CPU 新增的指令

指令名称	汇编指令	功能描述
有符号乘法	mult rs,rt	(HI,LO)=sign(GPR[rs])*sign(GPR[rt])
从 LO 寄存器取值	mflo rd	GPR[rd]=[LO]
从 HI 寄存器取值	mfhi rd	GPR[rd]=[HI]
向 LO 寄存器存值	mtlo rs	[LO]=GPR[rs]
向 HI 寄存器存值	mtthi rs	[HI]=GPR[rs]
从 cp0 寄存器取值	mfc0 rt,cs	GPR[rt]=CPR[cs]
向 cp0 寄存器存值	mtc0 rt,cd	CPR[cd]=GPR[rt]
系统调用	syscall	CPR[14.0]=PC, CPR[13.0][6:2]=01000, CPR[12.0][1]=1, PC=CPR[15.1]+0x180
异常返回	eret	CPR[12.0][1]=0, PC=CPR[14.0]

注：CPR 表示协处理器 0 寄存器，CPR[12.0]表示 STATUS 寄存器，CPR[13.0]表示 CAUSE 寄存器，CPR[14.0]表示 EPC 寄存器。CPR[15.1]为 EBASE 寄存器，syscall 指令会使用该寄存器确认 Exception 的入口地址，但在本参考设计中未实现该寄存器，并设置 Exception 的入口地址为 0 地址。

表 9.4 5 级流水线 CPU 实现的 mips 指令特性归纳

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
R 型指令	addu rd,rs,rt	000000 rs rt rd 00000 100001	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] + GPR[rt]$
	subu rd,rs,rt	000000 rs rt rd 00000 100011	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] - GPR[rt]$
	slt rd,rs,rt	000000 rs rt rd 00000 101010	[rs]	[rt]		rd	$GPR[rd] = (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$
	sltu rd,rs,rt	000000 rs rt rd 00000 101011	[rs]	[rt]		rd	$GPR[rd] = (\text{zero}(GPR[rs]) < \text{zero}(GPR[rt]))$
	jlr rs	000000 rs 00000 1111 00000 001001	[rs]			31	$GPR[31] = PC, PC = GPR[rs]$
	jr rs	000000 rs 000000 0000 00000 001000	[rs]				$PC = GPR[rs]$
	and rd,rs,rt	000000 rs rt rd 00000 100100	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \& GPR[rt]$
	nor rd,rs,rt	000000 rs rt rd 00000 100111	[rs]	[rt]		rd	$GPR[rd] = \sim(GPR[rs] GPR[rt])$
	or rd,rs,rt	000000 rs rt rd 00000 100101	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] GPR[rt]$
	xor rd,rs,rt	000000 rs rt rd 00000 100110	[rs]	[rt]		rd	$GPR[rd] = GPR[rs] \wedge GPR[rt]$
	sll rd,rt,shf	000000 00000 rt rd shf 000000		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \ll \text{shf}$
	sllv rd,rt,rs	000000 rs rt rd 00000 000100	[rs]	[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \ll (GPR[rs] \% 32)$
	sra rd,rt,shf	000000 00000 rt rd shf 000011		[rt]		rd	$GPR[rd] = \text{sign}(GPR[rt]) \gg \text{shf}$
	srav rd,rt,rs	000000 rs rt rd 00000 000111	[rs]	[rt]		rd	$GPR[rd] = \text{sign}(GPR[rt]) \gg (GPR[rs] \% 32)$
	srl rd,rt,shf	000000 00000 rt rd shf 000010		[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \gg \text{shf}$
	srlv rd,rt,rs	000000 rs rt rd 00000 000110	[rs]	[rt]		rd	$GPR[rd] = \text{zero}(GPR[rt]) \gg GPR[rs]$
	mult rs,rt	000000 rs rt 000000 00000 011000	[rs]	[rt]		(HI,LO)	$(HI,LO) = \text{sign}(GPR[rs]) * \text{sign}(GPR[rt])$
	mflo rd	000000 00000000 00 rd 00000 010010	[LO]			rd	$GPR[rd] = [LO]$
	mfhi rd	000000 00000000 00 rd 00000 010000	[HI]			rd	$GPR[rd] = [HI]$
	mtlo rs	000000 rs 000000 00000000 010011	[rs]			LO	$[LO] = GPR[rs]$
	mthi rs	000000 rs 000000 00000000 010001	[rs]			HI	$[HI] = GPR[rs]$
I 型指令	addiu rt,rs,imm	001001 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = GPR[rs] + \text{sign_ext}(imm)$
	slti rt,rs,imm	001010 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = (\text{sign}(GPR[rs]) < \text{sign_ext}(imm))$
	sltiu rt,rs,imm	001011 rs rt imm	[rs]	sign_ext(imm)		rt	$GPR[rt] = (\text{zero}(GPR[rs]) < \text{sign_ext}(imm))$

表 9.4 5 级流水线 CPU 实现的 mips 指令特性归纳（续）

指令类型	汇编指令	指令码	源操作数 1	源操作数 2	源操作数 3	目的寄存器	功能描述
I 型指令	beq rs,rt,offset	000100 rs rt offset	[rs]	[rt]			if GPR[rs]=GPR[rt] then PC= next_pc+ sign_ext (offset)<<2
	bgez rs,offset	000001 rs 00001 offset	[rs]				if GPR[rs]≥0 then PC= next_pc + sign_ext (offset)<<2
	bgtz rs,offset	000111 rs 00000 offset	[rs]				if GPR[rs]>0 then PC= next_pc + sign_ext (offset)<<2
	blez rs,offset	000110 rs 00000 offset	[rs]				if GPR[rs]≤0 then PC= next_pc + sign_ext (offset)<<2
	bltz rs,offset	000001 rs 00000 offset	[rs]				if GPR[rs]<0 then PC= next_pc + sign_ext (offset)<<2
	bne rs,rt,offset	000101 rs rt offset	[rs]	[rt]			if GPR[rs]≠GPR[rt] then PC= next_pc + sign_ext (offset)<<2
	lw rt,offset(b)	100011 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=Mem[GPR[b]+sign_ext (offset)]
	sw rt,offset(b)	101011 b rt offset	[b]	sign_ext (offset)	[rt]		Mem[GPR[b]+sign_ext (offset)]=GPR[rt]
	lb rt,offset(b)	100000 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=sign(Mem[GPR[b]+sign_ext (offset)])
	lbu rt,offset(b)	100100 b rt offset	[b]	sign_ext (offset)		rt	GPR[rt]=zero(Mem[GPR[b]+sign_ext (offset)])
	sb rt,offset(b)	101000 b rt offset	[b]	sign_ext (offset)	[rt]		Mem[GPR[b]+sign_ext (offset)]=GPR[rt]
	andi rt,rs,imm	001100 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]&zero_ext (imm)
	lui rt,imm	001111 00000 rt imm		{imm, 16'd0}		rt	GPR[rt]= {imm, 16'd0}
	ori rt,rs,imm	001101 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs] zero_ext (imm)
	xori rt,rs,imm	001110 rs rt imm	[rs]	zero_ext (imm)		rt	GPR[rt]=GPR[rs]^zero_ext (imm)
J 型指令	j target	000010 target					[PC]={next_pc[31:28],target<<2}
	jal target	000011 target					GPR[31]=PC,PC={next_pc[31:28],target<<2}
cp0 指令	mfc0 rt,cs	010000 00000 rt cs 00000000 sel	CPR[cs.sel]			rt	GPR[rt]=CPR[cs.sel]
	mtc0 rt,cd	010000 00100 rt cd 00000000 sel		GPR[rt]		CPR[cd.sel]	CPR[cd.sel]=GPR[rt]
	syscall	000000 code 001100					CPR[14.0]=PC,CPR[13.0][6:2]=01000,CPR[12.0][1]=1,PC=CPR[15.1]+0x180 并跳转
	Eret	010000 1 0000000000000000 011000					CPR[12.0][1]=0,PC=CPR[14.0]并跳转

注：分支跳转指令参与运算的不在是当前 PC 值，而是 next_pc, 即当前 PC+4，即延迟槽指令的 PC。

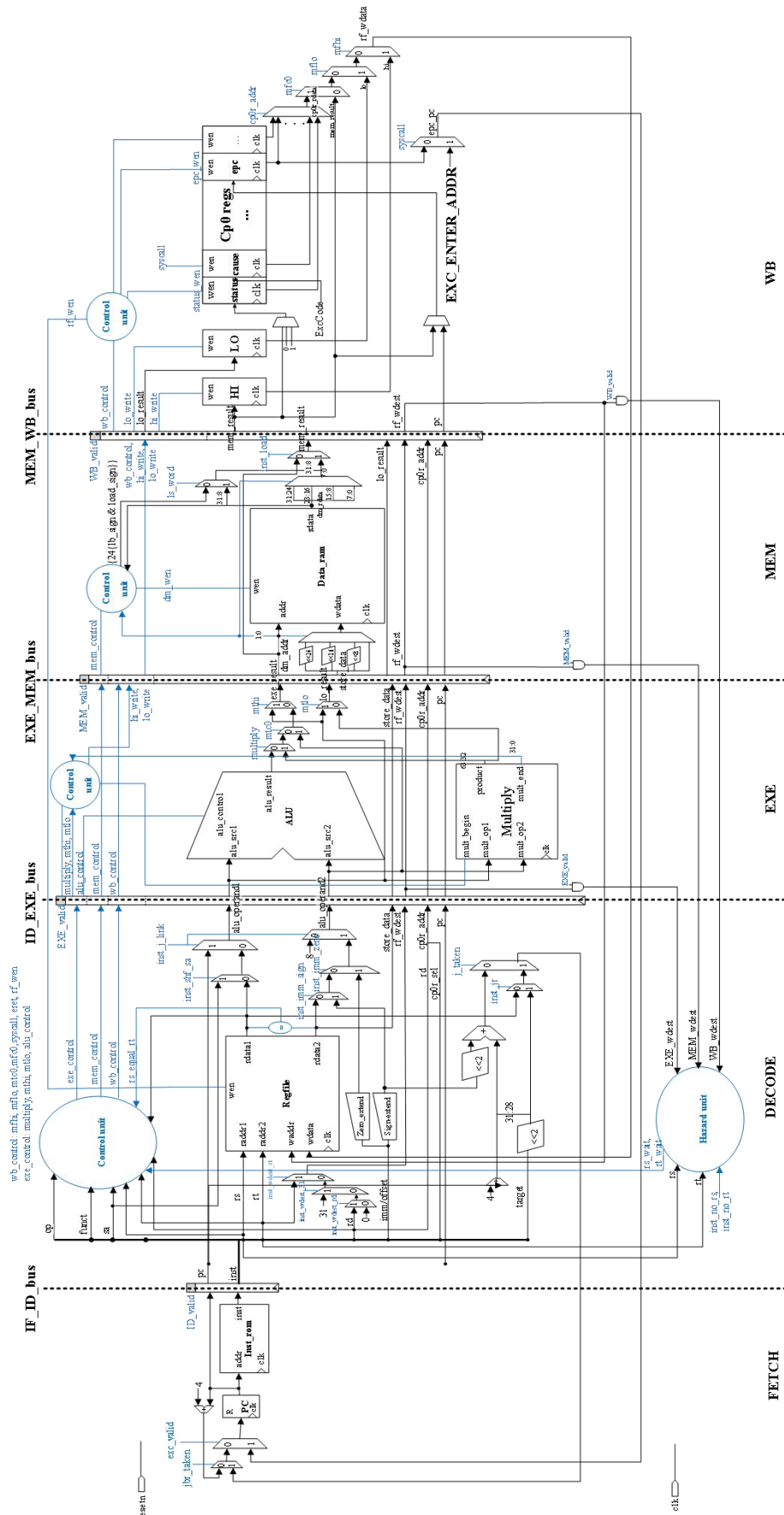


图 9.4 5 级流水线 CPU 的实现框图

利用 5 级流水 CPU 实现的 45 条指令，可以编写一段汇编程序，用于测试实现的静态 5 级流水 CPU 的正确性。表 9.5 给出了一个测试程序。

表 9.5 5 级流水线 CPU 测试所用汇编程序详述

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
Exception 入口地址，在 SYSCALL 指令执行后进入此处执行				
00H	sw \$1,#0(\$0)	Mem[0000_0000H] = 0000_0008H	AC010000	1010_1100_0000_0001_0000_0000_0000_0000
04H	sw \$2,#4(\$0)	Mem[0000_0004H] = 0000_0010H	AC020004	1010_1100_0000_0010_0000_0000_0000_0100
08H	sw \$3,#8(\$0)	Mem[0000_0008H] = 0000_0011H	AC030008	1010_1100_0000_0011_0000_0000_0000_1000
0CH	sw \$4,#12(\$0)	Mem[0000_000CH] = 0000_0004H	AC04000C	1010_1100_0000_0100_0000_0000_0000_1100
10H	sw \$5,#16(\$0)	Mem[0000_0010H] = 0000_000DH	AC050010	1010_1100_0000_0101_0000_0000_0001_0000
14H	sw \$6,#24(\$0)	Mem[0000_0018H] = FFFF_FFE2H	AC060018	1010_1100_0000_0110_0000_0000_0001_1000
18H	sw \$7,#112(\$0)	Mem[0000_0070H] = FFFF_FFF3H	AC070070	1010_1100_0000_0111_0000_0000_0111_0000
1CH	sw \$25,#116(\$0)	Mem[0000_0074H] = 0000_0001H	AC190074	1010_1100_0001_1001_0000_0000_0111_0100
20H	sw \$13,#24(\$0)	Mem[0000_0078H] = 0000_0000H	AC0D0078	1010_1100_0000_1101_0000_0000_0111_1000
24H	mfc0 \$1, cp0(14.0)	[\$1] = 0000_0104H	40017000	0100_0000_0000_0001_0111_0000_0000_0000
28H	addiu \$1,\$1,#4	[\$1] = 0000_0108H	24210004	0010_0100_0010_0001_0000_0000_0000_0100
2CH	mtc0 \$1, cp0(14.0)	cp0(14.0) = 0000_0108H	40817000	0100_0000_1000_0001_0111_0000_0000_0000
30H	eret	返回 108H	42000018	0100_0010_0000_0000_0000_0000_0001_1000
CPU 复位地址 0000_0034H				
34H	addiu \$1, \$0,#1	[\$1] = 0000_0001H	24010001	0010_0100_0000_0001_0000_0000_0000_0001
38H	sll \$2, \$1,#4	[\$2] = 0000_0010H	00011100	0000_0000_0000_0001_0001_0001_0000_0000
3CH	addu \$3, \$2,\$1	[\$3] = 0000_0011H	00411821	0000_0000_0100_0001_0001_1000_0010_0001
40H	srl \$4, \$2,#2	[\$4] = 0000_0004H	00022082	0000_0000_0000_0010_0010_0000_1000_0010
44H	slti \$25,\$4,#5	[\$25] = 0000_0001H	28990005	0010_1000_1001_1001_0000_0000_0000_0101
48H	bgez \$25,#14	跳转到 84H	0721000E	0000_0111_0010_0001_0000_0000_0000_1110
4CH	subu \$5, \$3,\$4	[\$5] = 0000_000DH	00642823	0000_0000_0110_0100_0010_1000_0010_0011
50H	sw \$5, #20(\$0)	Mem[0000_0014H] = 0000_000DH	AC050014	1010_1100_0000_0101_0000_0000_0001_0100
54H	nor \$6, \$5,\$2	[\$6] = FFFF_FFE2H	00A23027	0000_0000_1010_0010_0011_0000_0010_0111
58H	or \$7, \$6,\$3	[\$7] = FFFF_FFF3H	00C33825	0000_0000_1100_0011_0011_1000_0010_0101
5CH	xor \$8, \$7,\$6	[\$8] = 0000_0011H	00E64026	0000_0000_1110_0110_0100_0000_0010_0110
60H	beq \$8, \$3,#2	跳转到 6CH	11030002	0001_0001_0000_0011_0000_0000_0000_0010
64H	sw \$8, #28(\$0)	Mem[0000_001CH] = 0000_0011H	AC08001C	1010_1100_0000_1000_0000_0000_0001_1100
68H	slt \$9, \$1,\$2	不执行	0022482A	0000_0000_0010_0010_0100_1000_0010_1010
6CH	lw \$10,#28(\$0)	[\$10] = 0000_0011H	8C0A001C	1000_1100_0000_1010_0000_0000_0001_1100
70H	bne \$10,\$5,#2	跳转到 7CH	15450002	0001_0101_0100_0101_0000_0000_0000_0010

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
74H	and \$11,\$2,\$1	[\$11] = 0000_0000H	00415824	0000_0000_0100_0001_0101_1000_0010_0100
78H	sw \$11,#28(\$0)	不执行	AC0B001C	1010_1100_0000_1011_0000_0000_0001_1100
7CH	jal #38	跳转到 98H, [\$31] = 0000_0084H	0C000026	0000_1100_0000_0000_0000_0000_0010_0110
80H	sw \$4,#16(\$0)	Mem[0000_0010H] = 0000_0004H	AC040010	1010_1100_0000_0100_0000_0000_0001_0000
84H	lui \$12,#12	[\$12] = 000C_0000H	3C0C000C	0011_1100_0000_1100_0000_0000_0000_1100
88H	sra \$26,\$12,\$2	[\$26] = 0000_000CH	004CD007	0000_0000_0100_1100_1101_0000_0000_0111
8CH	addiu \$27,\$26,#68	[\$27] = 0000_0050H	275B0044	0010_0111_0101_1011_0000_0000_0100_0100
90H	jalr \$27	跳转到 50H, [\$31] = 0000_0098H	0360F809	0000_0011_0110_0000_1111_1000_0000_1001
94H	addiu \$1,\$0,#8	[\$1] = 0000_0008H	24010008	0010_0100_0000_0001_0000_0000_0000_1000
98H	sb \$26,#5(\$3)	MEM[0000_0014H] = 000C_000DH	A07A0005	1010_0000_0111_1010_0000_0000_0000_0101
9CH	sltu \$13,\$10,\$3	[\$13] = 0000_0000H	0143682B	0000_0001_0100_0011_0110_1000_0010_1011
A0H	bgtz \$13,#2	不跳转	1DA00002	0001_1101_1010_0000_0000_0000_0000_0010
A4H	sllv \$14,\$6,\$4	[\$14] = FFFF_FE20H	00867004	0000_0000_1000_0110_0111_0000_0000_0100
A8H	sra \$15,\$14,#2	[\$15] = FFFF_FF88H	000E7883	0000_0000_0000_1110_0111_1000_1000_0011
ACH	srlv \$16,\$15,\$1	[\$16] = 00FF_FFFFH	002F8006	0000_0000_0010_1111_1000_0000_0000_0110
B0H	blez \$16,#7	不跳转	1A000007	0001_1010_0000_0000_0000_0000_0000_0111
B4H	sra \$16,\$15,\$1	[\$16] = FFFF_FFFFH	002F8007	0000_0000_0010_1111_1000_0000_0000_0111
B8H	bltz \$16,#6	跳转到 D4H	06000006	0000_0110_0000_0000_0000_0000_0000_0110
BCH	sll \$11,\$26,#4	[\$11] = 0000_00C0H	001A5900	0000_0000_0001_1010_0101_1001_0000_0000
C0H	lw \$28,#3(\$10)	[\$28]=000C_000DH / 000C_880DH	8D5C0003	1000_1101_0101_1100_0000_0000_0000_0011
C4H	bne \$28,\$29,#7	不跳转/跳转 E4H	179D0007	0001_0111_1001_1101_0000_0000_0000_0111
C8H	sb \$15,#8(\$5)	Mem[0000_0014H] = 000C_8800H	A0AF0008	1010_0000_1010_1111_0000_0000_0000_1000
CCH	lb \$18,#8(\$5)	[\$18] = FFFF_FF88H	80B20008	1000_0000_1011_0010_0000_0000_0000_1000
D0H	lbu \$19,#8(\$5)	[\$19] = 0000_0088H	90B30008	1001_0000_1011_0011_0000_0000_0000_1000
D4H	sltiu \$24,\$15,#0xFFFF	[\$24] = 0000_0001H	2DF8FFFF	0010_1101_1111_1000_1111_1111_1111_1111
D8H	or \$29,\$12,\$5	[\$29] = 000C_000DH	0185E825	0000_0001_1000_0101_1110_1000_0010_0101
DCH	jr \$11	跳转到 C0H	01600008	0000_0001_0110_0000_0000_0000_0000_1000
E0H	andi \$20,\$15,#0xFFFF	[\$20] = 0000_FF88H	31F4FFFF	0011_0001_1111_0100_1111_1111_1111_1111
E4H	ori \$21,\$15,#0xFFFF	[\$21] = FFFF_FFFFH	35F5FFFF	0011_0101_1111_0101_1111_1111_1111_1111
E8H	xori \$22,\$15,#0xFFFF	[\$22] = FFFF_0077H	39F6FFFF	0011_1001_1111_0110_1111_1111_1111_1111
ECH	mult \$12,\$29	[HI] = 0000_0090H, [LO] = 009C_0000H	019D0018	0000_0001_1001_1101_0000_0000_0001_1000
F0H	mflo \$23	[\$23] = 009C_0000H	0000B812	0000_0000_0000_0000_1011_1000_0001_0010
F4H	mfhi \$30	[\$30] = 0000_0090H	0000F010	0000_0000_0000_0000_1111_0000_0001_0000
F8H	mtlo \$26	[LO] = 0000_000CH	03400013	0000_0011_0100_0000_0000_0000_0001_0011

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
FCH	mthi \$27	[HI] = 0000_0050H	03600011	0000_0011_0110_0000_0000_0001_0001
100H	mtc0 \$0,c14	cp0[14.0] = 0000_0000H	40807000	0100_0000_1000_0000_0111_0000_0000_0000
104H	syscall	cp0[14.0] =0000_0104H, cp0[13.0][6..2] =01000B, cp0[12.0][1]=1, 跳转 Exception 入口 地址, 00H	0000000C	0000_0000_0000_0000_0000_0000_0000_1100
108H	mfc0 \$2, cp0(14.0)	[\$2] = 0000_0108H	40027000	0100_0000_0000_0010_0111_0000_0000_0000
10CH	mfc0 \$3, cp0(13.0)	[\$3] = 0000_0020H	40036800	0100_0000_0000_0011_0110_1000_0000_0000
110H	mfc0 \$4, cp0(12.0)	[\$4] = 0000_0000H	40046000	0100_0000_0000_0100_0110_0000_0000_0000
114H	addiu \$1, \$0,#32	[\$1] = 0000_0020H	24010020	0010_0100_0000_0001_0000_0000_0010_0000
118H	slt \$17,\$15,\$14	[\$17] = 0000_0000H	01EE882A	0000_0001_1110_1110_1000_1000_0010_1010
11CH	lui \$17,#1234H	[\$17] = 1234_0000H	3C111234	0011_1100_0001_0001_0001_0010_0011_0100
120H	addiu \$17,\$17,#5678	[\$17] = 1234_5678H	26315678	0010_0110_0011_0001_0101_0110_0111_1000
124H	sw \$17,#0(\$1)	Mem[0000_0020H] =1234_5678H / Mem[0000_0024H] =2345_6780H / Mem[0000_0028H] =3456_7800H / Mem[0000_002CH] =4567_8000H / Mem[0000_0030H] =5678_0000H / Mem[0000_0034H] =6780_0000H / Mem[0000_0038H] =7800_0000H	AC310000	1010_1100_0011_0001_0000_0000_0000_0000
128H	sll \$17,\$17,#4	[\$17] = 2345_6780H / 3456_7800H / 4567_8000H / 5678_0000H / 6780_0000H / 7800_0000H / 8000_0000H	00118900	0000_0000_0001_0001_1000_1001_0000_0000
12CH	bgtz \$17,#-3	跳转到 124H /跳转/ 跳转/跳转/跳转/跳转 124H/不跳转	1E20FFFD	0001_1110_0010_0000_1111_1111_1111_1101
130H	addiu \$1,\$1,#4	[\$1] = 0000_0024H / 0000_0028H / 0000_002CH / 0000_0030H / 0000_0034H / 0000_0038H / 0000_003CH	24210004	0010_0100_0010_0001_0000_0000_0000_0100
134H	addiu \$2,\$0,#60	[\$2] = 0000_003CH	2402003C	0010_0100_0000_0010_0000_0000_0011_1100
138H	lw \$17,#-28(\$1)	[\$17] = 1234_5678H	8C31FFE4	1000_1100_0011_0001_1111_1111_1110_0100

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
13CH	srl \$17,\$17,#4	[\$17] = 0123_4567H / 0012_3456H / 0001_2345H / 0000_1234H / 0000_0123H / 0000_0012H / 0000_0001H / 0000_0000H	00118902	0000_0000_0001_0001_1000_1001_0000_0010
140H	sw \$17,#0(\$2)	Mem[0000_003CH] =0123_4567H / Mem [0000_0040H] =0012_3456H / Mem [0000_0044H] =0001_2345H / Mem [0000_0048H] =0000_1234H / Mem [0000_004CH] =0000_0123H / Mem [0000_0050H] =0000_0012H / Mem [0000_0054H] =0000_0001H / Mem [0000_0058H] =0000_0000H	AC510000	1010_1100_0101_0001_0000_0000_0000_0000
144H	bne \$17,\$0,#-3	跳转到 13CH /跳转/ 跳转/跳转/跳转/跳转/ /跳转 13CH/不跳转	1620FFFD	0001_0110_0010_0000_1111_1111_1111_1101
148H	addiu \$2,\$2,#4	[\$2] = 0000_0040H / 0000_0044H / 0000_0048H / 0000_004CH / 0000_0050H / 0000_0054H / 0000_0058H / 0000_005CH	24420004	0010_0100_0100_0010_0000_0000_0000_0100
14CH	addiu \$6,\$0,#68	[\$6] = 0000_0044H	24060044	0010_0100_0000_0110_0000_0000_0100_0100
150H	addiu \$7,\$0,#100	[\$7] = 0000_0064H	24070064	0010_0100_0000_0111_0000_0000_0110_0100
154H	lw \$3,#-28(\$1)	[\$3] = 1234_5678H / 2345_6780H / 3456_7800H / 4567_8000H / 5678_0000H / 6780_0000H / 7800_0000H	8C23FFE4	1000_1100_0010_0011_1111_1111_1110_0100
158H	lw \$4,#-4(\$2)	[\$4] = 0000_0000H / 0000_0001H / 0000_0012H / 0000_0123H / 0000_1234H / 0001_2345H / 0012_3456H	8C44FFFC	1000_1100_0100_0100_1111_1111_1111_1100
15CH	or \$5,\$3,\$4	[\$5] = 1234_5678H / 2345_6781H / 3456_7812H / 4567_8123H / 5678_1234H / 6781_2345H / 7812_3456H	00642825	0000_0000_0110_0100_0010_1000_0010_0101

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
160H	sb \$5,#0(\$7)	Mem[0000_0064H] =0000_0078H / Mem[0000_0064H] =0000_8178H / Mem[0000_0064H] =0012_8178H / Mem[0000_0064H] =2312_8178H / Mem[0000_0068H] =0000_0034H / Mem[0000_0068H] =0000_4534H / Mem[0000_0068H] =0056_4534H	A0E50000	1010_0000_1110_0101_0000_0000_0000
164H	addiu \$7,\$7,#1	[\$7] = 0000_0065H / 0000_0066H / 0000_0067H / 0000_0068H / 0000_0069H / 0000_006AH / 0000_006BH	24E70001	0010_0100_1110_0111_0000_0000_0001
168H	addiu \$1,\$1,#4	[\$1] = 0000_0040H / 0000_0044H / 0000_0048H / 0000_004CH / 0000_0050H / 0000_0054H / 0000_0058H	24210004	0010_0100_0010_0001_0000_0000_0100
16CH	bne \$2,\$6,#-7	跳转到 154H/跳转/跳 转/跳转/跳转/跳转 154H/不跳转	1446FFF9	0001_0100_0100_0110_1111_1111_1001
170H	ADDIU \$2,\$2,#-4	[\$2] = 0000_0058H / 0000_0054H / 0000_0050H / 0000_004CH / 0000_0048H / 0000_0044H / 0000_0040H	2442FFFC	0010_0100_0100_0010_1111_1111_1100
174H	addiu \$9,\$0,#100	[\$9] = 0000_0064H	24090064	0010_0100_0000_1001_0000_0000_0110_0100
178H	lbu \$9,#3(\$9)	[\$9] = 0000_0023H	91290003	1001_0001_0010_1001_0000_0000_0000_0011
17CH	addiu \$13,\$0,#104	[\$13] = 0000_0068H	240D0068	0010_0100_0000_1101_0000_0000_0110_1000
180H	lw \$13,#0(\$13)	[\$13] = 0056_4534H	8DAD0000	1000_1101_1010_1101_0000_0000_0000_0000
184H	ll \$9,\$9,#24	[\$9] = 2300_0000H	00094E00	0000_0000_0000_1001_0100_1110_0000_0000
188H	xori \$13,\$13,#9	[\$13] = 0056_453DH	39AD0009	0011_1001_1010_1101_0000_0000_0000_1001
18CH	sw \$13,#1(\$7)	MEM[0000_006CH]= 0056_453DH	ACED0001	1010_1100_1110_1101_0000_0000_0000_0001
190H	lw \$1,#0(\$0)	[\$1] = 0000_0008H	8C010000	1000_1100_0000_0001_0000_0000_0000_0000
194H	lw \$2,#4(\$0)	[\$2] = 0000_0010H	8C020004	1000_1100_0000_0010_0000_0000_0000_0100
198H	lw \$3,#8(\$0)	[\$3] = 0000_0011H	8C030008	1000_1100_0000_0011_0000_0000_0000_1000
19CH	lw \$4,#12(\$0)	[\$4] = 0000_0004H	8C04000C	1000_1100_0000_0100_0000_0000_0000_1100
1A0H	lw \$5,#16(\$0)	[\$5] = 0000_000DH	8C050010	1000_1100_0000_0101_0000_0000_0001_0000
1A4H	lw \$6,#24(\$0)	[\$6] = FFFF_FFE2H	8C060018	1000_1100_0000_0110_0000_0000_0001_1000

表 9.5 5 级流水线 CPU 测试所用汇编程序详述（续）

指令地址	汇编指令	结果描述	机器指令的机器码	
			16 进制	二进制
1A8H	lw \$7,#112(\$0)	[\$7] = FFFF_FFF3H	8C070070	1000_1100_0000_0111_0000_0000_0111_0000
1ACH	lw \$25,#116(\$0)	[\$25] = 0000_0001H	8C190074	1000_1100_0001_1001_0000_0000_0111_0100
1B0H	lw \$13,#120(\$0)	[\$13] = 0000_0000H	8C0D0078	1000_1100_0000_1101_0000_0000_0111_1000
1B4H	j #34H	跳转到 34H	0800000D	0000_1000_0000_0000_0000_0000_0000_1101

5.2 实验顶层模块框图

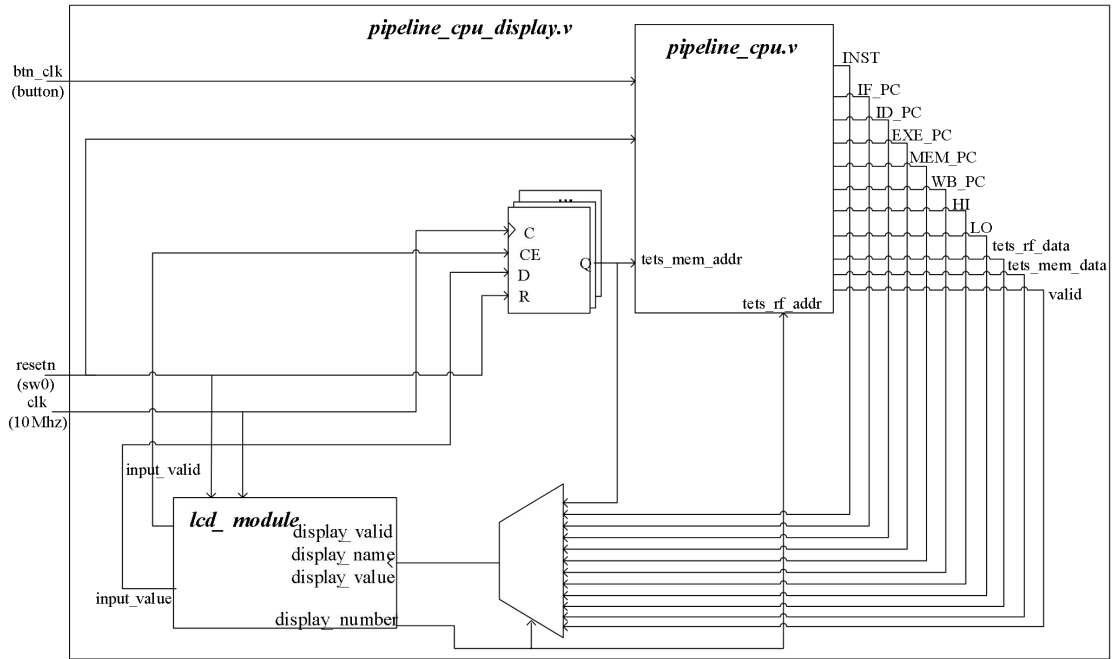


图 9.5 5 级流水线 CPU 参考设计的顶层模块框图

5.3 源代码

pipeline_cpu_display.v 源代码:

```
1 `timescale 1ns / 1ps
2 //*****
3 // > 文件名: pipeline_cpu_display.v
4 // > 描述 : 五级流水 CPU 显示模块, 调用 FPGA 板上的 IO 接口和触摸屏
5 // > 作者 : LOONGSON
6 // > 日期 : 2016-04-14
7 //*****
8 module pipeline_cpu_display( // 多周期 cpu
9     //时钟与复位信号
10     input clk,
11     input resetn, //后缀"n"代表低电平有效
12
13     //脉冲开关, 用于产生脉冲 clk, 实现单步执行
14     input btn_clk,
15
16     //触摸屏相关接口, 不需要更改
17     output lcd_rst,
```

```
18     output lcd_cs,
19     output lcd_rs,
20     output lcd_wr,
21     output lcd_rd,
22     inout[15:0] lcd_data_io,
23     output lcd_bl_ctr,
24     inout ct_int,
25     inout ct_sda,
26     output ct_scl,
27     output ct_rstn
28 );
29 //-----{时钟和复位信号}begin
30 //不需要更改，用于单步调试
31     wire cpu_clk;      //单周期 CPU 里使用脉冲开关作为时钟，以实现单步执行
32     reg btn_clk_r1;
33     reg btn_clk_r2;
34     always @(posedge clk)
35     begin
36         if (!resetn)
37         begin
38             btn_clk_r1<= 1'b0;
39         end
40         else
41         begin
42             btn_clk_r1 <= ~btn_clk;
43         end
44
45         btn_clk_r2 <= btn_clk_r1;
46     end
47
48     wire clk_en;
49     assign clk_en = !resetn || (!btn_clk_r1 && btn_clk_r2);
50     BUFGCE cpu_clk_cg(.I(clk),.CE(clk_en),.O(cpu_clk));
51 //-----{时钟和复位信号}end
52
53 //-----{调用多周期 CPU 模块}begin
54     //用于在 FPGA 板上显示结果
55     wire [ 4:0] rf_addr;    //扫描寄存器堆的地址
56     wire [31:0] rf_data;    //寄存器堆从调试端口读出的数据
57     reg  [31:0] mem_addr;    //要观察的内存地址
58     wire [31:0] mem_data;    //内存地址对应的数据
59     wire [31:0] IF_pc;       //IF 模块的 PC
60     wire [31:0] IF_inst;     //IF 模块取出的指令
61     wire [31:0] ID_pc;       //ID 模块的 PC
62     wire [31:0] EXE_pc;      //EXE 模块的 PC
63     wire [31:0] MEM_pc;      //MEM 模块的 PC
64     wire [31:0] WB_pc;       //WB 模块的 PC
65     wire [31:0] cpu_5_valid; //展示 CPU5 级的 valid 信号
66     wire [31:0] HI_data;     //展示 HI 寄存器的值
67     wire [31:0] LO_data;     //展示 LO 寄存器的值
68     pipeline_cpu cpu(
```

```
69         .clk      (cpu_clk ),
70         .resetn   (resetn  ),
71
72         .rf_addr  (rf_addr ),
73         .mem_addr (mem_addr),
74         .rf_data  (rf_data ),
75         .mem_data (mem_data),
76         .IF_pc    (IF_pc   ),
77         .IF_inst  (IF_inst ),
78         .ID_pc    (ID_pc   ),
79         .EXE_pc   (EXE_pc  ),
80         .MEM_pc   (MEM_pc  ),
81         .WB_pc    (WB_pc   ),
82         .cpu_5_valid (cpu_5_valid),
83         .HI_data  (HI_data ),
84         .LO_data  (LO_data ),
85     );
86 //-----{调用单周期 CPU 模块}end
87
88 //-----{调用触摸屏模块}begin-----//
89 //-----{实例化触摸屏}begin
90 //此小节不需要更改
91     reg      display_valid;
92     reg [39:0] display_name;
93     reg [31:0] display_value;
94     wire [5 :0] display_number;
95     wire      input_valid;
96     wire [31:0] input_value;
97
98     lcd_module lcd_module(
99         .clk      (clk      ),    //10Mhz
100        .resetn   (resetn   ),
101
102        //调用触摸屏的接口
103        .display_valid (display_valid ),
104        .display_name  (display_name  ),
105        .display_value (display_value ),
106        .display_number (display_number),
107        .input_valid   (input_valid   ),
108        .input_value   (input_value   ),
109
110        //lcd 触摸屏相关接口，不需要更改
111        .lcd_rst      (lcd_rst      ),
112        .lcd_cs       (lcd_cs       ),
113        .lcd_rs       (lcd_rs       ),
114        .lcd_wr       (lcd_wr       ),
115        .lcd_rd       (lcd_rd       ),
116        .lcd_data_io  (lcd_data_io  ),
117        .lcd_bl_ctr   (lcd_bl_ctr   ),
118        .ct_int       (ct_int       ),
119        .ct_sda       (ct_sda       ),
```

```
120         .ct_scl          (ct_scl          ),
121         .ct_rstn         (ct_rstn         )
122     );
123 //-----{实例化触摸屏}end
124
125 //-----{从触摸屏获取输入}begin
126 //根据实际需要输入的数修改此小节,
127 //建议对每一个数的输入, 编写单独一个 always 块
128     always @(posedge clk)
129     begin
130         if (!resetn)
131         begin
132             mem_addr <= 32'd0;
133         end
134         else if (input_valid)
135         begin
136             mem_addr <= input_value;
137         end
138     end
139     assign rf_addr = display_number-6'd13;
140 //-----{从触摸屏获取输入}end
141
142 //-----{输出到触摸屏显示}begin
143 //根据需要显示的数修改此小节,
144 //触摸屏上共有 44 块显示区域, 可显示 44 组 32 位数据
145 //44 块显示区域从 1 开始编号, 编号为 1~44,
146     always @(posedge clk)
147     begin
148         if (display_number >6'd12 && display_number <6'd45 )
149         begin //块号 5~36 显示 32 个通用寄存器的值
150             display_valid <= 1'b1;
151             display_name[39:16] <= "REG";
152             display_name[15: 8] <= {4'b0011,3'b000,rf_addr[4]};
153             display_name[7 : 0] <= {4'b0011,rf_addr[3:0]};
154             display_value      <= rf_data;
155         end
156         else
157         begin
158             case(display_number)
159             6'd1 : //显示 IF 模块的 PC
160             begin
161                 display_valid <= 1'b1;
162                 display_name  <= "IF_PC";
163                 display_value <= IF_pc;
164             end
165             6'd2 : //显示 IF 模块的指令
166             begin
167                 display_valid <= 1'b1;
168                 display_name  <= "IF_IN";
169                 display_value <= IF_inst;
170             end
```

```
171      6'd3 : //显示 ID 模块的 PC
172      begin
173          display_valid <= 1'b1;
174          display_name  <= "ID_PC";
175          display_value <= ID_pc;
176      end
177      6'd4 : //显示 EXE 模块的 PC
178      begin
179          display_valid <= 1'b1;
180          display_name  <= "EXEPC";
181          display_value <= EXE_pc;
182      end
183      6'd5 : //显示 MEM 模块的 PC
184      begin
185          display_valid <= 1'b1;
186          display_name  <= "MEMPC";
187          display_value <= MEM_pc;
188      end
189      6'd6 : //显示 WB 模块的 PC
190      begin
191          display_valid <= 1'b1;
192          display_name  <= "WB_PC";
193          display_value <= WB_pc;
194      end
195      6'd7 : //显示要观察的内存地址
196      begin
197          display_valid <= 1'b1;
198          display_name  <= "MADDR";
199          display_value <= mem_addr;
200      end
201      6'd8 : //显示该内存地址对应的数据
202      begin
203          display_valid <= 1'b1;
204          display_name  <= "MDATA";
205          display_value <= mem_data;
206      end
207      6'd9 : //显示 CPU 当前状态
208      begin
209          display_valid <= 1'b1;
210          display_name  <= "VALID";
211          display_value <= cpu_5_valid;
212      end
213      6'd11: //显示 HI 寄存器的值
214      begin
215          display_valid <= 1'b1;
216          display_name  <= "    HI";
217          display_value <= HI_data;
218      end
219      6'd12: //显示 LO 寄存器的值
220      begin
221          display_valid <= 1'b1;
```

```
222             display_name <= "    LO";
223             display_value <= LO_data;
224         end
225         default :
226         begin
227             display_valid <= 1'b0;
228             display_name <= 40'd0;
229             display_value <= 32'd0;
230         end
231     endcase
232 end
233 end
234 //-----{输出到触摸屏显示}end
235 //-----{调用触摸屏模块}end-----//
236 endmodule
```

pipeline_cpu.v 源代码:

```
1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: pipeline_cpu.v
4  //    > 描述   : 五级流水 CPU 模块, 共实现 XX 条指令
5  //    >         指令 rom 和数据 ram 均实例化 xilinx IP 得到, 为同步读写
6  //    > 作者   : LOONGSON
7  //    > 日期   : 2016-04-14
8  //*****
9  module pipeline_cpu( // 多周期 cpu
10     input clk,          // 时钟
11     input resetn,       // 复位信号, 低电平有效
12
13     //display data
14     input  [ 4:0] rf_addr,
15     input  [31:0] mem_addr,
16     output [31:0] rf_data,
17     output [31:0] mem_data,
18     output [31:0] IF_pc,
19     output [31:0] IF_inst,
20     output [31:0] ID_pc,
21     output [31:0] EXE_pc,
22     output [31:0] MEM_pc,
23     output [31:0] WB_pc,
24
25     //5 级流水新增
26     output [31:0] cpu_5_valid,
27     output [31:0] HI_data,
28     output [31:0] LO_data
29 );
30 //-----{5 级流水控制信号}begin-----//
31 //5 模块的 valid 信号
32 reg IF_valid;
33 reg ID_valid;
```

```
34     reg EXE_valid;
35     reg MEM_valid;
36     reg WB_valid;
37     //5 模块执行完成信号,来自各模块的输出
38     wire IF_over;
39     wire ID_over;
40     wire EXE_over;
41     wire MEM_over;
42     wire WB_over;
43     //5 模块允许下一级指令进入
44     wire IF_allow_in;
45     wire ID_allow_in;
46     wire EXE_allow_in;
47     wire MEM_allow_in;
48     wire WB_allow_in;
49
50     // syscall 和 eret 到达写回级时会发出 cancel 信号,
51     wire cancel;    // 取消已经取出的正在其他流水级执行的指令
52
53     //各级允许进入信号:本级无效,或本级执行完成且下级允许进入
54     assign IF_allow_in  = (IF_over & ID_allow_in) | cancel;
55     assign ID_allow_in  = ~ID_valid  | (ID_over  &
EXE_allow_in);
56     assign EXE_allow_in = ~EXE_valid | (EXE_over &
MEM_allow_in);
57     assign MEM_allow_in = ~MEM_valid | (MEM_over &
WB_allow_in );
58     assign WB_allow_in  = ~WB_valid  | WB_over;
59
60     //IF_valid, 在复位后,一直有效
61     always @(posedge clk)
62     begin
63         if (!resetn)
64             begin
65                 IF_valid <= 1'b0;
66             end
67         else
68             begin
69                 IF_valid <= 1'b1;
70             end
71     end
72
73     //ID_valid
74     always @(posedge clk)
75     begin
76         if (!resetn || cancel)
77             begin
78                 ID_valid <= 1'b0;
79             end
80         else if (ID_allow_in)
81             begin
82                 ID_valid <= IF_over;
```

```
83         end
84     end
85
86     //EXE_valid
87     always @(posedge clk)
88     begin
89         if (!resetsn || cancel)
90         begin
91             EXE_valid <= 1'b0;
92         end
93         else if (EXE_allow_in)
94         begin
95             EXE_valid <= ID_over;
96         end
97     end
98
99     //MEM_valid
100    always @(posedge clk)
101    begin
102        if (!resetsn || cancel)
103        begin
104            MEM_valid <= 1'b0;
105        end
106        else if (MEM_allow_in)
107        begin
108            MEM_valid <= EXE_over;
109        end
110    end
111
112    //WB_valid
113    always @(posedge clk)
114    begin
115        if (!resetsn || cancel)
116        begin
117            WB_valid <= 1'b0;
118        end
119        else if (WB_allow_in)
120        begin
121            WB_valid <= MEM_over;
122        end
123    end
124
125    //展示 5 级的 valid 信号
126    assign cpu_5_valid = {12'd0 , {4{IF_valid }}, {4{ID_valid}},
127    {4{EXE_valid}}, {4{MEM_valid}}, {4{WB_valid}}};
128    //-----{5 级流水控制信号}end-----//
129
130    //-----{5 级间的总线}begin-----
131    //
131    wire [ 63:0] IF_ID_bus;    // IF->ID 级总线
```

```
132    wire [166:0] ID_EXE_bus;    // ID->EXE 级总线
133    wire [153:0] EXE_MEM_bus;  // EXE->MEM 级总线
134    wire [117:0] MEM_WB_bus;   // MEM->WB 级总线
135
136    //锁存以上总线信号
137    reg [ 63:0] IF_ID_bus_r;
138    reg [166:0] ID_EXE_bus_r;
139    reg [153:0] EXE_MEM_bus_r;
140    reg [117:0] MEM_WB_bus_r;
141
142    //IF 到 ID 的锁存信号
143    always @(posedge clk)
144    begin
145        if(IF_over && ID_allow_in)
146        begin
147            IF_ID_bus_r <= IF_ID_bus;
148        end
149    end
150    //ID 到 EXE 的锁存信号
151    always @(posedge clk)
152    begin
153        if(ID_over && EXE_allow_in)
154        begin
155            ID_EXE_bus_r <= ID_EXE_bus;
156        end
157    end
158    //EXE 到 MEM 的锁存信号
159    always @(posedge clk)
160    begin
161        if(EXE_over && MEM_allow_in)
162        begin
163            EXE_MEM_bus_r <= EXE_MEM_bus;
164        end
165    end
166    //MEM 到 WB 的锁存信号
167    always @(posedge clk)
168    begin
169        if(MEM_over && WB_allow_in)
170        begin
171            MEM_WB_bus_r <= MEM_WB_bus;
172        end
173    end
174    //-----{5 级间的总线}end-----
175    //
176    //-----{其他交互信号}begin-----
177    //
178    //跳转总线
179    wire [ 32:0] jbr_bus;
180
181    //IF 与 inst_rom 交互
```

```
181     wire [31:0] inst_addr;
182     wire [31:0] inst;
183
184     //ID 与 EXE、MEM、WB 交互
185     wire [ 4:0] EXE_wdest;
186     wire [ 4:0] MEM_wdest;
187     wire [ 4:0] WB_wdest;
188
189     //MEM 与 data_ram 交互
190     wire [ 3:0] dm_wen;
191     wire [31:0] dm_addr;
192     wire [31:0] dm_wdata;
193     wire [31:0] dm_rdata;
194
195     //ID 与 regfile 交互
196     wire [ 4:0] rs;
197     wire [ 4:0] rt;
198     wire [31:0] rs_value;
199     wire [31:0] rt_value;
200
201     //WB 与 regfile 交互
202     wire        rf_wen;
203     wire [ 4:0] rf_wdest;
204     wire [31:0] rf_wdata;
205
206     //WB 与 IF 间的交互信号
207     wire [32:0] exc_bus;
208     //-----{其他交互信号}end-----
209     //
210     //-----{各模块实例化}begin-----
211     //
212     wire next_fetch; //即将运行取指模块，需要先锁存 PC 值
213     //IF 允许进入时，即锁存 PC 值，取下一条指令
214     assign next_fetch = IF_allow_in;
215     fetch IF_module( // 取指级
216         .clk      (clk      ), // I, 1
217         .resetn    (resetn   ), // I, 1
218         .IF_valid  (IF_valid ), // I, 1
219         .next_fetch(next_fetch), // I, 1
220         .inst      (inst     ), // I, 32
221         .jbr_bus   (jbr_bus  ), // I, 33
222         .inst_addr (inst_addr), // O, 32
223         .IF_over   (IF_over  ), // O, 1
224         .IF_ID_bus (IF_ID_bus), // O, 64
225
226         //5 级流水新增接口
227         .exc_bus   (exc_bus  ), // I, 32
228
229         //展示 PC 和取出的指令
230         .IF_pc     (IF_pc    ), // O, 32
```

```

230         .IF_inst      (IF_inst      )    // O, 32
231     );
232
233     decode ID_module(                      // 译码级
234         .ID_valid      (ID_valid      ),    // I, 1
235         .IF_ID_bus_r   (IF_ID_bus_r),    // I, 64
236         .rs_value      (rs_value      ),    // I, 32
237         .rt_value      (rt_value      ),    // I, 32
238         .rs             (rs            ),    // O, 5
239         .rt             (rt            ),    // O, 5
240         .jbr_bus       (jbr_bus       ),    // O, 32
241     //     .inst_jbr     (inst_jbr     ),    // O, 1
242         .ID_over       (ID_over       ),    // O, 1
243         .ID_EXE_bus    (ID_EXE_bus    ),    // O, 167
244
245         //5 级流水新增
246         .IF_over       (IF_over       ),    // I, 1
247         .EXE_wdest     (EXE_wdest     ),    // I, 5
248         .MEM_wdest     (MEM_wdest     ),    // I, 5
249         .WB_wdest      (WB_wdest      ),    // I, 5
250
251         //展示 PC
252         .ID_pc         (ID_pc         )    // O, 32
253     );
254
255     exe EXE_module(                        // 执行级
256         .EXE_valid      (EXE_valid      ),    // I, 1
257         .ID_EXE_bus_r   (ID_EXE_bus_r),    // I, 167
258         .EXE_over       (EXE_over       ),    // O, 1
259         .EXE_MEM_bus    (EXE_MEM_bus    ),    // O, 154
260
261         //5 级流水新增
262         .clk            (clk            ),    // I, 1
263         .EXE_wdest      (EXE_wdest      ),    // O, 5
264
265         //展示 PC
266         .EXE_pc         (EXE_pc         )    // O, 32
267     );
268
269     mem MEM_module(                      // 访存级
270         .clk            (clk            ),    // I, 1
271         .MEM_valid      (MEM_valid      ),    // I, 1
272         .EXE_MEM_bus_r   (EXE_MEM_bus_r),    // I, 154
273         .dm_rdata       (dm_rdata       ),    // I, 32
274         .dm_addr        (dm_addr        ),    // O, 32
275         .dm_wen         (dm_wen         ),    // O, 4
276         .dm_wdata       (dm_wdata       ),    // O, 32
277         .MEM_over       (MEM_over       ),    // O, 1
278         .MEM_WB_bus     (MEM_WB_bus     ),    // O, 118
279
280         //5 级流水新增接口

```

```

281         .MEM_allow_in (MEM_allow_in ), // I, 1
282         .MEM_wdest     (MEM_wdest     ), // O, 5
283
284         //展示 PC
285         .MEM_pc         (MEM_pc         ) // O, 32
286     );
287
288     wb WB_module( // 写回级
289         .WB_valid      (WB_valid      ), // I, 1
290         .MEM_WB_bus_r(MEM_WB_bus_r), // I, 118
291         .rf_wen        (rf_wen        ), // O, 1
292         .rf_wdest      (rf_wdest      ), // O, 5
293         .rf_wdata      (rf_wdata      ), // O, 32
294         .WB_over       (WB_over       ), // O, 1
295
296         //5 级流水新增接口
297         .clk           (clk           ), // I, 1
298         .resetn        (resetn        ), // I, 1
299         .exc_bus       (exc_bus       ), // O, 32
300         .WB_wdest      (WB_wdest      ), // O, 5
301         .cancel        (cancel        ), // O, 1
302
303         //展示 PC 和 HI/LO 值
304         .WB_pc         (WB_pc         ), // O, 32
305         .HI_data       (HI_data       ), // O, 32
306         .LO_data       (LO_data       ) // O, 32
307     );
308
309     inst_rom inst_rom_module( // 指令存储器
310         .clka          (clk          ), // I, 1 ,时钟
311         .addra         (inst_addr[9:2]), // I, 8 ,指令地址
312         .douta         (inst         ) // O, 32,指令
313     );
314
315     regfile rf_module( // 寄存器堆模块
316         .clk           (clk           ), // I, 1
317         .wen           (rf_wen        ), // I, 1
318         .raddr1        (rs            ), // I, 5
319         .raddr2        (rt            ), // I, 5
320         .waddr         (rf_wdest      ), // I, 5
321         .wdata         (rf_wdata      ), // I, 32
322         .rdata1        (rs_value      ), // O, 32
323         .rdata2        (rt_value      ), // O, 32
324
325         //display rf
326         .test_addr(rf_addr), // I, 5
327         .test_data(rf_data) // O, 32
328     );
329
330     data_ram data_ram_module( // 数据存储模块
331         .clka          (clk          ), // I, 1, 时钟

```

```

332         .wea      (dm_wen      ), // I, 1,  写使能
333         .addra    (dm_addr[9:2]), // I, 8,  读地址
334         .dina      (dm_wdata    ), // I, 32, 写数据
335         .douta     (dm_rdata     ), // O, 32, 读数据
336
337         //display mem
338         .clkb      (clk          ), // I, 1,  时钟
339         .web        (4'd0         ), // 不使用端口 2 的写功能
340         .addrb      (mem_addr[9:2]), // I, 8,  读地址
341         .doutb      (mem_data     ), // I, 32, 写数据
342         .dinb       (32'd0        ) // 不使用端口 2 的写功能
343     );
344     //-----{各模块实例化}end-----
345     endmodule

```

fetch.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: fetch.v
4  //  > 描述   :五级流水 CPU 的取指模块
5  //  > 作者   : LOONGSON
6  //  > 日期   : 2016-04-14
7  //*****
8  `define STARTADDR 32'H00000034 // 程序起始地址为 34H
9  module fetch( // 取指级
10     input          clk, // 时钟
11     input          resetn, // 复位信号, 低电平有效
12     input          IF_valid, // 取指级有效信号
13     input          next_fetch, // 取下一条指令, 用来锁存 PC 值
14     input          [31:0] inst, // inst_rom 取出的指令
15     input          [32:0] jbr_bus, // 跳转总线
16     output         [31:0] inst_addr, // 发往 inst_rom 的取指地址
17     output reg      IF_over, // IF 模块执行完成
18     output         [63:0] IF_ID_bus, // IF->ID 总线
19
20     //5 级流水新增接口
21     input          [32:0] exc_bus, // Exception pc 总线
22
23     //展示 PC 和取出的指令
24     output         [31:0] IF_pc,
25     output         [31:0] IF_inst
26 );
27
28 //-----{程序计数器 PC}begin
29     wire [31:0] next_pc;
30     wire [31:0] seq_pc;
31     reg  [31:0] pc;
32
33     //跳转 pc

```

```
34     wire          jbr_taken;
35     wire [31:0] jbr_target;
36     assign {jbr_taken, jbr_target} = jbr_bus; // 跳转总线传是否跳转和目标地址
37
38     //Exception PC
39     wire          exc_valid;
40     wire [31:0] exc_pc;
41     assign {exc_valid,exc_pc} = exc_bus;
42
43     //pc+4
44     assign seq_pc[31:2]      = pc[31:2] + 1'b1; // 下一指令地址: PC=PC+4
45     assign seq_pc[1:0]      = pc[1:0];
46
47     // 新指令: 若有 Exception,则 PC 为 Exceptio 入口地址
48     //           若指令跳转, 则 PC 为跳转地址; 否则为 pc+4
49     assign next_pc = exc_valid ? exc_pc :
50                   jbr_taken ? jbr_target : seq_pc;
51     always @(posedge clk) // PC 程序计数器
52     begin
53         if (!resetsn)
54             begin
55                 pc <= `STARTADDR; // 复位, 取程序起始地址
56             end
57         else if (next_fetch)
58             begin
59                 pc <= next_pc; // 不复位, 取新指令
60             end
61     end
62 //-----{程序计数器 PC}end
63
64 //-----{发往 inst_rom 的取指地址}begin
65     assign inst_addr = pc;
66 //-----{发往 inst_rom 的取指地址}end
67
68 //-----{IF 执行完成}begin
69     //由于指令 rom 为同步读写的,
70     //取数据时, 有一拍延时
71     //即发地址的下一拍时钟才能得到对应的指令
72     //故取指模块需要两拍时间
73     //故每次 PC 刷新, IF_over 都要置 0
74     //然后将 IF_valid 锁存一拍即是 IF_over 信号
75     always @(posedge clk)
76     begin
77         if (!resetsn || next_fetch)
78             begin
79                 IF_over <= 1'b0;
80             end
81         else
82             begin
83                 IF_over <= IF_valid;
84             end
85     end
```

```

85     end
86     //如果指令 rom 为异步读的, 则 IF_valid 即是 IF_over 信号,
87     //即取指一拍完成
88 //-----{IF 执行完成}end
89
90 //-----{IF->ID 总线}begin
91     assign IF_ID_bus = {pc, inst}; // 取指级有效时, 锁存 PC 和指令
92 //-----{IF->ID 总线}end
93
94 //-----{展示 IF 模块的 PC 值和指令}begin
95     assign IF_pc     = pc;
96     assign IF_inst   = inst;
97 //-----{展示 IF 模块的 PC 值和指令}end
98 endmodule

```

decode.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //  > 文件名: decode.v
4  //  > 描述   : 五级流水 CPU 的译码模块
5  //  > 作者   : LOONGSON
6  //  > 日期   : 2016-04-14
7  //*****
8  module decode(                                // 译码级
9      input                                ID_valid, // 译码级有效信号
10     input      [ 63:0] IF_ID_bus_r, // IF->ID 总线
11     input      [ 31:0] rs_value, // 第一源操作数值
12     input      [ 31:0] rt_value, // 第二源操作数值
13     output     [  4:0] rs, // 第一源操作数地址
14     output     [  4:0] rt, // 第二源操作数地址
15     output     [ 32:0] jbr_bus, // 跳转总线
16 // 要 output     inst_jbr, // 指令为跳转分支指令, 五级流水不需
17     output     ID_over, // ID 模块执行完成
18     output     [166:0] ID_EXE_bus, // ID->EXE 总线
19
20     //5 级流水新增
21     input     IF_over, //对于分支指令, 需要该信号
22     input     [  4:0] EXE_wdest, // EXE 级要写回寄存器堆的目标地址
23     input     [  4:0] MEM_wdest, // MEM 级要写回寄存器堆的目标地址
24     input     [  4:0] WB_wdest, // WB 级要写回寄存器堆的目标地址号
25
26     //展示 PC
27     output     [ 31:0] ID_pc
28 );
29 //-----{IF->ID 总线}begin
30     wire [31:0] pc;
31     wire [31:0] inst;

```

```

32     assign {pc, inst} = IF_ID_bus_r; // IF->ID 总线传 PC 和指令
33 //-----{IF->ID 总线}end
34
35 //-----{指令译码}begin
36     wire [5:0] op;
37     wire [4:0] rd;
38     wire [4:0] sa;
39     wire [5:0] funct;
40     wire [15:0] imm;
41     wire [15:0] offset;
42     wire [25:0] target;
43     wire [2:0] cp0r_sel;
44
45     assign op      = inst[31:26]; // 操作码
46     assign rs      = inst[25:21]; // 源操作数 1
47     assign rt      = inst[20:16]; // 源操作数 2
48     assign rd      = inst[15:11]; // 目标操作数
49     assign sa      = inst[10:6];  // 特殊域, 可能存放偏移量
50     assign funct   = inst[5:0];   // 功能码
51     assign imm     = inst[15:0];  // 立即数
52     assign offset  = inst[15:0];  // 地址偏移量
53     assign target  = inst[25:0];  // 目标地址
54     assign cp0r_sel= inst[2:0];   // cp0 寄存器的 select 域
55
56     // 实现指令列表
57     wire inst_ADDU, inst_SUBU , inst_SLT , inst_AND;
58     wire inst_NOR , inst_OR   , inst_XOR , inst_SLL;
59     wire inst_SRL , inst_ADDIU, inst_BEQ , inst_BNE;
60     wire inst_LW  , inst_SW   , inst_LUI , inst_J;
61     wire inst_SLTU, inst_JALR , inst_JR  , inst_SLLV;
62     wire inst_SRA , inst_SRAV , inst_SRLV, inst_SLTIU;
63     wire inst_SLTI, inst_BGEZ , inst_BGTZ, inst_BLEZ;
64     wire inst_BLTZ, inst_LB   , inst_LBU , inst_SB;
65     wire inst_ANDI, inst_ORI  , inst_XORI, inst_JAL;
66     wire inst_MULT, inst_MFLO , inst_MFHI, inst_MTLO;
67     wire inst_MTHI, inst_MFC0 , inst_MTC0;
68     wire inst_ERET, inst_SYSCALL;
69     wire op_zero; // 操作码全 0
70     wire sa_zero; // sa 域全 0
71     assign op_zero = ~(|op);
72     assign sa_zero = ~(|sa);
73     assign inst_ADDU = op_zero & sa_zero & (funct == 6'b100001); // 无符号加法
74     assign inst_SUBU = op_zero & sa_zero & (funct == 6'b100011); // 无符号减法
75     assign inst_SLT  = op_zero & sa_zero & (funct == 6'b101010); // 小于则置位
76     assign inst_SLTU = op_zero & sa_zero & (funct == 6'b101011); // 无符号小则置
77     assign inst_JALR = op_zero & (rt==5'd0) & (rd==5'd31)
78                     & sa_zero & (funct == 6'b001001); // 跳转寄存器并链接
79     assign inst_JR   = op_zero & (rt==5'd0) & (rd==5'd0 )
80                     & sa_zero & (funct == 6'b001000); // 跳转寄存器
81     assign inst_AND  = op_zero & sa_zero & (funct == 6'b100100); // 与运算
82     assign inst_NOR  = op_zero & sa_zero & (funct == 6'b100111); // 或非运算

```

```

83    assign inst_OR      = op_zero & sa_zero    & (funct == 6'b100101); //或运算
84    assign inst_XOR     = op_zero & sa_zero    & (funct == 6'b100110); //异或运算
85    assign inst_SLL     = op_zero & (rs==5'd0) & (funct == 6'b000000); //逻辑左移
86    assign inst_SLLV    = op_zero & sa_zero    & (funct == 6'b000100); //变量逻辑左移
87    assign inst_SRA     = op_zero & (rs==5'd0) & (funct == 6'b000011); //算术右移
88    assign inst_SRAV    = op_zero & sa_zero    & (funct == 6'b000111); //变量算术右移
89    assign inst_SRL     = op_zero & (rs==5'd0) & (funct == 6'b000010); //逻辑右移
90    assign inst_SRLV    = op_zero & sa_zero    & (funct == 6'b000110); //变量逻辑右移
91    assign inst_MULT    = op_zero & (rd==5'd0)
92                                & sa_zero & (funct == 6'b011000);          //乘法
93    assign inst_MFLO    = op_zero & (rs==5'd0) & (rt==5'd0)
94                                & sa_zero & (funct == 6'b010010);          //从 LO 读取
95    assign inst_MFHI    = op_zero & (rs==5'd0) & (rt==5'd0)
96                                & sa_zero & (funct == 6'b010000);          //从 HI 读取
97    assign inst_MTLO    = op_zero & (rt==5'd0) & (rd==5'd0)
98                                & sa_zero & (funct == 6'b010011);          //向 LO 写数据
99    assign inst_MTHI    = op_zero & (rt==5'd0) & (rd==5'd0)
100                               & sa_zero & (funct == 6'b010001);          //向 HI 写数据
101    assign inst_ADDIU   = (op == 6'b001001);          //立即数无符号加法
102    assign inst_SLTI    = (op == 6'b001010);          //小于立即数则置位
103    assign inst_SLTIU   = (op == 6'b001011);          //无符号小于立即数则置位
104    assign inst_BEQ     = (op == 6'b000100);          //判断相等跳转
105    assign inst_BGEZ    = (op == 6'b000001) & (rt==5'd1); //大于等于 0 跳转
106    assign inst_BGTZ    = (op == 6'b000111) & (rt==5'd0); //大于 0 跳转
107    assign inst_BLEZ    = (op == 6'b000110) & (rt==5'd0); //小于等于 0 跳转
108    assign inst_BLTZ    = (op == 6'b000001) & (rt==5'd0); //小于 0 跳转
109    assign inst_BNE     = (op == 6'b000101);          //判断不等跳转
110    assign inst_LW      = (op == 6'b100011);          //从内存装载字
111    assign inst_SW      = (op == 6'b101011);          //向内存存储字
112    assign inst_LB      = (op == 6'b100000);          //load 字节 (符号扩展)
113    assign inst_LBU     = (op == 6'b100100);          //load 字节 (无符号扩展)
114    assign inst_SB      = (op == 6'b101000);          //向内存存储字节
115    assign inst_ANDI    = (op == 6'b001100);          //立即数与
116    assign inst_LUI     = (op == 6'b001111) & (rs==5'd0); //立即数装载高半字节
117    assign inst_ORI     = (op == 6'b001101);          //立即数或
118    assign inst_XORI    = (op == 6'b001110);          //立即数异或
119    assign inst_J       = (op == 6'b000010);          //跳转
120    assign inst_JAL     = (op == 6'b000011);          //跳转和链接
121    assign inst_MFC0    = (op == 6'b010000) & (rs==5'd0)
122                                & sa_zero & (funct[5:3] == 3'b000); // 从 cp0 寄存器装载
123    assign inst_MTC0    = (op == 6'b010000) & (rs==5'd4)
124                                & sa_zero & (funct[5:3] == 3'b000); // 向 cp0 寄存器存储
125    assign inst_SYSCALL = (op == 6'b000000) & (funct == 6'b001100); // 系统调用
126    assign inst_ERET    = (op == 6'b010000) & (rs==5'd16) & (rt==5'd0)
127                                & (rd==5'd0) & sa_zero & (funct == 6'b011000); //异常返回
128
129    //跳转分支指令
130    wire inst_jr;        //寄存器跳转指令
131    wire inst_j_link;    //链接跳转指令
132    wire inst_jbr;       //所有分支跳转指令
133    assign inst_jr       = inst_JALR | inst_JR;

```

```

134     assign inst_j_link = inst_JAL | inst_JALR;
135     assign inst_jbr = inst_J      | inst_JAL  | inst_jr
136                   | inst_BEQ  | inst_BNE  | inst_BGEZ
137                   | inst_BGTZ | inst_BLEZ | inst_BLTZ;
138
139     //load store
140     wire inst_load;
141     wire inst_store;
142     assign inst_load  = inst_LW | inst_LB | inst_LBU;  // load 指
143     assign inst_store = inst_SW | inst_SB;             // store 指
144
145     //alu 操作分类
146     wire inst_add, inst_sub, inst_slt,inst_sltu;
147     wire inst_and, inst_nor, inst_or, inst_xor;
148     wire inst_sll, inst_srl, inst_sra,inst_lui;
149     assign inst_add = inst_ADDU | inst_ADDIU | inst_load
150                   | inst_store | inst_j_link;          // 做加法
151     assign inst_sub = inst_SUBU;                       // 减
152     assign inst_slt = inst_SLT | inst_SLTI;            // 有符号小于置
153     assign inst_sltu= inst_SLTIU | inst_SLTU;          // 无符号小于置
154     assign inst_and = inst_AND | inst_ANDI;           // 逻辑与
155     assign inst_nor = inst_NOR;                       // 逻辑或非
156     assign inst_or  = inst_OR  | inst_ORI;           // 逻辑或
157     assign inst_xor = inst_XOR | inst_XORI;          // 逻辑异或
158     assign inst_sll = inst_SLL | inst_SLLV;          // 逻辑左移
159     assign inst_srl = inst_SRL | inst_SRLV;          // 逻辑右移
160     assign inst_sra = inst_SRA | inst_SRAV;          // 算术右移
161     assign inst_lui = inst_LUI;                      // 立即数装载高
162
163     //使用 sa 域作为偏移量的移位指令
164     wire inst_shf_sa;
165     assign inst_shf_sa =  inst_SLL | inst_SRL | inst_SRA;
166
167     //依据立即数扩展方式分类
168     wire inst_imm_zero; //立即数 0 扩展
169     wire inst_imm_sign; //立即数符号扩展
170     assign inst_imm_zero = inst_ANDI | inst_LUI | inst_ORI |
inst_XORI;
171     assign inst_imm_sign = inst_ADDIU | inst_SLTI | inst_SLTIU
172                   | inst_load | inst_store;
173
174     //依据目的寄存器号分类
175     wire inst_wdest_rt; // 寄存器堆写入地址为 rt 的指令
176     wire inst_wdest_31; // 寄存器堆写入地址为 31 的指令
177     wire inst_wdest_rd; // 寄存器堆写入地址为 rd 的指令

```

```

178     assign inst_wdest_rt = inst_imm_zero | inst_ADDIU | inst_SLTI
179                               | inst_SLTIU | inst_load | inst_MFC0;
180     assign inst_wdest_31 = inst_JAL;
181     assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SLT |
inst_SLTU
182                               | inst_JALR | inst_AND | inst_NOR |
inst_OR
183                               | inst_XOR | inst_SLL | inst_SLLV |
inst_SRA
184                               | inst_SRAV | inst_SRL | inst_SRLV
185                               | inst_MFHI | inst_MFLO;
186
187     //依据源寄存器号分类
188     wire inst_no_rs; //指令 rs 域非 0, 且不是从寄存器堆读 rs 的数据
189     wire inst_no_rt; //指令 rt 域非 0, 且不是从寄存器堆读 rt 的数据
190     assign inst_no_rs = inst_MTC0 | inst_SYSCALL | inst_ERET;
191     assign inst_no_rt = inst_ADDIU | inst_SLTI | inst_SLTIU
192                               | inst_BGEZ | inst_load | inst_imm_zero
193                               | inst_J | inst_JAL | inst_MFC0
194                               | inst_SYSCALL;
195     //-----{指令译码}end
196
197     //-----{分支指令执行}begin
198     //bd_pc, 分支跳转指令参与计算的为延迟槽指令的 PC 值, 即当前分支指令的 PC+4
199     wire [31:0] bd_pc; //延迟槽指令 PC 值
200     assign bd_pc = pc + 3'b100;
201
202     //无条件跳转
203     wire j_taken;
204     wire [31:0] j_target;
205     assign j_taken = inst_J | inst_JAL | inst_jr;
206     //寄存器跳转地址为 rs_value, 其他跳转为{bd_pc[31:28], target, 2'b00}
207     assign j_target = inst_jr ? rs_value :
{bd_pc[31:28], target, 2'b00};
208
209     //branch 指令
210     wire rs_equql_rt;
211     wire rs_ez;
212     wire rs_ltz;
213     assign rs_equql_rt = (rs_value == rt_value); //
GPR[rs]==GPR[rt]
214     assign rs_ez = ~(|rs_value); // rs 寄存器值为
0
215     assign rs_ltz = rs_value[31]; // rs 寄存器值小
于 0
216     wire br_taken;
217     wire [31:0] br_target;
218     assign br_taken = inst_BEQ & rs_equql_rt // 相等跳转
219                               | inst_BNE & ~rs_equql_rt // 不等跳转
220                               | inst_BGEZ & ~rs_ltz // 大于等于 0
跳转
221                               | inst_BGTZ & ~rs_ltz & ~rs_ez // 大于 0 跳转

```

```

222                                     | inst_BLEZ & (rs_ltz | rs_ez) // 小于等于 0
    跳转
223                                     | inst_BLTZ & rs_ltz; // 小于 0 跳转
224 // 分支跳转目标地址: PC=PC+offset<<2
225     assign br_target[31:2] = bd_pc[31:2] + {{14{offset[15]}}},
    offset};
226     assign br_target[1:0] = bd_pc[1:0];
227
228 //jump and branch 指令
229     wire jbr_taken;
230     wire [31:0] jbr_target;
231     assign jbr_taken = (j_taken | br_taken) & ID_over;
232     assign jbr_target = j_taken ? j_target : br_target;
233
234 //ID 到 IF 的跳转总线
235     assign jbr_bus = {jbr_taken, jbr_target};
236 //-----{分支指令执行}end
237
238 //-----{ID 执行完成}begin
239 //由于是流水的, 存在数据相关
240     wire rs_wait;
241     wire rt_wait;
242     assign rs_wait = ~inst_no_rs & (rs!=5'd0)
243                     & ( (rs==EXE_wdest) | (rs==MEM_wdest) |
    (rs==WB_wdest) );
244     assign rt_wait = ~inst_no_rt & (rt!=5'd0)
245                     & ( (rt==EXE_wdest) | (rt==MEM_wdest) |
    (rt==WB_wdest) );
246
247 //对于分支跳转指令, 只有在 IF 执行完成后, 才可以算 ID 完成;
248 //否则, ID 级先完成了, 而 IF 还在取指令, 则 next_pc 不能锁存到 PC 里去,
249 //那么等 IF 完成, next_pc 能锁存到 PC 里去时, jbr_bus 上的数据已变成无
    效,
250 //导致分支跳转失败
251 //(~inst_jbr | IF_over)即是(~inst_jbr | (inst_jbr & IF_over))
252     assign ID_over = ID_valid & ~rs_wait & ~rt_wait & (~inst_jbr
    | IF_over);
253 //-----{ID 执行完成}end
254
255 //-----{ID->EXE 总线}begin
256 //EXE 需要用到的信息
257     wire multiply; //乘法 MULT
258     wire mthi; //MTHI
259     wire mtlo; //MTLO
260     assign multiply = inst_MULT;
261     assign mthi = inst_MTHI;
262     assign mtlo = inst_MTLO;
263 //ALU 两个源操作数和控制信号
264     wire [11:0] alu_control;
265     wire [31:0] alu_operand1;
266     wire [31:0] alu_operand2;
267

```

```

268 //所谓链接跳转是将跳转返回的 PC 值存放到 31 号寄存器里
269 //在流水 CPU 里，考虑延迟槽，故链接跳转需要计算 PC+8，存放到 31 号寄存器
    里
270 assign alu_operand1 = inst_j_link ? pc :
271                     inst_shf_sa ? {27'd0,sa} : rs_value;
272 assign alu_operand2 = inst_j_link ? 32'd8 :
273                     inst_imm_zero ? {16'd0, imm} :
274                     inst_imm_sign ? {{16{imm[15]}}}, imm} :
    rt_value;
275 assign alu_control = {inst_add,          // ALU 操作码，独热编码
276                     inst_sub,
277                     inst_slt,
278                     inst_sltu,
279                     inst_and,
280                     inst_nor,
281                     inst_or,
282                     inst_xor,
283                     inst_sll,
284                     inst_srl,
285                     inst_sra,
286                     inst_lui};
287 //访存需要用到的 load/store 信息
288 wire lb_sign; //load 一字节为有符号 load
289 wire ls_word; //load/store 为字节还是字,0:byte;1:word
290 wire [3:0] mem_control; //MEM 需要使用的控制信号
291 wire [31:0] store_data; //store 操作的存的数据
292 assign lb_sign = inst_LB;
293 assign ls_word = inst_LW | inst_SW;
294 assign mem_control = {inst_load,
295                     inst_store,
296                     ls_word,
297                     lb_sign };
298
299 //写回需要用到的信息
300 wire mfhi;
301 wire mflo;
302 wire mtc0;
303 wire mfc0;
304 wire [7 :0] cp0r_addr;
305 wire syscall; //syscall 和 eret 在写回级有特殊的操作
306 wire eret;
307 wire rf_wen; //写回的寄存器写使能
308 wire [4:0] rf_wdest; //写回的目的寄存器
309 assign syscall = inst_SYSCALL;
310 assign eret = inst_ERET;
311 assign mfhi = inst_MFHI;
312 assign mflo = inst_MFLO;
313 assign mtc0 = inst_MTC0;
314 assign mfc0 = inst_MFC0;
315 assign cp0r_addr= {rd,cp0r_sel};
316 assign rf_wen = inst_wdest_rt | inst_wdest_31 |

```

```

    inst_wdest_rd;
317     assign rf_wdest = inst_wdest_rt ? rt :      //在不写寄存器堆时设
        置为 0
318                                     inst_wdest_31 ? 5'd31 :  //以便能准确判断数据
        相关
319                                     inst_wdest_rd ? rd : 5'd0;
320     assign store_data = rt_value;
321     assign ID_EXE_bus = {multiply,mthi,mtlo,
        //EXE 需用信息,新增
322     alu_control,alu_operand1,alu_operand2,//EXE 需用的信息
        mem_control,store_data,
323     //MEM 需用的信号
        mfhi,mflo,
324     //WB 需用的信号,新增
        mtc0,mfc0,cp0r_addr,syscall,eret,
325     //WB 需用的信号,新增
        rf_wen, rf_wdest,
326     //WB 需用的信号
        pc};
327     //PC 值
328     //-----{ID->EXE 总线}end
329
330     //-----{展示 ID 模块的 PC 值}begin
331     assign ID_pc = pc;
332     //-----{展示 ID 模块的 PC 值}end
333 endmodule

```

exe.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: exe.v
4  //    > 描述   :五级流水 CPU 的执行模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  module exe(                                // 执行级
9      input                                EXE_valid,    // 执行级有效信号
10     input      [166:0] ID_EXE_bus_r, // ID->EXE 总线
11     output      EXE_over,    // EXE 模块执行完成
12     output      [153:0] EXE_MEM_bus, // EXE->MEM 总线
13
14     //5 级流水新增
15     input      clk,          // 时钟
16     output      [ 4:0] EXE_wdest, // EXE 级要写回寄存器堆的目标地址号
17
18     //展示 PC
19     output      [ 31:0] EXE_pc
20 );
21 //-----{ID->EXE 总线}begin
22     //EXE 需要用到的信息

```

```

23    wire multiply;                //乘法
24    wire mthi;                   //MTHI
25    wire mtlo;                   //MTLO
26    wire [11:0] alu_control;
27    wire [31:0] alu_operand1;
28    wire [31:0] alu_operand2;
29
30    //访存需要用到的 load/store 信息
31    wire [3:0] mem_control; //MEM 需要使用的控制信号
32    wire [31:0] store_data; //store 操作的存的数据
33
34    //写回需要用到的信息
35    wire mfhi;
36    wire mflo;
37    wire mtc0;
38    wire mfc0;
39    wire [7 :0] cp0r_addr;
40    wire        syscall; //syscall 和 eret 在写回级有特殊的操作
41    wire        eret;
42    wire        rf_wen; //写回的寄存器写使能
43    wire [4:0] rf_wdest; //写回的目的寄存器
44
45    //pc
46    wire [31:0] pc;
47    assign {multiply,
48            mthi,
49            mtlo,
50            alu_control,
51            alu_operand1,
52            alu_operand2,
53            mem_control,
54            store_data,
55            mfhi,
56            mflo,
57            mtc0,
58            mfc0,
59            cp0r_addr,
60            syscall,
61            eret,
62            rf_wen,
63            rf_wdest,
64            pc          } = ID_EXE_bus_r;
65    //-----{ID->EXE 总线}end
66
67    //-----{ALU}begin
68    wire [31:0] alu_result;
69
70    alu alu_module(
71        .alu_control  (alu_control ), // I, 12, ALU 控制信号
72        .alu_src1      (alu_operand1), // I, 32, ALU 操作数 1
73        .alu_src2      (alu_operand2), // I, 32, ALU 操作数 2

```

```

74         .alu_result    (alu_result )    // 0, 32, ALU 结果
75     );
76 //-----{ALU}end
77
78 //-----{乘法器}begin
79     wire                mult_begin;
80     wire [63:0] product;
81     wire                mult_end;
82
83     assign mult_begin = multiply & EXE_valid;
84     multiply multiply_module (
85         .clk            (clk            ),
86         .mult_begin(mult_begin ),
87         .mult_op1    (alu_operand1),
88         .mult_op2    (alu_operand2),
89         .product     (product ),
90         .mult_end    (mult_end )
91     );
92 //-----{乘法器}end
93
94 //-----{EXE 执行完成}begin
95     //对于 ALU 操作，都是 1 拍可完成，
96     //但对于乘法操作，需要多拍完成
97     assign EXE_over = EXE_valid & (~multiply | mult_end);
98 //-----{EXE 执行完成}end
99
100 //-----{EXE 模块的 dest 值}begin
101     //只有在 EXE 模块有效时，其写回目的寄存器号才有意义
102     assign EXE_wdest = rf_wdest & {5{EXE_valid}};
103 //-----{EXE 模块的 dest 值}end
104
105 //-----{EXE->MEM 总线}begin
106     wire [31:0] exe_result;    //在 exe 级能确定的最终写回结果
107     wire [31:0] lo_result;
108     wire        hi_write;
109     wire        lo_write;
110     //要写入 HI 的值放在 exe_result 里，包括 MULT 和 MTHI 指令，
111     //要写入 LO 的值放在 lo_result 里，包括 MULT 和 MTLO 指令，
112     assign exe_result = mthi      ? alu_operand1 :
113                        mtc0      ? alu_operand2 :
114                        multiply ? product[63:32] : alu_result;
115     assign lo_result  = mtlo ? alu_operand1 : product[31:0];
116     assign hi_write   = multiply | mthi;
117     assign lo_write   = multiply | mtlo;
118
119     assign EXE_MEM_bus = {mem_control,store_data, //load/store 信息和 store 数
据
120                        exe_result,                //exe 运算结果
121                        lo_result,                 //乘法低 32 位结果，新增
122                        hi_write,lo_write,         //HI/LO 写使能，新增
123                        mfhi,mflo,                  //WB 需用的信号，新增

```

```

124          mtc0,mfc0,cp0r_addr,syscall,eret, //WB 需用的信号,新
增
125          rf_wen,rf_wdest,          //WB 需用的信号
126          pc};                      //PC
127 //-----{EXE->MEM 总线}end
128
129 //-----{展示 EXE 模块的 PC 值}begin
130     assign EXE_pc = pc;
131 //-----{展示 EXE 模块的 PC 值}end
132 endmodule

```

mem.v 源代码:

```

1  `timescale 1ns / 1ps
2  /**
3   * > 文件名: mem.v
4   * > 描述 : 五级流水 CPU 的访存模块
5   * > 作者 : LOONGSON
6   * > 日期 : 2016-04-14
7   */
8  module mem(                                // 访存级
9      input          clk,                    // 时钟
10     input          MEM_valid,              // 访存级有效信号
11     input          [153:0] EXE_MEM_bus_r, // EXE->MEM 总线
12     input          [ 31:0] dm_rdata,       // 访存读数据
13     output         [ 31:0] dm_addr,        // 访存读写地址
14     output reg     [ 3:0] dm_wen,          // 访存写使能
15     output reg     [31:0] dm_wdata,       // 访存写数据
16     output         MEM_over,              // MEM 模块执行完成
17     output         [117:0] MEM_WB_bus,    // MEM->WB 总线
18
19     //5 级流水新增接口
20     input          MEM_allow_in, // MEM 级允许下级进入
21     output         [ 4:0] MEM_wdest, // MEM 级要写回寄存器堆的目标地址
22
23     //展示 PC
24     output         [31:0] MEM_pc
25 );
26 //-----{EXE->MEM 总线}begin
27 //访存需要用到的 load/store 信息
28 wire [3:0] mem_control; //MEM 需要使用的控制信号
29 wire [31:0] store_data; //store 操作的存的数据
30
31 //EXE 结果和 HI/LO 数据
32 wire [31:0] exe_result;
33 wire [31:0] lo_result;
34 wire        hi_write;
35 wire        lo_write;
36
37 //写回需要用到的信息

```

```
38     wire mfhi;
39     wire mflo;
40     wire mtc0;
41     wire mfc0;
42     wire [7 :0] cp0r_addr;
43     wire      syscall;    //syscall 和 eret 在写回级有特殊的操作
44     wire      eret;
45     wire      rf_wen;     //写回的寄存器写使能
46     wire [4:0] rf_wdest;  //写回的目的寄存器
47
48     //pc
49     wire [31:0] pc;
50     assign {mem_control,
51            store_data,
52            exe_result,
53            lo_result,
54            hi_write,
55            lo_write,
56            mfhi,
57            mflo,
58            mtc0,
59            mfc0,
60            cp0r_addr,
61            syscall,
62            eret,
63            rf_wen,
64            rf_wdest,
65            pc          } = EXE_MEM_bus_r;
66 //-----{EXE->MEM 总线}end
67
68 //-----{load/store 访存}begin
69     wire inst_load;  //load 操作
70     wire inst_store; //store 操作
71     wire ls_word;    //load/store 为字节还是字,0:byte;1:word
72     wire lb_sign;    //load 一字节为有符号 load
73     assign {inst_load,inst_store,ls_word,lb_sign} = mem_control;
74
75     //访存读写地址
76     assign dm_addr = exe_result;
77
78     //store 操作的写使能
79     always @ (*)    // 内存写使能信号
80     begin
81         if (MEM_valid && inst_store) // 访存级有效时,且为 store 操作
82         begin
83             if (ls_word)
84             begin
85                 dm_wen <= 4'b1111; // 存储字指令,写使能全 1
86             end
87             else
88             begin // SB 指令,需要依据地址底两位,确定对应的写使能
```

```

89             case (dm_addr[1:0])
90                 2'b00 : dm_wen <= 4'b0001;
91                 2'b01 : dm_wen <= 4'b0010;
92                 2'b10 : dm_wen <= 4'b0100;
93                 2'b11 : dm_wen <= 4'b1000;
94                 default : dm_wen <= 4'b0000;
95             endcase
96         end
97     end
98     else
99     begin
100         dm_wen <= 4'b0000;
101     end
102 end
103
104 //store 操作的写数据
105 always @ (*) // 对于 SB 指令，需要依据地址底两位，移动 store 的字节至对应位置
106 begin
107     case (dm_addr[1:0])
108         2'b00 : dm_wdata <= store_data;
109         2'b01 : dm_wdata <= {16'd0, store_data[7:0],
110 8'd0};
111         2'b10 : dm_wdata <= {8'd0, store_data[7:0],
112 16'd0};
113         2'b11 : dm_wdata <= {store_data[7:0], 24'd0};
114         default : dm_wdata <= store_data;
115     endcase
116 end
117
118 //load 读出的数据
119 wire load_sign;
120 wire [31:0] load_result;
121 assign load_sign = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7] :
122 (dm_addr[1:0]==2'd1) ? dm_rdata[15] :
123 (dm_addr[1:0]==2'd2) ? dm_rdata[23] :
124 dm_rdata[31] ;
125 assign load_result[7:0] = (dm_addr[1:0]==2'd0) ? dm_rdata[ 7:0 ] :
126 (dm_addr[1:0]==2'd1) ? dm_rdata[15:8 ] :
127 (dm_addr[1:0]==2'd2) ? dm_rdata[23:16] :
128 dm_rdata[31:24] ;
129 assign load_result[31:8]= ls_word ? dm_rdata[31:8] :{24{lb_sign&load_sign}};
130 //-----{load/store 访存}end
131
132 //-----{MEM 执行完成}begin
133 //由于数据 RAM 为同步读写的，
134 //故对 load 指令，取数据时，有一拍延时
135 //即发地址的下一拍时钟才能得到 load 的数据
136 //故 mem 在进行 load 操作时有需要两拍时间才能取到数据
137 //而对其他操作，则只需要一拍时间
138 reg MEM_valid_r;
139 always @(posedge clk)
140 begin

```

```

138         if (MEM_allow_in)
139         begin
140             MEM_valid_r <= 1'b0;
141         end
142         else
143         begin
144             MEM_valid_r <= MEM_valid;
145         end
146     end
147     assign MEM_over = inst_load ? MEM_valid_r : MEM_valid;
148     //如果数据 ram 为异步读的, 则 MEM_valid 即是 MEM_over 信号,
149     //即 load 一拍完成
150 //-----{MEM 执行完成}end
151
152 //-----{MEM 模块的 dest 值}begin
153     //只有在 MEM 模块有效时, 其写回目的寄存器号才有意义
154     assign MEM_wdest = rf_wdest & {5{MEM_valid}};
155 //-----{MEM 模块的 dest 值}end
156
157 //-----{MEM->WB 总线}begin
158     wire [31:0] mem_result; //MEM 传到 WB 的 result 为 load 结果或 EXE 结果
159     assign mem_result = inst_load ? load_result : exe_result;
160
161     assign MEM_WB_bus = {rf_wen,rf_wdest,                //WB 需要使用的信号
162                         mem_result,                    //最终要写回寄存器的数据
163                         lo_result,                    //乘法低 32 位结果, 新增
164                         hi_write,lo_write,            //HI/LO 写使能, 新增
165                         mfhi,mflo,                    //WB 需要使用的信号, 新增
166                         mtc0,mfc0,cp0r_addr,syscall,eret, //WB 需要使用的信号, 新增
167                         pc};                          // PC 值
168 //-----{MEM->WB 总线}begin
169
170 //-----{展示 MEM 模块的 PC 值}begin
171     assign MEM_pc = pc;
172 //-----{展示 MEM 模块的 PC 值}end
173 endmodule

```

wb.v 源代码:

```

1  `timescale 1ns / 1ps
2  //*****
3  //    > 文件名: wb.v
4  //    > 描述   :五级流水 CPU 的写回模块
5  //    > 作者   : LOONGSON
6  //    > 日期   : 2016-04-14
7  //*****
8  `define EXC_ENTER_ADDR 32'd0    // Exception 入口地址,
9                                   // 此处实现的 Exception 只有 SYSCALL
10 module wb(                       // 写回级
11     input          WB_valid,      // 写回级有效
12     input  [117:0] MEM_WB_bus_r,  // MEM->WB 总线

```

```

13     output            rf_wen,           // 寄存器写使能
14     output [ 4:0] rf_wdest,           // 寄存器写地址
15     output [ 31:0] rf_wdata,          // 寄存器写数据
16     output            WB_over,        // WB 模块执行完成
17
18     //5 级流水新增接口
19     input             clk,            // 时钟
20     input             resetn,         // 复位信号，低电平有效
21     output [ 32:0] exc_bus,           // Exception pc 总线
22     output [ 4:0] WB_wdest,          // WB 级要写回寄存器堆的目标地址号
23     output            cancel,         //syscall 和 eret 到达写回级时会发出 cancel 信
号,
24                                     // 取消已经取出的正在其他流水级执行的指令
25
26     //展示 PC 和 HI/LO 值
27     output [ 31:0] WB_pc,
28     output [ 31:0] HI_data,
29     output [ 31:0] LO_data
30 );
31 //-----{MEM->WB 总线}begin
32     //MEM 传来的 result
33     wire [31:0] mem_result;
34     //HI/LO 数据
35     wire [31:0] lo_result;
36     wire        hi_write;
37     wire        lo_write;
38
39     //寄存器堆写使能和写地址
40     wire wen;
41     wire [4:0] wdest;
42
43     //写回需要用到的信息
44     wire mfhi;
45     wire mflo;
46     wire mtc0;
47     wire mfc0;
48     wire [7 :0] cp0r_addr;
49     wire        syscall; //syscall 和 eret 在写回级有特殊的操作
50     wire        eret;
51
52     //pc
53     wire [31:0] pc;
54     assign {wen,
55             wdest,
56             mem_result,
57             lo_result,
58             hi_write,
59             lo_write,
60             mfhi,
61             mflo,
62             mtc0,

```

```
63         mfc0,
64         cp0r_addr,
65         syscall,
66         eret,
67         pc} = MEM_WB_bus_r;
68 //-----{MEM->WB 总线}end
69
70 //-----{HI/LO 寄存器}begin
71     //HI 用于存放乘法结果的高 32 位
72     //LO 用于存放乘法结果的低 32 位
73     reg [31:0] hi;
74     reg [31:0] lo;
75
76     //要写入 HI 的数据存放在 mem_result 里
77     always @(posedge clk)
78     begin
79         if (hi_write)
80         begin
81             hi <= mem_result;
82         end
83     end
84     //要写入 LO 的数据存放在 lo_result 里
85     always @(posedge clk)
86     begin
87         if (lo_write)
88         begin
89             lo <= lo_result;
90         end
91     end
92 //-----{HI/LO 寄存器}end
93
94 //-----{cp0 寄存器}begin
95 // cp0 寄存器即是协处理器 0 寄存器
96 // 由于目前设计的 CPU 并不完备，所用到的 cp0 寄存器也很少
97 // 故暂时只实现 STATUS(12.0), CAUSE(13.0), EPC(14.0) 这三个
98 // 每个 CP0 寄存器都是使用 5 位的 cp0 号
99     wire [31:0] cp0r_status;
100     wire [31:0] cp0r_cause;
101     wire [31:0] cp0r_epc;
102
103     //写使能
104     wire status_wen;
105     //wire cause_wen;
106     wire epc_wen;
107     assign status_wen = mtc0 & (cp0r_addr=={5'd12,3'd0});
108     assign epc_wen     = mtc0 & (cp0r_addr=={5'd14,3'd0});
109
110     //cp0 寄存器读
111     wire [31:0] cp0r_rdata;
112     assign cp0r_rdata = (cp0r_addr=={5'd12,3'd0}) ? cp0r_status :
113                       (cp0r_addr=={5'd13,3'd0}) ? cp0r_cause :
```

```
114         (cp0r_addr=={5'd14,3'd0}) ? cp0r_epc :
115     32'd0;
116     //STATUS 寄存器
117     //目前只实现 STATUS[1]位,即 EXL 域
118     //EXL 域为软件可读写,故需要 statu_wen
119     reg status_exl_r;
120     assign cp0r_status = {30'd0,status_exl_r,1'b0};
121     always @(posedge clk)
122     begin
123         if (!resetn || eret)
124         begin
125             status_exl_r <= 1'b0;
126         end
127         else if (syscall)
128         begin
129             status_exl_r <= 1'b1;
130         end
131         else if (status_wen)
132         begin
133             status_exl_r <= mem_result[1];
134         end
135     end
136
137     //CAUSE 寄存器
138     //目前只实现 CAUSE[6:2]位,即 ExcCode 域,存放 Exception 编码
139     //ExcCode 域为软件只读,不可写,故不需要 cause_wen
140     reg [4:0] cause_exc_code_r;
141     assign cp0r_cause = {25'd0,cause_exc_code_r,2'd0};
142     always @(posedge clk)
143     begin
144         if (syscall)
145         begin
146             cause_exc_code_r <= 5'd8;
147         end
148     end
149
150     //EPC 寄存器
151     //存放产生例外的地址
152     //EPC 整个域为软件可读写的,故需要 epc_wen
153     reg [31:0] epc_r;
154     assign cp0r_epc = epc_r;
155     always @(posedge clk)
156     begin
157         if (syscall)
158         begin
159             epc_r <= pc;
160         end
161         else if (epc_wen)
162         begin
163             epc_r <= mem_result;
```

```

164         end
165     end
166
167     //syscall 和 eret 发出的 cancel 信号
168     assign cancel = (syscall | eret) & WB_over;
169 //-----{cp0 寄存器}begin
170
171 //-----{WB 执行完成}begin
172     //WB 模块所有操作都可在一拍内完成
173     //故 WB_valid 即是 WB_over 信号
174     assign WB_over = WB_valid;
175 //-----{WB 执行完成}end
176
177 //-----{WB->regfile 信号}begin
178     assign rf_wen    = wen & WB_over;
179     assign rf_wdest  = wdest;
180     assign rf_wdata  = mfhi ? hi :
181                       mflo ? lo :
182                       mfc0 ? cp0r_rdata : mem_result;
183 //-----{WB->regfile 信号}end
184
185 //-----{Exception pc 信号}begin
186     wire          exc_valid;
187     wire [31:0]   exc_pc;
188     assign exc_valid = (syscall | eret) & WB_valid;
189     //eret 返回地址为 EPC 寄存器的值
190     //SYSCALL 的 excPC 应该为{EBASE[31:10],10'h180},
191     //但作为实验,先设置 EXC_ENTER_ADDR 为 0,方便测试程序的编写
192     assign exc_pc = syscall ? `EXC_ENTER_ADDR : cp0r_epc;
193
194     assign exc_bus = {exc_valid,exc_pc};
195 //-----{Exception pc 信号}end
196
197 //-----{WB 模块的 dest 值}begin
198     //只有在 WB 模块有效时,其写回目的寄存器号才有意义
199     assign WB_wdest = rf_wdest & {5{WB_valid}};
200 //-----{WB 模块的 dest 值}end
201
202 //-----{展示 WB 模块的 PC 值和 HI/LO 寄存器的值}begin
203     assign WB_pc = pc;
204     assign HI_data = hi;
205     assign LO_data = lo;
206 //-----{展示 WB 模块的 PC 值和 HI/LO 寄存器的值}end
207 endmodule

```

静态 5 级流水线 CPU 参考设计使用的 inst_rom 和 data_ram 均为实例化的同步存储器 IP。另外,参考设计中还使用的实验二、三、四里实现的 multiply.v、regfile.v 和 alu.v,源代码见对应实验的参考设计说明,本实验中直接通过“Add Copy of Source”添加到工程中即可。其中 alu.v 还调用了实验一中的 adder.v。

约束文件 pipeline_cpu.xdc 源代码:

```
1      #时钟信号连接
2      set_property PACKAGE_PIN AC19 [get_ports clk]
3
4      #脉冲开关，用于输入作为复位信号，低电平有效
5      set_property PACKAGE_PIN Y3 [get_ports resetn]
6
7      #脉冲开关，用于输入作为单步执行的 clk
8      set_property PACKAGE_PIN Y5 [get_ports btn_clk]
9
10     set_property IOSTANDARD LVCMOS33 [get_ports clk]
11     set_property IOSTANDARD LVCMOS33 [get_ports resetn]
12     set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
13
14     #触摸屏引脚连接
15     set_property PACKAGE_PIN J25 [get_ports lcd_rst]
16     set_property PACKAGE_PIN H18 [get_ports lcd_cs]
17     set_property PACKAGE_PIN K16 [get_ports lcd_rs]
18     set_property PACKAGE_PIN L8 [get_ports lcd_wr]
19     set_property PACKAGE_PIN K8 [get_ports lcd_rd]
20     set_property PACKAGE_PIN J15 [get_ports lcd_bl_ctr]
21     set_property PACKAGE_PIN H9 [get_ports {lcd_data_io[0]}]
22     set_property PACKAGE_PIN K17 [get_ports {lcd_data_io[1]}]
23     set_property PACKAGE_PIN J20 [get_ports {lcd_data_io[2]}]
24     set_property PACKAGE_PIN M17 [get_ports {lcd_data_io[3]}]
25     set_property PACKAGE_PIN L17 [get_ports {lcd_data_io[4]}]
26     set_property PACKAGE_PIN L18 [get_ports {lcd_data_io[5]}]
27     set_property PACKAGE_PIN L15 [get_ports {lcd_data_io[6]}]
28     set_property PACKAGE_PIN M15 [get_ports {lcd_data_io[7]}]
29     set_property PACKAGE_PIN M16 [get_ports {lcd_data_io[8]}]
30     set_property PACKAGE_PIN L14 [get_ports {lcd_data_io[9]}]
31     set_property PACKAGE_PIN M14 [get_ports {lcd_data_io[10]}]
32     set_property PACKAGE_PIN F22 [get_ports {lcd_data_io[11]}]
33     set_property PACKAGE_PIN G22 [get_ports {lcd_data_io[12]}]
34     set_property PACKAGE_PIN G21 [get_ports {lcd_data_io[13]}]
35     set_property PACKAGE_PIN H24 [get_ports {lcd_data_io[14]}]
36     set_property PACKAGE_PIN J16 [get_ports {lcd_data_io[15]}]
37     set_property PACKAGE_PIN L19 [get_ports ct_int]
38     set_property PACKAGE_PIN J24 [get_ports ct_sda]
39     set_property PACKAGE_PIN H21 [get_ports ct_scl]
40     set_property PACKAGE_PIN G24 [get_ports ct_rstn]
41
42     set_property IOSTANDARD LVCMOS33 [get_ports lcd_rst]
43     set_property IOSTANDARD LVCMOS33 [get_ports lcd_cs]
44     set_property IOSTANDARD LVCMOS33 [get_ports lcd_rs]
45     set_property IOSTANDARD LVCMOS33 [get_ports lcd_wr]
46     set_property IOSTANDARD LVCMOS33 [get_ports lcd_rd]
47     set_property IOSTANDARD LVCMOS33 [get_ports lcd_bl_ctr]
48     set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[0]}]
```

```
49 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[1]}]
50 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[2]}]
51 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[3]}]
52 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[4]}]
53 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[5]}]
54 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[6]}]
55 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[7]}]
56 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[8]}]
57 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[9]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[10]}]
59 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[11]}]
60 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[12]}]
61 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[13]}]
62 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[14]}]
63 set_property IOSTANDARD LVCMOS33 [get_ports {lcd_data_io[15]}]
64 set_property IOSTANDARD LVCMOS33 [get_ports ct_int]
65 set_property IOSTANDARD LVCMOS33 [get_ports ct_sda]
66 set_property IOSTANDARD LVCMOS33 [get_ports ct_scl]
67 set_property IOSTANDARD LVCMOS33 [get_ports ct_rstn]
68
69 set_property IOSTANDARD LVCMOS33 [get_ports btn_clk]
```

九、 课程设计拓展 优化 CPU 系统

1 课程设计拓展目的

1. 加深对计算机组成原理和体系结构理论知识的理解。
2. 培养对 CPU 设计的兴趣，在理解现有 CPU 架构的基础上，引发对体系结构的思考和创新。
3. 培养创新思维能力，并通过实践验证新想法。

2 实验设备

1. 装有 xilinx ISE 的计算机一台。
2. LS-CPU-EXB-002 教学系统实验箱一套。

3 课程设计拓展题说明

本拓展是对之前课程设计的拔高，是为大家在学有余力的时候设定的，旨在发挥大家的创新思维能力，为大家提供一个学习反思验证的环境，可以尽情地提出自己关于计算机组成原理和体系结构的想法，大胆地对书中的知识提出疑问，通过设计实验方案验证自己的想法，从而获得对课本知识的升华。

因此本拓展会提出几个拓展题，供大家思考，大家可以选择性地实现，也可以全部实现，当然更鼓励大家地实现自己的新想法。为充分发挥大家的主观能动性，也为了不限制大家的思维，各拓展题只会给出现有问题，并不会像以往的实验说明给出初步的实验方案。

4 课程设计拓展题

1. 分析静态 5 级流水 CPU 中的流水线阻塞情况，包括数据相关、控制相关、结构相关等，优化流水线设计，尽可能减少流水线阻塞情况，比如前递技术等。
2. 对于分支跳转指令，mips 架构中有延迟槽指令的设定，利用这一点，在静态 5 级流水 CPU 中，可实现分支指令永远不阻塞后续指令，大家可以检查自己的流水线设定，进行优化实现这一点。
3. 针对第 2 点，此时，分支跳转指令就不需要进行转移猜测了，但大家可以将流水线结构改为 x86 中无延迟槽技术的设定，此时分支跳转指令与后续真正需要执行的指令至少会堵塞一拍，此时可以考虑实现转移预测技术，提升流水线结构。
4. 在学习的过程中，大家一定会有很多自己的想法，比如，为什么取指、译码、执行、访存、写回称为经典的 5 级流水结构，可以实现 3 级、4 级、6 级流水结构的 CPU 吗？答案肯定是可以的，甚至在各类产品的 CPU 中采用经典 5 级流水结构的都很少。所以希望大家尽情地发挥自己的想法，大改流水结构，验证自己关于流水结构的想法，您可以实现 3 级（比如：取指、译码、执行）、4 级、6 级等等各类流水结构的 CPU。

5. 课程设计要求大家实现的 mips 指令有限，大家可以分析其余 mips 指令，加以实现。

6. 目前课程设计实现的指令存储器和数据存储器是同步读的机制的，故在当前拍数（时钟周期）发出读数据的地址请求时，在下一拍才能获得读的数据，因此取值级和访存级的 load 都需要两拍时间。其实发地址请求在下一拍获得读的数据，明显也是一个可以流水做的工作，故可以考虑对流水线设计方案作稍微修改，使得取指级和访存级的 load 不需要多等一拍。

5 课程设计拓展要求

拓展题无特别要求，如果想做的话，需要自己花较多心思，在充分理解的基础上加以思考。在确定自己准备实现的想法后，请依然按照以往实验要求，画出设计框图，给出相应设计方案和验证方案。实现完后向老师充分阐述自己的改动，并进行演示。建议最后也形成一份实验报告的文档。文档还是很重要的，要养成一个好习惯。

附录 A 实现的 MIPS 指令集

附 A1 单周期 CPU 实现指令

(1) 无符号加法

ADDU rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100001						
6	5	5	5	5	6						

$GPR[rd] \leftarrow GPR[rs] + GPR[rt]$

(2) 无符号减法

SUBU rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100011						
6	5	5	5	5	6						

$GPR[rd] \leftarrow GPR[rs] - GPR[rt]$

(3) 有符号比较, 小于置位

SLT rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	101010						
6	5	5	5	5	6						

$GPR[rd] \leftarrow (\text{sign}(GPR[rs]) < \text{sign}(GPR[rt]))$

(4) 按位与

AND rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100100						
6	5	5	5	5	6						

$GPR[rd] \leftarrow GPR[rs] \& GPR[rt]$

(5) 按位或非

NOR rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100111						
6	5	5	5	5	6						

$GPR[rd] \leftarrow \sim(GPR[rs] | GPR[rt])$

(6) 按位或

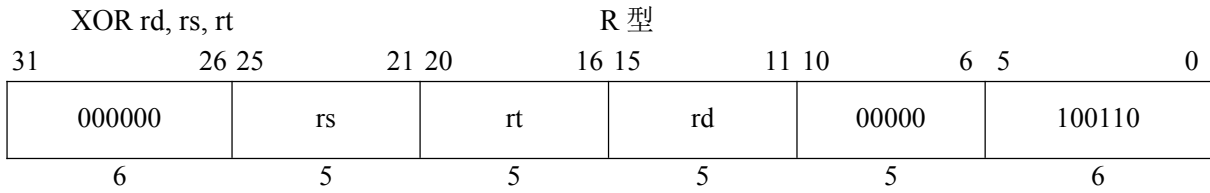
OR rd, rs, rt

R 型

31	26	25	21	20	16	15	11	10	6	5	0
000000	rs	rt	rd	00000	100101						
6	5	5	5	5	6						

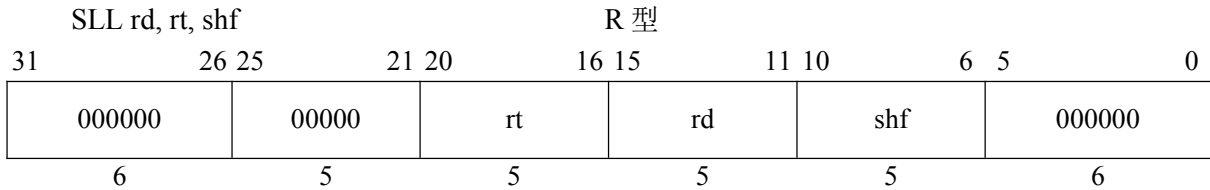
$GPR[rd] \leftarrow GPR[rs] | GPR[rt]$

(7) 按位异或



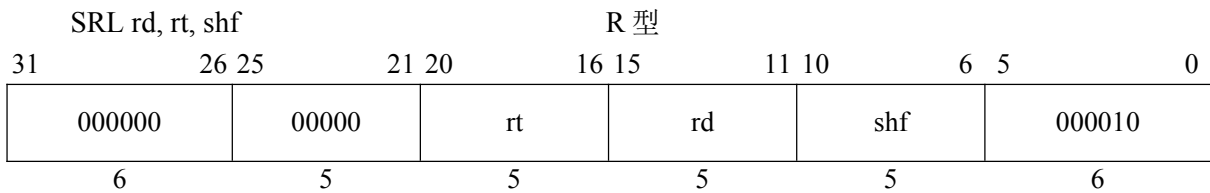
$GPR[rd] \leftarrow GPR[rs] \wedge GPR[rt]$

(8) 逻辑左移



$GPR[rd] \leftarrow \text{zero}(GPR[rt]) \ll \text{shf}$

(9) 逻辑右移



$GPR[rd] \leftarrow \text{zero}(GPR[rt]) \gg \text{shf}$

(10) 立即数、无符号加法



$GPR[rt] \leftarrow GPR[rs] + \text{sign_ext}(\text{imm})$

(11) 相等跳转



if $GPR[rs] = GPR[rt]$ then $PC \leftarrow B_PC + \text{sign_ext}(\text{offset}) \ll 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(12) 不等跳转



if $GPR[rs] \neq GPR[rt]$ then $PC \leftarrow B_PC + \text{sign_ext}(\text{offset}) \ll 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(13) 装载字

LW rt, offset(base) I 型

31	26 25	21 20	16 15	0
100011	base	rt	offset	
6	5	5	16	

$GPR[rt] \leftarrow Mem[GPR[base] + sign_ext(offset)]$

(14) 存储字

SW rt, offset(base) I 型

31	26 25	21 20	16 15	0
101011	base	rt	offset	
6	5	5	16	

$Mem[GPR[base] + sign_ext(offset)] \leftarrow GPR[rt]$

(15) 立即数装载高位

LUI rt, imm I 型

31	26 25	21 20	16 15	0
001111	00000	rt	imm	
6	5	5	16	

$GPR[rt] \leftarrow \{imm, 16'd0\}$

(16) 直接跳转

J target J 型

31	26 25	0
000010	target	
6	26	

$PC \leftarrow \{B_PC[31:28], target \ll 2\}$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

附 A2 多周期 CPU 新增实现指令

(17) 无符号小于置位

SLTU rd, rs, rt R 型

31	26 25	21 20	16 15	11 10	6 5	0
000000	rs	rt	rd	00000	101011	
6	5	5	5	5	6	

$GPR[rd] \leftarrow (zero(GPR[rs]) < zero(GPR[rt]))$

(18) 跳转寄存器并链接

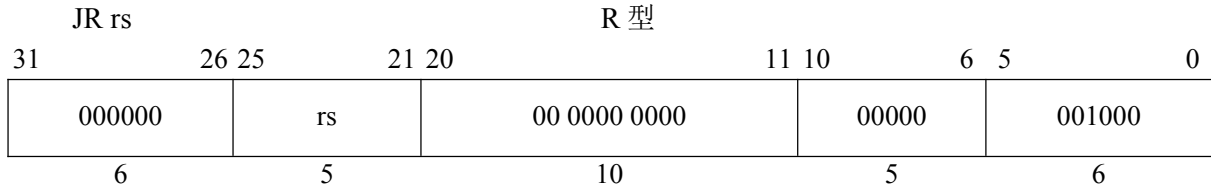
JALR rs R 型

31	26 25	21 20	16 15	11 10	6 5	0
000000	rs	00000	11111	00000	001001	
6	5	5	5	5	6	

$GPR[31] \leftarrow B_PC + 4, PC \leftarrow GPR[rs]$

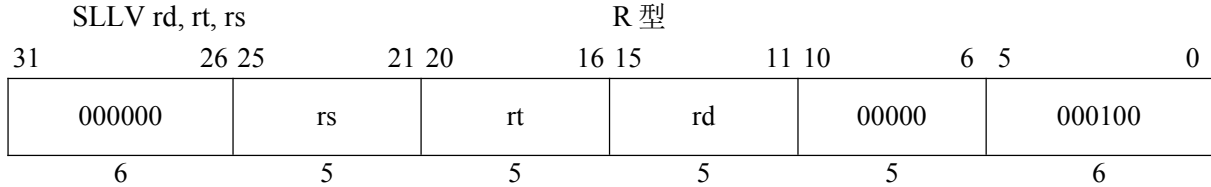
B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(19) 跳转寄存器



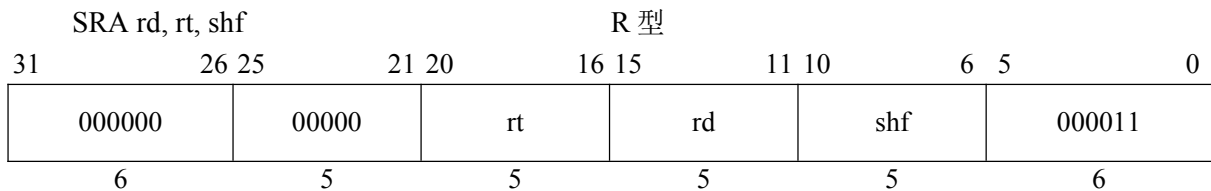
$PC \leftarrow GPR[rs]$

(20) 变量逻辑左移



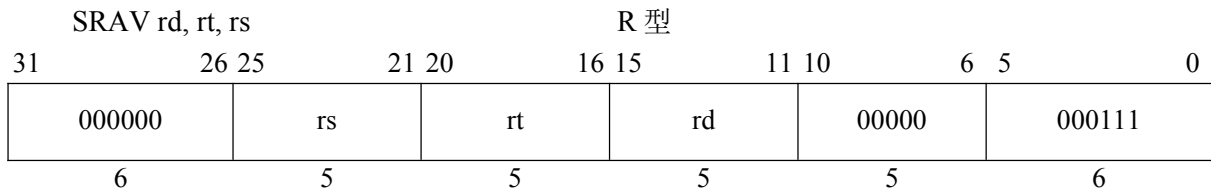
$GPR[rd] \leftarrow zero(GPR[rt]) \ll GPR[rs]$

(21) 算术右移



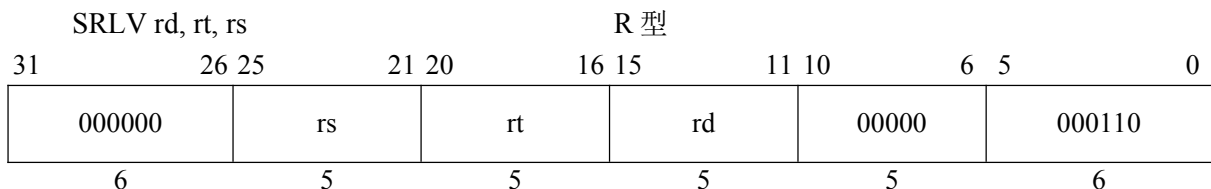
$GPR[rd] \leftarrow sign(GPR[rt]) \gg shf$

(22) 变量算术右移



$GPR[rd] \leftarrow sign(GPR[rt]) \gg GPR[rs]$

(23) 变量逻辑右移



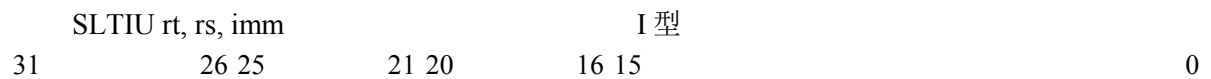
$GPR[rd] \leftarrow zero(GPR[rt]) \gg GPR[rs]$

(24) 立即数有符号比较, 小于置位



$GPR[rt] \leftarrow (sign(GPR[rs]) < sign_ext(imm))$

(25) 立即数无符号比较, 小于置位



001011	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow (zero(GPR[rs]) < sign_ext(imm))$

(26) 大于或等于零跳转

BGEZ rs, offset I 型

31	26	25	21	20	16	15	0
000001	rs	00001	offset				
6	5	5	16				

if $GPR[rs] \geq 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(27) 大于零跳转

BGTZ rs, offset I 型

31	26	25	21	20	16	15	0
000111	rs	00000	offset				
6	5	5	16				

if $GPR[rs] > 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(28) 小于或等于零跳转

BLEZ rs, offset I 型

31	26	25	21	20	16	15	0
000110	rs	00000	offset				
6	5	5	16				

if $GPR[rs] \leq 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(29) 小于零跳转

BLTZ rs, offset I 型

31	26	25	21	20	16	15	0
000001	rs	00000	offset				
6	5	5	16				

if $GPR[rs] < 0$ then $PC \leftarrow B_PC + sign_ext(offset) < 2$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

(30) 装载字节, 并作符号扩展

LB rt, offset(base) I 型

31	26	25	21	20	16	15	0
----	----	----	----	----	----	----	---

100000	base	rt	offset
6	5	5	16

$GPR[rt] \leftarrow \text{sign}(\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})])$

(31) 装载字节，并作无符号扩展

LBU rt, offset(base) I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

100100	base	rt	offset
6	5	5	16

$GPR[rt] \leftarrow \text{zero}(\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})])$

(32) 存储字节

SB rt, offset(base) I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

101000	base	rt	offset
6	5	5	16

$\text{Mem}[GPR[\text{base}] + \text{sign_ext}(\text{offset})] \leftarrow GPR[rt]$

(33) 立即数按位与

ANDI rt, rs, imm I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

001100	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow GPR[rs] \& \text{zero_ext}(\text{imm})$

(34) 立即数按位或

ORI rt, rs, imm I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

001101	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow GPR[rs] | \text{zero_ext}(\text{imm})$

(35) 立即数按位异或

XORI rt, rs, imm I 型

31	26 25	21 20	16 15	0
----	-------	-------	-------	---

001110	rs	rt	imm
6	5	5	16

$GPR[rt] \leftarrow GPR[rs] \wedge \text{zero_ext}(\text{imm})$

(36) 跳转和链接

JAL target J 型

31	26 25			0
----	-------	--	--	---

000011	target
6	26

$GPR[31] \leftarrow B_PC + 4, PC \leftarrow \{B_PC[31:28], target \ll 2\}$

B_PC: 分支跳转参与运算的 PC, 在不考虑延迟槽时为分支跳转指令的 PC, 考虑延迟槽时为延迟槽指令的 PC, 即分支跳转指令的 PC+4。

附 A3 静态 5 级流水 CPU 新增实现指令

(37) 有符号字乘法

MULT rs, rt		R 型			
31	26 25	21 20	16 15	6 5	0
000000	rs	rt	00 0000 0000	011000	
6	5	5	10	6	

$(HI, LO) \leftarrow sign(GPR[rs]) * sign(GPR[rt])$

(38) 从 LO 寄存器取值

MFLO rd			R 型			
31	26 25	21 20	16 15	11 10	6 5	0
000000		00 0000 0000		rd	00000	010010
6		10		5	5	6

$GPR[rd] \leftarrow [LO]$

(39) 从 HI 寄存器取值

MFHI rd		R 型				
31	26 25	21 20	16 15	11 10	6 5	0
000000		00 0000 0000		rd	00000	010000
6		10		5	5	6

$GPR[rd] \leftarrow [HI]$

(40) 向 LO 寄存器存值

MTLO rs		R 型			
31	26 25	21 20	16 15	6 5	0
000000		rs	000 0000 0000 0000		010011
6		5	15		6

$[LO] \leftarrow GPR[rs]$

(41) 向 HI 寄存器存值

MTHI rs		R 型			
31	26 25	21 20	16 15	6 5	0
000000		rs	000 0000 0000 0000		010001
6		5	15		6

$[HI] \leftarrow GPR[rs]$

(42) 从协处理器 0 寄存器取值

MFC0 rt, cs.sel			R 型		
31	26 25	21 20	16 15	11 10	3 2 0

010000	00000	rt	cs	00000000	sel
6	5	5	5	8	3

$GPR[rt] \leftarrow CPR[cs.sel]$

(43) 向协处理器 0 寄存器存值

MTC0 rt, cd.sel

R 型

31	26	25	21	20	16	15	11	10	3	2	0
010000	00100	rt	cd	00000000	sel						
6	5	5	5	8	3						

$CPR[cd.sel] \leftarrow GPR[rt]$

(44) 系统调用

SYSCALL

31	26	25	6	5	0
000000	code	001100			
6	20	6			

$CPR[14.0] \leftarrow PC$, $CPR[13.0][6:2] \leftarrow 01000$, $CPR[12.0][1] \leftarrow 1$, $PC \leftarrow EXC_ENTER_ADDR$

EXC_ENTER_ADDR 为例外入口地址，原本应为 $CPR[15.1] + 0x180$ ，但在课程设计中为方便编写测试程序，将 EXC_ENTER_ADDR 设置为 0。

(45) 异常返回

ERET

31	26	25	21	20	16	15	11	10	6	5	0
010000	1	000 0000 0000 0000 0000	011000								
6	1	19	6								

$CPR[12.0][1] \leftarrow 0$, $PC \leftarrow CPR[14.0]$

附 B3 例外程序计数器 EPC（CP0 寄存器 14，选择 0）

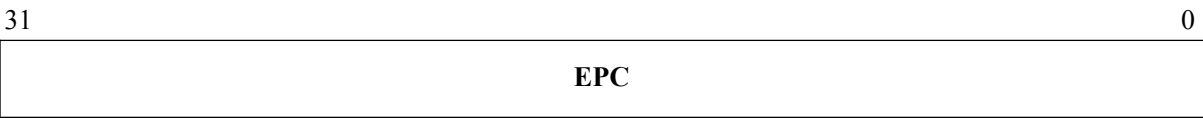


表 B-5 EPC（Exception Program Counter）域

域		描述	读/写	复位状态	规则
Name	Bits				
EPC	31..0	例外程序计数器	可读/写	Undefined	Required